

Technische Universität Berlin
Fakultät IV Elektrotechnik und Informatik

**Temporal Multi-Robot Task and
Motion Planning for Mechanical
Disassembly Tasks**

Master Thesis

Niklas Hargus
Matriculation Number: 486828

Supervisor:
Dr. Andreas Orthey

Examiners:
Prof. Dr. Wolfgang Hönig
Prof. Dr. Marc Toussaint

Eigenständigkeitserklärung

Hiermit versichere ich, dass ich die vorliegende Arbeit ohne Hilfe Dritter und ausschließlich unter Verwendung der aufgeführten Quellen und Hilfsmittel angefertigt habe. Alle Stellen, die den benutzten Quellen und Hilfsmitteln unverändert oder inhaltlich entnommen sind, habe ich als solche kenntlich gemacht.

Sofern generative KI-Tools verwendet wurden, habe ich Produktnamen, Hersteller, die jeweils verwendete Softwareversion und die jeweiligen Einsatzzwecke (z.B. sprachliche Überprüfung und Verbesserung der Texte, systematische Recherche) benannt. Ich verantworte die Auswahl, die Übernahme und sämtliche Ergebnisse des von mir verwendeten KI-generierten Outputs vollumfänglich selbst.

Die Satzung zur Sicherung guter wissenschaftlicher Praxis an der TU Berlin vom 15. Februar 2023 ¹ habe ich zur Kenntnis genommen.

Ich erkläre weiterhin, dass ich die Arbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegt habe.

Berlin, den 28.05.2026


.....
Niklas Hargus

¹https://www.static.tu.berlin/fileadmin/www/10002457/K3-AMBI/Amtsblatt_2023/Amtliches_Mitteilungsblatt_Nr._16_vom_30.05.2023.pdf

Declaration of Independence

I hereby certify that I have completed this thesis without the assistance of third parties and exclusively using the sources and resources listed. I have clearly identified all passages that are taken verbatim or adapted from the sources and resources used.

Where generative AI tools were used, I have specified the product names, manufacturers, the software versions used in each case, and the respective purposes (e.g., linguistic review and improvement of the texts, systematic research). I take full responsibility for the selection, the incorporation, and all results of the AI-generated output I have used.

I have taken note of the Charter on Ensuring Good Scientific Practice at TU Berlin dated February 15, 2023 ².

I further declare that I have not yet submitted this work in the same or a similar form to any other examination authority.

Berlin, May 28, 2026


.....
Niklas Hargus

²https://www.static.tu.berlin/fileadmin/www/10002457/K3-AMBI/Amtsblatt_2023/Amtliches_Mitteilungsblatt_Nr._16_vom_30.05.2023.pdf

Zusammenfassung

Das Recycling elektro-mechanischer Baugruppen stellt einen zentralen Engpass beim Übergang zu einer Kreislaufwirtschaft dar. Um das Recycling in großem Maßstab zu ermöglichen, ist die universelle Multi-Roboter-Demontage eine Schlüsseltechnologie. Allerdings müssen dabei eine große Anzahl an Robotern in beengten Arbeitsräumen während des Demontageprozesses gleichzeitig agieren. Dies stellt ein bislang weitgehend unerforschtes Problem dar. Diese Arbeit schließt diese Lücke, indem sie den Demontageprozess als langfristiges Problem der Aufgaben- und Bewegungsplanung mit mehreren Robotern formuliert. Das Problem besteht aus einer Gruppe von Robotern und einer Baugruppe, die durch einen Präzedenzgraphen und die Zielpositionen definiert ist. Die Lösung des Problems ist eine Reihe von Aufgaben, die den Demontageprozess beschreiben und dabei die gesamte Ausführungszeit minimieren. Dieses Problem wird mittels einer prioritätsbasierten sequenziellen Planungsmethodik angegangen, die das Problem in einzelne Aufgaben zerlegt, die aus primitiven Abläufen wie Greifen, Ablegen und Bewegungsbefehlen bestehen. Die Implementierung des Vorgehens kombiniert Space-Time Rapidly Exploring Random Trees (ST-RRT*) mit einer Weiterleitung von zeitlichen Einschränkungen. Dies ermöglicht eine zeitliche Abstimmung von unterschiedlichen Aufgaben bei unbekannter Bearbeitungsdauer. Zusätzlich wird ein teilweises Zurücksetzen bei nicht lösaren Teilproblemen ermöglicht. Der Ansatz wurde anhand von sechs Demontageszenarien bewertet, die von rein sequenziellen bis hin zu hochgradig parallelisierbaren Problemen reichen. In den Szenarien erreicht ST-RRT* die gleiche Bearbeitungszeit wie Rapidly Exploring Random Trees (RRT*), jedoch mit einer Erfolgsquote von 100% im Vergleich zu 40% bei RRT*. Die Auswertung zeigt, dass das Vorgehen mithilfe der zeitlichen Einschränkungen die Bewegungsplanung bei verschiedensten Demontageproblemen effizient im Raum-Zeit-Zustandsraum lösen kann. Dies führt zu einer sechsfachen Beschleunigung der Bearbeitungszeit bei nicht trivialen Szenarien im Vergleich zu einem Roboter. Darüber hinaus wurden verbleibende Hindernisse für den Demontageprozess identifiziert, die bei zukünftigen Forschungsarbeiten näher untersucht werden müssen. Dieser Ansatz stellt eine wichtige Grundlage für die Automatisierung des Recyclingprozesses in einer Kreislaufwirtschaft dar.

Abstract

Recycling of electromechanical objects is a key bottleneck in the transition to a circular economy. To recycle at scale, general-purpose, multi-robot disassembly is key enabler technology. However, robots need to operate in confined workspaces with a large number of robots involved in the disassembly process, making this a challenging problem, which remains largely unexplored. This thesis addresses this gap by framing the disassembly process as a long-horizon, multi-robot, task and motion planning problem. The input to this problem comprises a set of robots and an object assembly specified by its precedence graph and goal locations. The output is a set of tasks that coordinate the disassembly process in order to minimize the makespan, the total execution time of the solution. This problem is approached using a prioritised sequential planning framework that decomposes the problem into high-level tasks comprised of primitive tasks such as picking, placing, and moving. The framework’s implementation combines Space-Time Rapidly Exploring Random Trees (ST-RRT*) with a temporal constraint propagation mechanism to enable temporal coordination with unknown arrival times. This is combined with a rollback mechanism that can recover from infeasible subproblems. Six disassembly scenarios are introduced to evaluate the approach, spanning purely sequential to highly parallelizable structures. On these scenarios, ST-RRT* achieves the same makespan as Rapidly Exploring Random Trees (RRT*), while achieving 100% success rate compared to RRT*’s 40%. The evaluation shows that the proposed framework can solve disassembly problems across a wide range of structural complexities by efficiently exploiting space-time motion planning together with temporal constraint propagation, yielding a sixfold speedup in makespan compared to the one-robot baseline for non-trivial scenarios. Furthermore, it identifies bottlenecks in the disassembly process that require further investigation in future research. Nevertheless, the proposed framework lays an important foundation for automating the recycling process in a circular economy.

Contents

List of Figures	VII
List of Tables	X
List of Algorithms	XI
List of Acronyms	XIII
List of Symbols	XIV
1 Introduction	1
2 Related Work	4
3 Background	10
3.1 Topological Spaces	10
3.2 Kinematics	11
3.2.1 Forward Kinematics	11
3.2.2 Inverse Kinematics	13
3.3 Motion Planning	14
3.3.1 Geometric Motion Planning	14
3.3.2 Sampling-Based Motion Planning	14
4 Approach	21
4.1 Assumptions	21
4.2 High Level Task and Motion Planning	22
4.2.1 Problem Statement	22
4.2.2 Object Dependency	22
4.2.3 Overview	23
4.2.4 Algorithm	24
4.2.5 Completeness and Optimality	25
4.2.6 Temporal Dependency Constraints	27
4.3 Task Representation	29
4.4 Compound Task: Pick and Place	37

4.5	Grasp Pose Generation	41
4.5.1	Surface Sampling	41
4.5.2	Goal State Generation	42
4.6	Setting the Simulation State	43
4.7	Implementation Details	46
4.7.1	Simulation Environment	46
4.7.2	Input File Format	47
4.7.3	Assignment Policy	48
4.7.4	Motion Planning	48
4.7.5	Collision Checking	50
5	Results	52
5.1	Benchmark Setup	52
5.1.1	Benchmark Hardware	54
5.2	Tower Problem	55
5.3	Wall Problem	61
5.4	Crate Problem	67
5.5	Motor Disassembly	73
5.6	Battery Disassembly	80
5.7	Gearbox Disassembly	86
5.8	Failure Analysis	93
5.9	Comparison Against RRT*	99
6	Discussion	102
6.1	Results	102
6.2	Limitations and Alternative Approaches	104
6.3	Insights	106
7	Conclusion	108
	Bibliography	110
A	Disassembly Sequences	117

List of Figures

1.1 Drive assembly with its partial dependency graph	2
4.1 Example of a dependency graph	23
4.2 Screw regrasp problem for orientation	26
4.3 Task ordering problem	26
4.4 Example timeline for no time constraints	27
4.5 Example timeline for time constraint on the pick time	28
4.6 Example timeline for time constraint on the place time	28
4.7 Example timeline for time constraint on pick and place time	28
4.8 Task queue visualization	29
4.9 Timeline illustration for future collision check	32
4.10 Storage representation of robot paths	43
4.11 Storage representation of attachment data	43
4.12 Franka Emika Panda with SE2 base in DART simulation	46
4.13 Visualization of exit regions	49
5.1 Tower scenario and its dependency structure	55
5.2 Execution and planning time comparison for the tower scenario	56
5.3 Speedup factor and calculation time increase for the tower scenario	57
5.4 Pareto front of the makespan and calculation time for the tower scenario	57
5.5 Planner success rate for the tower scenario	58
5.6 Example timeline of tower scenario with ten robots	59
5.7 Wall scenario in simulation	61
5.8 Wall dependency graph	61
5.9 Makespan and calculation time for the wall scenario	62
5.10 Execution speedup and calculation time increase for the wall scenario	63
5.11 Pareto front for the wall scenario	63
5.12 Success rate and cost decay for the wall scenario	64
5.13 Example timeline for the wall scenario	65
5.14 Crate scenario in simulation	67
5.15 Dependency graph for the crate scenario	67
5.16 Makespan and calculation time for the crate scenario	68

5.17 Execution speedup and calculation time increase for the crate	69
5.18 Pareto front for the crate scenario	69
5.19 Success rate and cost decay for the crate scenario	70
5.20 Example timeline for the crate scenario	71
5.21 Motor scenario in simulation	73
5.22 Motor dependency graph	73
5.23 Makespan and calculation time for the motor scenario	74
5.24 Execution speedup and calculation time increase for the motor scenario	75
5.25 Pareto front for the motor scenario	76
5.26 Success rate and cost decay for the motor scenario	77
5.27 Example timeline for the motor scenario	78
5.28 Battery simulation without top housing in simulation	80
5.29 Battery dependency graph	80
5.30 Makespan and calculation time for the battery scenario	81
5.31 Makespan speedup and calculation time increase for the battery scenario	82
5.32 Pareto front for the battery scenario	82
5.33 Success rate and cost decay for the battery scenario	83
5.34 Example timeline for the battery scenario	84
5.35 Gearbox example with and without casing in simulation	86
5.36 Gearbox dependency graph	87
5.37 Makespan and calculation time for the gearbox scenario	88
5.38 Execution speedup and calculation time increase for the gearbox scenario	89
5.39 Pareto front for the gearbox scenario	89
5.40 Success rate and cost decay for the gearbox scenario	90
5.41 Example timeline for the gearbox scenario	91
5.42 Failure reasons for the tower scenario	94
5.43 Failure reasons for the crate scenario	95
5.44 Failure reasons for the motor scenario	96
5.45 Failure reasons for the battery scenario	97
5.46 Makespan and calculation time comparison between ST-RRT* and RRT*	100
5.47 Success rate comparison of ST-RRT* and RRT*	100
A.1 Tower disassembly sequence with ten robots	118
A.2 Wall disassembly sequence ten robots	119
A.3 Crate disassembly sequence with ten robots	120
A.4 Motor disassembly sequence with six robots	121
A.5 Battery disassembly sequence with six robots	122

A.6 Gearbox disassembly sequence with nine robots	123
---	-----

List of Tables

5.1	Object composition of the tower scenario	56
5.2	Object composition of the wall scenario	61
5.3	Object composition of the crate scenario.	67
5.4	Object composition of the motor scenario	74
5.5	Object composition of the battery scenario	81
5.6	Object composition of the gearbox scenario	87

List of Algorithms

1	RRT	16
2	RRT*	17
3	ST-RRT*	19
4	High Level Multi Robot Planning	25
5	Move Task	30
6	Check Future Collision	33
7	Exit Task	34
8	Follow Path	37
9	Pick and Place Task	40
10	Goal State Generation	42
11	Find AttachData	44
12	Set Simulation State	45
13	Collision Check	50
14	Interpolation	51

List of Acronyms

AABB	Axis-Aligned Bounding Box
BIT*	Batch Informed Trees
CAD	Computer-Aided Design
CBS	Conflict-Based Search
CPU	Central Processing Unit
DAG	Directed Acyclic Graph
DART	Dynamic Animation and Robotics Toolkit
DoF	Degrees of Freedom
dRRT, dRRT*	Discrete RRT and its asymptotically optimal variant
DSP	Disassembly Sequence Planning
DSSG	Disassembly Sequence Structure Graph
EE	End-Effector
FK	Forward Kinematics
GA	Genetic Algorithm
IK	Inverse Kinematics
KOMO	k -Order Markov Optimisation
LGP	Logic-Geometric Programming
LQR	Linear Quadratic Regulator
MAPF	Multi-Agent Path Finding
MR-TAMP	Multi-Robot Task and Motion Planning
NLP	Non-Linear Program
OMPL	Open Motion Planning Library
PBS	Priority-Based Search
PDDL	Planning Domain Definition Language
PDDLStream	Stream-based extension of PDDL
PRM, PRM*	Probabilistic Roadmap and its asymptotically optimal variant
PTC	Planner Termination Condition
RRT, RRT*	Rapidly-Exploring Random Trees and its asymptotically optimal variant
SIPP	Safe Interval Path Planning
ST-RRT*	Space-Time RRT*
TAMP	Task and Motion Planning
TB-RRT	Time-Based RRT
URDF	Universal Robot Description Format
YAML	YAML Ain't Markup Language

List of Symbols

Sets and Graphs

V	Set of vertices
E	Set of edges
R	Set of robots
O	Set of objects
$\mathcal{T}$	Set of trajectories
$\mathcal{T}^*$	Optimal solution set of trajectories
$\mathcal{X}_{\text{near}}$	Set of vertices within the rewiring radius of a new sample (RRT*)
$\mathcal{T}_a, \mathcal{T}_b$	Start tree and goal forest in ST-RRT*
X_{goal}	Goal region in space-time
$\mathcal{B}$	Bound variables maintained by ST-RRT*

Topological and Configuration Spaces

$\mathbb{R}$	Real number space
$\mathbb{R}_{\geq 0}$	Non-negative reals
S^1	Unit circle
D^2	Closed unit disk
$\mathcal{Q}$	Configuration space of a single robot
$\mathcal{Q}_i$	Configuration space of robot
$\mathcal{Q}_{\text{free}}$	Collision-free subset of the configuration space
$\mathcal{Q}_{\text{free}}(t)$	Time-dependent free configuration space
$\mathcal{W}$	Workspace
$\mathcal{W}_{\text{free}}$	Free workspace
$\mathcal{X}$	Compound state space
$\mathcal{B}$	Configuration map

Robot Kinematics and Transformations

$\mathbf{q}$	Configuration of a robot
q_l, q_u	Lower and upper joint limits
n	Number of robots
d	Combined dimension state space
d_i	Dimension of robot
k	Number of objects
m	Number of waypoints in a constrained path
$\mathbf{T}$	Homogeneous transformation matrix
$\mathbf{T}_i^j$	Homogeneous transformation from frame i to frame j
$\mathbf{T}_{EE}$	Homogeneous transformation of the end-effector
$\mathbf{T}_{EE}^{obj}$	Relative transform between end-effector and grasped object
$\mathbf{T}_{world}^{obj}$	Pose of an object expressed in world coordinates
$\mathbf{T}_{world}^{EE}$	Pose of the end-effector expressed in world coordinates
$\mathbf{T}_{start}^j, \mathbf{T}_{goal}^j$	Start and goal pose of object o_j
$\mathbf{T}_{des}$	Desired end-effector transformation
$\mathbf{R}$	Rotation matrix
${}^j\mathbf{p}_i$	Translation vector from frame i to frame j
$f_{FK}(\mathbf{q})$	Forward kinematics map
$f(\mathbf{T}_{des})$	Inverse kinematics map
$\mathbf{J}(q)$	Jacobian of the forward kinematics
$\mathbf{J}^\dagger$	Pseudoinverse of the Jacobian
$\dot{\mathbf{x}}, \dot{\mathbf{q}}$	End-effector and joint velocities

Sampling-Based Motion Planning

$G = (V, E)$	Roadmap or tree built by a sampling-based planner
τ	Geometric path
π, π_i	Trajectory (configuration-time pairs) for the system and for robot r_i
$x_{\text{start}}, x_{\text{goal}}$	Start and goal sample of a planning query
x_{init}	Initial sample used to seed the tree
x_{rand}	Randomly drawn sample
x_{near}	Nearest vertex
x_{new}	New sample
x_{min}	Lowest-cost parent
n_{retry}	Maximum number of planner retries per task
η	Step size of the steering function
$r(V)$	Shrinking rewiring radius in RRT*
p_{goal}	Probability of new goal sample
$N_{i,p}(s)$	B-spline basis function
c_i	Control points of a B-spline
A_i	Area of triangular face i
v	Vertex of a triangular face
r	Uniform random variables for barycentric sampling

Time Variables and Tolerances

t	Simulation time
t_{at}	Query time
$t_{\text{start}}, t_{\text{end}}$	Start and end time of a task
$t_{\text{attach}}, t_{\text{detach}}$	Times at which an object is attached to / detached from a robot
$t_{\text{pick}}, t_{\text{place}}$	Pick and place time constraint
t_{exit}	Time at which a robot reaches its exit configuration
t_{max}	Adaptive upper time bound
$t_{\text{timelimit}}$	Maximum computation time
Δt	Discrete time delta
ϵ	Small positive time constant
c	Interpolation coefficient

Optimisation and Cost Functions

$J(q)$	IK cost function
α	Trade-off weight between time progression and joint displacement
λ	IK regularisation weight

1 Introduction

The current economic model of consumption hinges on the extraction of materials, the manufacturing of products and the disposal of obsolete goods in other parts of the world. This is not sustainable long term due to the ever increasing global resource usage and compounding environmental impact [1]. An alternative economic model, the circular economy, focuses on existing products to extend the life cycle of goods [2].

The circular economy relies on key industries such as recycling and upcycling, both of which require products to be disassembled. This process is tedious and time-consuming, making it ideal for automation. However, three major problems arise in widespread adoption of automation. First, the economics of recycling and upcycling are usually more challenging than for general manufacturing, due to difficulties in automation. This is caused by many production steps still requiring manual labour to disassemble objects, resulting in less favourable scaling compared to the production of new goods. Second, many disassembly environments are hazardous to human workers such as lithium-ion battery cells, which can release toxic electrolytes [3][4] or enter thermal runaway [5] when damaged, or electronics, like charging circuits containing heavy metals [6]. Third, the workforce willing to perform repetitive, physically demanding work is declining in industrialized economies [7], while the volume of manufacturing and end-of-life products continues to grow. Automating the disassembly process is therefore one of the most significant bottlenecks towards a circular economy [8].

However, automating the disassembly process remains a highly challenging task. This is because assemblies resemble three-dimensional physical puzzles that have to be taken apart in a particular sequence without causing damage to the individual parts [8][9]. Solving these puzzles consists in determining the order of disassembly and the manipulation sequence required to remove the objects. This leads to long horizon planning problems with complex contact dynamics with interlocking objects, such as screws, clips and connectors.

Using multiple robots can further increase the efficiency of disassembly operations. However, this provides further difficulties. While it can enhance the speed and capabilities of the process, it also increases the complexity of coordination to ensure

safety and effectiveness. Further improvements to the disassembly time can be achieved by parallelization of movements when possible. These problems require substantial coordination and can rarely be solved using brute force.

The aim of this thesis is to develop an algorithm for disassembling structures of arbitrary size using multiple robots. This algorithm takes a set of robots and an assembly with its goal positions as an input.

Since the assembly consists of interlocking mechanical parts, a dependency graph is provided, which specifies the order in which the parts must be removed. Fig. 1.1 shows such an assembly and dependency graph. The sequence in which the structure must be disassembled can be derived from its geometry using different methodologies such as genetic algorithms [10] or physical simulation [11].

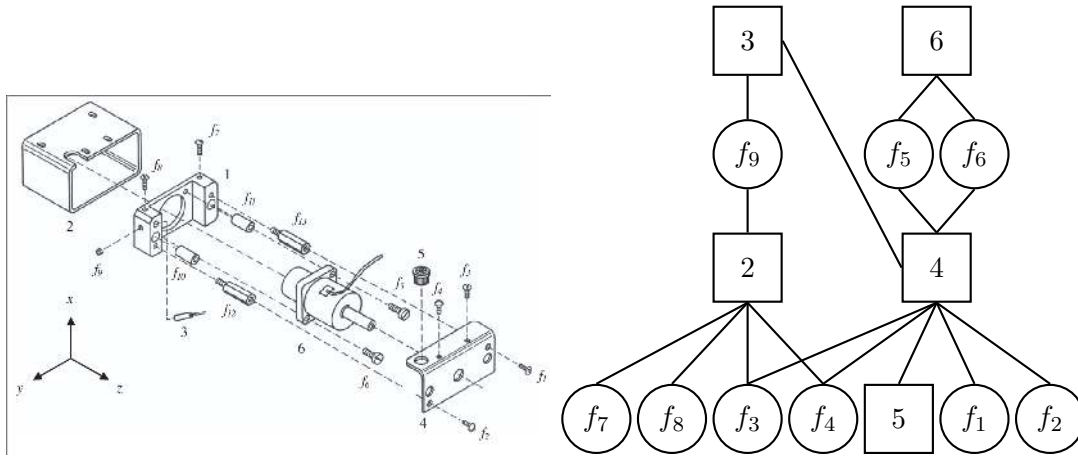


Figure 1.1: Drive assembly [12] with its partial dependency graph [13]

The algorithm's output is a set of tasks that describe the required steps for the disassembly process. In order to generate this sequence of tasks for each robot, the dependency graph is queried for the leaf nodes. These leaf nodes represent objects that need to be extracted next. To extract these objects, all available robots are assigned to a leaf node object, based on an assignment policy. This policy may be a random assignment or be based on a heuristic, such as the Euclidean distance. The robot-object pairings create a pick-and-place task in the robot's task queue. This high-level pick-and-place task is then decomposed into five primitive phases. First, the robot approaches the object. Then, it attaches the object to the end-effector. Next, it moves to the goal location and detaches the object. Finally, the robot leaves the area to avoid obstructing other robots. The task execution uses a prioritized planning approach in order to avoid the dimensionality explosion of a combined state space.

The robots must coordinate with each other to enable sequential manipulation of the objects. This is achieved through the propagation of temporal constraints between nodes in the dependency graph. Each node in the dependency graph can constrain the nodes that depend on it. These time constraints determine when the dependent objects can be picked up or placed down.

In order to evaluate the framework it is implemented on top of the Open Motion Planning Library (OMPL) for sampling-based planners and the Dynamic Animation and Robotics Toolkit (DART) for kinematics and collision checking. ST-RRT* is used as the primary planner in this thesis due to its ability to handle unknown arrival times by dynamically adapting the time bound at runtime. Furthermore, it plans directly in the space-time state space, enabling the already planned robots to be regarded as dynamic obstacles.

The framework is evaluated using six disassembly scenarios. Each scenario is designed to characterize different properties. These scenarios range from deep, sequential manipulation and spatially constrained workspaces to heavily parallelizable scenarios. The scenarios are tested with different numbers of robots to identify how execution time decreases and computation increases at scale. Furthermore, the performance of ST-RRT* is tested in terms of success rate and cost decay and compared to RRT* with a fixed time state space. Finally, a comprehensive failure analysis shows individual performance in the following areas: inverse kinematics generation, moving to the object, moving to the goal with the object, extracting objects and exiting the workspace.

The contributions of this thesis are:

- A prioritized sequential task and motion planning framework for multi-robot disassembly for makespan minimization. The framework uses individual task queues to schedule and balance workloads among available robots. Each task can be composed of primitive base tasks.
- Implementation of the proposed framework with ST-RRT* as the underlying motion planner to extend to unknown arrival times.
- Temporal constraint propagation mechanism to coordinate execution across individual task boundaries
- Evaluation on six disassembly scenarios to identify scaling behaviour, planner performance and remaining bottlenecks

The thesis is organized as follows: First, Ch. 2 reviews the related work and Ch. 3 briefly introduces the necessary background information. Then, the approach is discussed in Ch. 4. The implementation is evaluated in Ch. 5, with the results discussed in Ch. 6. Finally, the thesis concludes in Ch. 7.

2 Related Work

This chapter discusses the related work for this thesis. First, the evolution of sampling-based motion planning is examined, as this forms the basis of the planner used in this work. Then, the discussion extends to multi-robot motion planning, weighing up the pros and cons of the different available approaches. Task and motion planning is summarized to identify how the symbolic representation of tasks interacts with the geometric feasibility of the corresponding motion plan. Next, multi-robot extensions to the task and motion planning problem are reviewed to assess how multi-robot task assignment can be handled. Finally, a review of dependency graph generation for robot disassembly is presented to establish a methodology for the input of the algorithm proposed in this thesis.

Sampling-Based Motion Planning

Sampling-based planning is one of the primary paradigms for high-dimensional robotic path planning. It leverages black-box collision checking on randomly drawn samples to avoid an obstacle representation in configuration space.

Most work can be divided into multi-query graph construction and single-query tree growth. The Probabilistic Roadmap (PRM) of Kavraki et al. [14] established the multi-query graph paradigm by using collision-free samples as vertices and connecting them through local planners. Queries are connected to the graph and solved through graph search.

Rapidly-Exploring Random Trees (RRTs) is the single-query alternative proposed by LaValle [15], which exploits the Voronoi bias to rapidly explore the freespace. This algorithm was already able to handle nonholonomic constraints, dynamics and high degrees of freedom. Early work by Kuffner and LaValle [16] extended this algorithm to RRT-Connect by growing a tree from start and goal simultaneously, connecting the two with a greedy connection heuristic, which leads to lower computation time at the cost of solution cost. All of these planners are probabilistically complete but do not offer any claims on path quality.

Karaman and Frazzoli [17] formally proved the probabilistic completeness of prevalent sampling-based planners and their convergence to sub-optimal solutions. They introduced RRT* and PRM*, the first planners to guarantee asymptotic optimality. This is achieved by rewiring the sampled states with a shrinking radius. However, the planner is slow in practice due to its uninformed sampling across the whole configuration space. This was remedied by Gammell et al. with the introduction of sampling in an admissible ellipsoidal heuristic [18] and later Batch Informed Trees (BIT*) [19], combining graph and tree based approaches.

The sampling based planners can handle differential constraints with kinodynamic planning. First formalized by LaValle and Kuffner [20], the algorithm extends the construction of a RRT by generating the samples based on the dynamics of the systems. Asymptotic optimality for kinodynamic planning was first achieved by Webb and van den Berg [21] with a Linear-Quadratic-Regulator (LQR) steering function for controllable linear systems. Li et al. [22] achieved optimality without a steering function with significant speedups for near optimal solutions with Stable Sparse RRT. Another branch of trajectory planning focuses on dynamic obstacle avoidance with known obstacle trajectories. Hsu et al. [23] introduced the state-time configuration space for car dynamics with dynamic obstacles. Van den Berg and Overmars [24], [25] developed a roadmap-based dynamic planning approach for arrival-time minimization. Phillips et al. [26] introduce safe interval path planning, which represents the time dimension through intervals during which a configuration remains collision-free. The single-query tree alternative for space time planning was first introduced by Sintov and Shapiro [27] with Time-Based RRT (TB-RRT) for rendezvous planning of two dynamic systems. However, it is limited by a fixed arrival time and without any optimality guarantees. Grothe et al.'s [28] extension to RRT* treats time itself as a planning dimension. This allows RRTs to directly plan in space-time to find asymptotically optimal solutions with regards to arrival time. It grows a bidirectional tree from start and finish at an unknown arrival time with progressively widening time horizon.

Multi-Robot Motion Planning

The two major approaches to multi-robot motion planning are separated by how the robots' individual configuration spaces are coupled. The centralized approach combines all configuration spaces together, which increases the complexity of the problem but remains optimal and complete, whereas the decoupled methods remain scalable, sacrificing completeness and optimality.

The discrete problem of Multi-Agent Path Finding (MAPF), stated by Yu and LaValle [29], for time- and distance-optimal MAPF on connected, undirected graphs, is NP-hard. The difficulties of the naive planning problem motivated Sharon et al. [30] to introduce Conflict-Based Search (CBS), a two stage decomposition into a high-level conflict-tree and low-level search in which conflicts are solved. CBS is optimal and complete on a grid-world and has grounded a large family of multi-agent path finders. The graph search approach of Phillips et al. [26] was extended by Andreychuk et al. [31] to multi-agent pathfinding with continuous time. This approach struggles to extend well to the high-dimensional configuration spaces of articulated manipulators.

Sampling-based multi-robot planning works in continuous, high-dimensional configuration spaces. Wagner and Choset [32] introduced M^* , which searches an implicit representation of the combined roadmaps rather than constructing the joined roadmap, resulting in significant speedup. However, it is inefficient for higher dimensions due to the exponential growth of neighbors. This led to Solovey et al. [33] introducing discrete-RRT (dRRT), which extends M^* with an oracle function to pick the best neighbor. Shome et al. [34] contributed the asymptotically optimal extension dRRT*, resulting in convergence to an optimum even in multi-robot manipulator spaces.

Erdmann and Lozano-Pérez [35] introduced prioritized planning to circumvent the complexity increase. They propose to assign an ordering to the robots and plan each one in turns against higher-priority robots, who are treated as dynamic obstacles in time. This approach trades completeness and optimality for gain in tractability.

Bennewitz et al. [36] demonstrated that fixed orderings can fail to find valid solutions, even though they exist. Čáp et al. [37] provides a characterization instance for which a revised prioritized algorithm is complete. Priority-Based Search (PBS) by Ma et al. [38] bridges prioritized planning and CBS by searching over partial orderings. However, these claims are only evaluated for mobile robots, rather than manipulators.

Task and Motion Planning

Task and Motion Planning (TAMP) addresses the problem in which a robot must simultaneously decide on what to do at a symbolic level and how to do it at a geometric level. The survey by Garrett et al. [39] distinguishes into three broad

paradigms: classical hierarchical interface-based, sampling-based multi-modal, and optimization-based methods.

The classical TAMP formulation uses a symbolic planner to generate candidates. Kaelbling et al. [40] introduced an early work which provides a general strategy for hierarchical planning and execution by limiting the search space. Scrivastava et al. [41] improves upon existing task and motion planning solutions by introducing a predicate-level interface between geometric and symbolic layer. Garrett et al. introduce PDDLStream [42], an extension to the Planning Domain Definition Language (PDDL) [43], which supports black-box implementations for grasps, placements and trajectories by providing adaptive algorithms that interleave symbolic search and constraint satisfaction.

Sampling-based TAMP uses mode switching in hybrid configuration spaces of intersecting sub-manifolds. Each mode switch amounts to sampling mode transitions and concatenating intra-mode paths. Siméon et al. [44] establish this perspective with their PRM-based manipulation planner, Hauser and Latombe [45] provide a more modern formalization with probabilistic completeness. Krontiris and Bekris [46] address non-monotone object rearrangement and Vega-Brown and Roy [47] provide the first asymptotically optimal guarantee with piecewise-analytic differential constraints for TAMP.

Toussaint [48] introduced Logic-Geometric Programming (LGP), in which a symbolic search over action sequences is interleaved with a non-linear program. This collapses the multi-level approach from previous approaches into a single optimization problem. KOMO [49] is used as the trajectory-level solver, a k-order Markov constrained NLP solved by Gauss-Newton with augmented Lagrangian. This integrated optimization-based approach was demonstrated to extend to contact dynamics and tool usage [50].

Multi-Robot Task and Motion Planning

Multi-robot task and motion planning (MR-TAMP) shares the same structure as single-robot TAMP with high-level decision making and low-level execution, but expands to task allocation, motion coordination and often makespan reduction.

Pan et al. [51] extended classic PDDL-based TAMP to multi-robot systems. They introduce a scheduler between the symbolic task planner and multi-robot motion planner. This scheduler is responsible for reducing the number of task planner calls and biasing the search based on geometric insights.

Toussaint and Lopes [52] generalized LGP to cooperative manipulation between a humanoid and Baxter robot but do not push to a larger team of robots. Hartmann et al. [53] demonstrates long-horizon multi-robot planning for construction work by decomposing large scale TAMP problems into smaller one robot subproblems. They leverage Logic-Geometric Programming [48] to solve for keyframes, which are then connected by ST-RRT* [28]. This was followed by formalizing the multi-arm multi-task planning problem based on greedy descent with random restarts to optimize the total makespan [54].

Chen et al. [55] approach multi-arm assembly problems by formulating a high-level mixed-integer program to solve for the task assignment under precedence constraints and a CBS-based low level planner to route the robots collision-free, yielding high-quality makespans at substantial computational cost.

Disassembly Sequence Planning

Disassembly through robots connects the field of disassembly sequence planning (DSP) to robotics, by using the output of DSP as the input for robot task and motion planning.

Lambert’s survey [56] provides the classical foundation for DSP, which are formalized by graph- and AND/OR-based representations. Smith et al. [13] extend this representational line with Disassembly Sequence Structure Graphs (DSSG), which handle multi-target selective disassembly by encoding precedence and accessibility in a single structure.

The graph-based approach is contrasted by treating DSP as optimization based problem. This line of work focuses on genetic algorithms (GA). Kongar et al. [10] and Lazzarini et al. [57] provide early work on GA-DSP, while Kheder et al. [58] provide extensions to address tool changes and accessibility violations under precedence constraints.

The precomputed nature of all DSP approaches sits in stark contrast to recent work by Poschmann et al. [59], which shifts to information-driven and learning-based approaches to acquire information at runtime about the assembly state.

Positioning of Contributions Relative to Prior Work

The existing work provides valuable insights into individual areas of multi-robot disassembly, including multi-robot motion planning and task scheduling, but these components are typically addressed in isolation rather than jointly considering the full disassembly process. This work formulates mechanical disassembly as a dependency-aware, long-horizon, temporally coordinated multi-robot task and motion planning problem. The approach combines methods from long-horizon multi-robot planning [53], ST-RRT* [28], and prioritized planning [35] into a unified framework. Furthermore, the approach extends these methods through DSP [13] and temporal constraint propagation to address task dependencies, unknown arrival times and temporal coordination.

3 Background

This chapter introduces the concepts on which later chapters are based. The topics consist of relevant notations and algorithms from the fields of robotics, task and motion planning. The chapter begins with a topological description of spaces and their composition. This is followed by a recap of robotic kinematics and motion planning.

3.1 Topological Spaces

In order to describe different robots and environments, topological spaces provide a fundamental mathematical framework for reasoning about continuity, connectivity and spatial structure. The topological space associated with a robot allows easier identification of similarities between systems and can lead to a more abstract view of the problem [60].

The most primitive spaces are

$$\mathbb{R}^1 =] - \infty, +\infty[\quad (3.1)$$

$$S^1 = \{(x, y) \in \mathbb{R}^2 \mid x^2 + y^2 = 1\} \quad (3.2)$$

$$D^2 = \{(x, y) \in \mathbb{R}^2 \mid x^2 + y^2 \leq 1\} \quad (3.3)$$

These spaces can be combined with the Cartesian product of the spaces to create compound spaces.

$$X \times Y = \{(x, y) \mid x \in X \text{ and } y \in Y\} \quad (3.4)$$

In robotics, these compound spaces are used to describe different types of joints or robot types [61], for example: the product of real number spaces is used for robot manipulators

$$\mathbb{R}^d = \mathbb{R}^1 \times \mathbb{R}^1 \times \dots \times \mathbb{R}^1 \quad (3.5)$$

and

$$\text{SE}(2) = \mathbb{R}^2 \times S^1 \quad (3.6)$$

as the description of a disk robot with a direction.

The configuration space $\mathcal{Q}$ is the set of all possible states, or configurations, a robot can assume. Most often it is constrained by collision or stability constraints. These are imposed by the physical space of the robot and its environment. These constraints guarantee that no collision with itself or other objects can occur.

The goal for a robot is often defined in the workspace $\mathcal{W}$ which can be different from the configuration space. It is the description of the space the robot inhabits. Often it is also the space of the position and orientation of the end-effector of a manipulator.

The free workspace

$$\mathcal{W}_{\text{free}} = \mathcal{W} \setminus \mathcal{O} \quad (3.7)$$

is defined as a subset of the complete workspace $\mathcal{W}$. Most often $\mathcal{W}$ is a subspace of the Euclidean space like $\mathbb{R}^2, \mathbb{R}^3$ or $SE(3)$.

The configuration map $\mathcal{B}$ is used to convert between the configuration space $\mathcal{Q}$ and the workspace $\mathcal{W}$.

$$\mathcal{B} : \mathcal{Q} \rightarrow \mathcal{W} \quad (3.8)$$

3.2 Kinematics

Most often, robots are described as a system of rigid bodies connected by joints. The pose or configuration $\mathbf{q}$ is the position of a robot within a configuration space $\mathcal{Q}$. The configuration space has d dimensions, representing all joints of the robot [60].

$$\mathbf{q} = \begin{bmatrix} q_1 \\ q_2 \\ \vdots \\ q_n \end{bmatrix} \in \mathcal{Q} \quad (3.9)$$

3.2.1 Forward Kinematics

Forward kinematics is a fundamental concept in robotics which determines the position and orientation of a robot's end-effector given the joint configuration, e.g. angles for revolute joints and displacements for prismatic joints.

$$\mathbf{T}_{\text{EE}} = f(\mathbf{q}) \in SE(3)$$

It is the configuration map $\mathcal{B}$ between the configuration space $\mathcal{Q}$ and the workspace $\mathcal{W}$.

The homogeneous transformation matrix $\mathbf{T}_{EE}$ of the end-effector for robotic manipulators can be computed by a kinematic chain of homogeneous transformations of the individual links and joints [60]. The translation part of the transformation is the relative displacement between two frames

$${}^j\mathbf{p}_i = \begin{bmatrix} {}^jp_i^x \\ {}^jp_i^y \\ {}^jp_i^z \end{bmatrix} \quad (3.10)$$

The rotation part is represented by a rotation matrix $\mathbf{R}$ which is an elementary rotation around a singular axis

$$\mathbf{R}_z(\theta) = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (3.11)$$

$$\mathbf{R}_y(\theta) = \begin{bmatrix} \cos(\theta) & -0 & \sin(\theta) \\ 0 & 1 & 0 \\ -\sin(\theta) & 0 & \cos(\theta) \end{bmatrix} \quad (3.12)$$

$$\mathbf{R}_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\theta) & -\sin(\theta) \\ 0 & \sin(\theta) & \cos(\theta) \end{bmatrix} \quad (3.13)$$

The homogeneous transformation matrix $\mathbf{T}_i^j$ from frame i to frame j can be written as

$$\mathbf{T}_i^j = \begin{bmatrix} \mathbf{R}_i^j & \mathbf{p}_i^j \\ \mathbf{0}^\top & 1 \end{bmatrix} \quad (3.14)$$

The end-effector transformation is obtained by

$$\mathbf{T}_{EE} = \mathbf{T}_0^1 \mathbf{T}_1^2 \dots \mathbf{T}_{n-1}^n \quad (3.15)$$

3.2.2 Inverse Kinematics

Often it is required to map from the workspace $\mathcal{W}$ into the configuration space $\mathcal{Q}$ to move the end-effector into a desired position and orientation.

$$\mathbf{q}_{\text{des}} = f(\mathbf{T}_{\text{des}}) \quad (3.16)$$

The solution to inverse kinematics is far more nuanced than forward kinematics. Since the transformation between configuration space $\mathcal{Q}$ and workspace $\mathcal{W}$ is not bijective, certain points in the workspace may not be reachable by the end-effector. Other points in the workspace might have multiple solutions. Singularities can occur due to an ill-defined problem, resulting in a rapid change in the solution.

Closed-form solutions can be derived through a geometric or algebraic approach using a system of non-linear differential equations. These solutions only apply to a specific set of kinematic structures. Due to the limitation of closed-form solutions, numerical methods are more common in practice. They are performed iteratively to converge to the solution over time.

The Jacobian $\mathbf{J}$ describes the change in end-effector position due to a change in the joint configuration.

$$\mathbf{J}(\mathbf{q}) = \frac{\partial}{\partial \mathbf{q}} \mathbf{f}_{\text{FK}}(\mathbf{q}) \quad (3.17)$$

$$\dot{\mathbf{x}} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}} \quad (3.18)$$

$$\dot{\mathbf{q}} = \mathbf{J}(\mathbf{q})^{-1}\dot{\mathbf{x}} \quad (3.19)$$

Since the inverse of the Jacobian is not always defined, the pseudo inverse $\mathbf{J}^\dagger$ is used instead.

$$\mathbf{J}^\dagger = \mathbf{J}^\top (\mathbf{J}\mathbf{J}^\top)^{-1} \quad (3.20)$$

The problem can also be formulized as an optimization problem

$$\min_{\mathbf{q}} J(\mathbf{q}) \quad \text{s.t. } \mathbf{q} \in \mathcal{Q} \quad (3.21)$$

with

$$J(\mathbf{q}) = \|\mathbf{T}(\mathbf{q}) - \mathbf{T}_{\text{des}}\|^2 \quad (3.22)$$

as the cost function describing the position error between the forward kinematics and goal transform.

The optimization problem is an elegant formulation of inverse kinematics since extensions to the cost function and constraints result in more robust solutions.

3.3 Motion Planning

3.3.1 Geometric Motion Planning

Geometric motion planning deals with the spatial feasibility of a path. It does not take the dynamics of the system into account unlike kinodynamic motion planning. Only the geometry of the robot and objects in the environment are considered. The objective is to find a path τ such that

$$\begin{aligned}\tau(t) &\rightarrow \mathcal{Q}_{\text{free}} \text{ s.t.} \\ \tau(0) &= q_{\text{start}} \\ \tau(1) &= q_{\text{goal}} \\ t &\in [0, 1]\end{aligned}\tag{3.23}$$

The path τ can be a continuous description as a spline or discrete set of states, which have no collision in between them.

It relies on a cost function or metric to determine the quality of the states and therefore the path. The cost function can encode arbitrary values. Most commonly distance metrics are used like the Euclidean distance or Manhattan distance.

The planner needs to be given a function, which can determine if the state is in collision. Collision is determined by a signed distance function which will result in positive values if two bodies intersect. Since convexity is a requirement for efficient distance computation, algorithms like the Gilbert-Johnson-Keerthi distance [62] are applied in practice to check non-convex objects by iteratively computing the Minkowski difference between convex approximations of varying complexities.

3.3.2 Sampling-Based Motion Planning

With a description of the robot's kinematics and geometry and its environment, the problem is to find a continuous path from a start configuration to a goal configuration that lies in $\mathcal{Q}_{\text{free}}$. Sampling-based motion planning has emerged as a dominant paradigm to handle high-dimensional configuration spaces efficiently. The key idea is to generate random configuration samples and use them to incrementally build a path.

The two primary approaches for sampling-based planners are Probabilistic Roadmap (PRM) [14] and Rapidly-Exploring Random Trees (RRT) [15], each with their variants to address certain shortcomings.

Probabilistic Roadmap

With PRM, the planner creates a graph of valid states in the free configuration space $\mathcal{Q}_{\text{free}}$, connecting states which can be connected without collision e.g. draw a straight line in configuration space. Once the graph is grown to a desired size the start and goal states are connected to the existing graph. A graph search algorithm, like Dijkstra [63] or A* [64], can be used to find a valid state sequence. This type of planner is useful if multiple queries in the same space have to be made, since a lot of computation can be reused.

Rapidly-Exploring Random Trees (RRT)

RRT is a single-query planner specific to the start and goal states provided. The samples created are used to extend a tree growing from the start state. If they change or the algorithm is restarted, a different path will be computed.

RRT is a sampling-based motion planning algorithm which relies on the Voronoi bias of randomly generated poses within a space. The algorithm connects a start pose to a goal pose. Typically, the samples are stored in a tree structure to allow for fast nearest neighbor queries. The tree is maintained with each node connected to the closest neighbor.

One of the major effects helping with high dimensional spaces is the Voronoi bias which results in different exploration strategies throughout the sampling stages. In early stages, the planner has a higher chance to sample unexplored areas, resulting in a fast expansion. With more samples, the unexplored space gets smaller and a finer expansion of the space starts.

There are a lot of variations of RRT to improve convergence, such as RRT-Connect [16].

The pseudocode for RRT, in Alg. 1, starts by initializing the graph G with start sample x_{start} (Line 1). The main loop continues as long as no sample in the graph is closer than ϵ to the goal x_{goal} (Line 2). A random sample x_{random} is drawn from the configuration space (Line 3). The nearest neighbor from the graph G is selected (Line 4). A new sample x_{new} is generated by moving from x_{near} to x_{random} for at most the step size of μ (Line 5). If the steering function cannot generate a new sample, the iteration is skipped (Line 6–7). The new vertex is added to the graph (Line 8) and an edge is added between the new sample x_{new} and the nearest neighbor x_{near} (Line 9). If the x_{new} is within ϵ of the goal, the path is reconstructed by following the parent pointers and finally returned (Line 10–11).

Algorithm 1: RRT 15

Input: $x_{\text{start}}, x_{\text{goal}}, \epsilon, \mu$
Output: $x_1, \dots, x_i$

1. $G.\text{addNode}(q_{\text{start}})$
2. **while** $\text{distance}(q_{\text{goal}}, G) \leq \epsilon$ **do**
3. $x_{\text{random}} = \text{sample}()$
4. $x_{\text{near}} = \text{nearestNeighbor}(x_{\text{random}})$
5. $x_{\text{new}} = \text{steer}(x_{\text{near}}, x_{\text{new}}, \mu)$
6. **if** $x_{\text{near}} == x_{\text{new}}$ **then**
7. $\perp$ **continue**
8. $G.\text{addNode}(x_{\text{new}})$
9. $G.\text{addEdge}(x_{\text{near}}, x_{\text{new}})$
10. **if** $\text{distance}(x_{\text{new}}, x_{\text{goal}}) \leq \epsilon$ **then**
11. $\perp$ **return** $\text{path}(x_{\text{new}})$

Asymptotically Optimal Rapidly-Exploring Random Trees (RRT*)

RRT* 17 extends RRT with asymptotic optimality. While RRT provides probabilistic completeness, RRT* guarantees to converge to the most optimal path as the number of samples approaches infinity. This is achieved by selecting the best parent for every new node and rewiring the surrounding tree structure.

The pseudocode of the extension in Alg. 2 starts by initializing the graph G with the start sample x_{init} (Line 1), with the edge set E being implicitly empty. The algorithm runs until n iterations are completed (Line 2), rather than to terminate after an initial solution is found. A random sample x_{rand} is drawn from the free configuration space (Line 3). The nearest vertex to x_{rand} from the set of vertices V is determined (Line 4). A new sample is generated by steering from x_{near} to x_{rand} with the step size η (Line 5). If the path between x_{near} and x_{new} is collision free (Line 6), the iteration continues, else, it continues further steps. The set X_{near} contains all vertices that lie within the radius $\min\{r(|V|), \eta\}$ of x_{new} (Line 7). The radius $\{r(|V|)\}$ depends on the number of vertices, keeping the expected number of neighbors limited. The new sample is added to the set of vertices (Line 8). x_{near} is initialized as a baseline for the parent vertex (Line 9). The initial cost to go to x_{new} is calculated by the cost to go to x_{near} and the distance between the samples (Line 10). Every vertex, which was previously (Line 7) near enough to be considered a neighbor, is checked if it has a lower cost to connect to the new sample x_{new} (Line 11–15). After the lowest cost vertex is found, an edge between x_{new} and x_{min} is added to the edge set E (Line 16). Afterwards, the set of neighbors

X_{near} is traversed (Line 17) to check if the new sample x_{new} has a collision free path (Line 18) with a lower cost (Line 19) to one of its neighbors. If this is the case, the parent of the neighbor is updated to x_{near} to the new sample x_{new} (Line 20–21). After n iterations, the graph G is returned. The final path is extracted by following the parent pointers from the goal node with the lowest cost back to the root.

Algorithm 2: RRT*^[65]

Input: $x_{\text{start}}, x_{\text{goal}}, \eta$
Output: G

1. $V = x_{\text{init}}$
2. **for** $j = 1$ **to** n **do**
3. $x_{\text{rand}} \leftarrow \text{sampleFree}()$
4. $x_{\text{near}} \leftarrow \text{nearest}(x_{\text{rand}}, V)$
5. $x_{\text{new}} \leftarrow \text{steer}(x_{\text{near}}, x_{\text{rand}}, \eta)$
6. **if** $\text{collisionFree}(x_{\text{near}}, x_{\text{new}})$ **then**
7. $X_{\text{near}} = \text{near}(x_{\text{new}}, V, \min\{r(|V|), \eta\})$
8. $V = V \cup \{x_{\text{new}}\}$
9. $x_{\text{min}} = x_{\text{near}}$
10. $c_{\text{min}} = \text{cost}(x_{\text{near}}) + \|x_{\text{new}} - x_{\text{near}}\|$
11. **for** $x_{\text{near}} \in X_{\text{near}}$ **do**
12. **if** $\text{collisionFree}(x_{\text{near}}, x_{\text{new}})$ **then**
13. **if** $\text{cost}(x_{\text{near}}) + \|x_{\text{new}} - x_{\text{near}}\| < c_{\text{min}}$ **then**
14. $x_{\text{min}} = x_{\text{near}}$
15. $c_{\text{min}} = \text{cost}(x_{\text{near}}) + \|x_{\text{new}} - x_{\text{near}}\|$
16. $E = E \cup \{(x_{\text{min}}, x_{\text{new}})\}$
17. **for** $x_{\text{near}} \in X_{\text{near}}$ **do**
18. **if** $\text{collisionFree}(x_{\text{new}}, x_{\text{near}})$ **then**
19. **if** $\text{cost}(x_{\text{new}}) + \|x_{\text{near}} - x_{\text{new}}\| < \text{cost}(x_{\text{near}})$ **then**
20. $x_{\text{parent}} = \text{parent}(x_{\text{near}})$
21. $E = E \cup \{(x_{\text{new}}, x_{\text{near}})\} \setminus \{(x_{\text{parent}}, x_{\text{near}})\}$
22. **return** $G = (V, E)$

Space-Time RRT*

Classical, sampling-based planners work by sampling the configuration space $\mathcal{Q}$. This assumes that the configuration space is a static environment and does not

change over time. In practice, obstacles move and the planner has to account for that. Classical planners have to represent this by updating the collision free configuration space $\mathcal{Q}_{\text{free}}$ according to the obstacle. Space-Time RRT* (ST-RRT*) [28] address this by embedding time directly in the time space as an additional dimension.

$$\mathcal{X} = \mathcal{Q} \times \mathbb{R}_{\geq 0} \quad (3.24)$$

This approach requires each edge in the tree to respect the monotonicity of time in the direction of the edge, i.e. $t_{\text{new}} > t_{\text{near}}$. This guarantees that no motion back in time is allowed. As a result of this formulation, goal states are typically defined as goal regions in space-time.

The algorithm leverages a bidirectional approach, growing a tree from the start and goal simultaneously. The main algorithm follows the approach from RRT-Connect and extends it by a conditional goal sampler, a state sampler over the bounded space-time state space and an adaptive upper time bound.

The pseudocode for ST-RRT* is described in Alg. 3. ST-RRT* maintains a start tree T_a rooted at its start sample x_{start} and a goal forest T_b , which grows from the goal states (Line 1). The goal forest is initialized empty, while the start tree is initialized with the start sample. Initial bound variables are determined (Line 2) until a valid solution can be found. The planner runs until it is terminated (Line 3). With the probability p_{goal} (Line 5) a new goal is sampled and added as a root of T_b (Line 5–6). Afterwards a sample x_{rand} is generated in the reachable area from the start and goal states based on the velocity and time constraints (Line 7). The start tree T_a is extended to x_{rand} with at most the step size d (Line 8). If it fails the sample is skipped. The batch and global sample counters are increased (Line 9–10). Then, the goal forest is rewired around the new sample x_{new} generated by the extend step (Line 11). Then ST-RRT* attempts to connect x_{new} to T_b , with a successful connection resulting in a complete trajectory from start to goal (Line 12). If a connection is possible the solution is updated by reconstructing the trajectory (Line 13). The time to reach the goal becomes the new upper bound (Line 14). The probability of the batch is set to one to ensure new samples are sampled from the bounded region (Line 15). With the new time bound in place, samples which no longer can improve the solution are pruned (Line 16). As the last step of the loop, T_a and T_b are swapped to ensure equal effort between both (Line 17). Upon termination, the planner returns the best trajectory found (Line 18).

Algorithm 3: ST-RRT* [28]

Input: $\mathcal{X}, x_{\text{start}}, \mathcal{X}_{\text{goal}}, d, \text{PTC}, t_{\text{max}}, p_{\text{goal}}, P$
Output: solution

1. $T_b \leftarrow \{x_{\text{start}}\}$
2. $T_a \leftarrow \emptyset$
3. $B \leftarrow \text{initializedBoundVariables}(P)$
4. **while** $\neg \text{PTC}$ **do**
5. $B \leftarrow \text{updateGoalRegion}(B, P, t_{\text{max}})$
6. **if** $p_{\text{goal}} \geq \text{RND}(0, 1)$ **then**
7. $B \leftarrow \text{sampleGoal}(x_{\text{start}}, \mathcal{X}_{\text{goal}}, T_{\text{goal}}, t_{\text{max}}, B)$
8. $x_{\text{rand}} \leftarrow \text{sampleConditionally}(x_{\text{start}}, \mathcal{X}, B, d)$
9. **if not extend** $(T_a, x_{\text{rand}}, d) == \text{Trapped}$ **then**
10. $B.\text{samplesInBatch} += 1$
11. $B.\text{totalSamples} += 1$
12. $\text{rewireTree}(T_a, T_{\text{goal}}, x_{\text{new}})$
13. **if connect** $(T_b, x_{\text{new}}, d) == \text{Reached}$ **then**
14. $\text{solution} \leftarrow \text{updateSolution}(x_{\text{new}})$
15. $t_{\text{max}} \leftarrow \text{costPath}(\text{solution})$
16. $B.\text{batchProbability} \leftarrow 1$
17. $\text{pruneTrees}(t_{\text{max}}, T_a, T_b)$
18. $\text{swap}(T_a, T_b)$
19. **return** solution

Multi-Robot-Planning

When multiple robots have to act in the same workspace $\mathcal{W}$, the problem definition extends to finding a valid, collision-free path for every robot.

The same approach to the multi-robot as to the single-robot problem combines the state space of every robot to a single state space.

$$\mathcal{X} = \mathcal{Q}_1 \times \mathcal{Q}_2 \times \cdots \times \mathcal{Q}_n \quad (3.25)$$

This formulation results in a state space, which grows linearly with the number of robots. Algorithms, such as RRT*, can find optimal solutions in these state spaces. However, most complete algorithms grow exponentially with the dimension, which makes them unfeasible in practice. Sampling-based planners scale better, but can also struggle to find solutions in high dimensions [66].

This scaling problem motivated the development of alternative approaches to multi-robot planning. One approach is prioritized planning [35], which avoids the dimensionality explosion by decomposing the planning problem into n sub-problems, with n being the number of robots. Each robot is planned sequentially and previously planned robots are treated as dynamic obstacles by the next robot. Each planning problem has the same complexity as a single-robot planning problem, which makes it computationally feasible even for large number numbers of robots. The drawback is that this approach is neither complete nor optimal.

Short Cutting

Short cutting [67] is a method to improve path quality, while maintaining a collision-free path. The algorithm works on a piecewise linear path and iterates through the whole path. Each iteration, two samples are sampled from the path. Both samples are connected by a straight line in configuration space and connected if the intermediate states are collision-free. Afterwards, the redundant, longer path is pruned. This is repeated for a fixed number of iterations or no improvements are longer possible. Short cutting is not a replacement for finding optimal paths through dedicated planners. However, it can consistently produce shorter paths for a low computational cost.

Spline Interpolation

The solution of a sampling-based planner is a piecewise linear path through a sequence of samples $q_0, q_1, \dots, q_n$ from the configuration space $\mathcal{Q}$. Since the planner is only concerned with finding a geometrically collision-free path, discontinuities of the first derivative occur, leading to an unbounded acceleration required to follow this path. To remedy this behaviour spline interpolation is applied to smooth out the path. The spline is a piecewise polynomial function with C^{p-1} continuity. The B-spline is a linear combination

$$q(s) = \sum_{i=0}^n N_{i,p}(s) c_i \quad (3.26)$$

of basis function $N_{i,p}$ over a set of control points c_i [68]. This spline must be parameterized such that the resulting trajectory respects the kinodynamic bound of the robot.

4 Approach

This chapter will discuss the approach to the disassembly of arbitrary assemblies using multiple robots. After stating the assumptions and problem definition, a high-level planner is proposed which takes a precedence graph as an input and returns the trajectories required for the disassembly. Finally, the implementation details are discussed, which are later used in experimental evaluation of the approach.

4.1 Assumptions

This thesis provides a first feasible solution, which can be used as a reference trajectory for controller designs and implementations. This allows the controller to focus on robot and contact dynamics, while providing coordination between the robots.

Since this thesis focuses solely on the geometric problem, three major assumptions are introduced.

First, no dynamics simulation is used for planning the trajectory. Therefore, effects such as stick-slip or object resting behaviors do not have to be considered. Although the controller allows for a certain degree of compliance, the focus of this thesis is on the motion planning problem. As a result of this simplification, grasping an object is reduced to a rigid connection between object and end-effector.

Second, the end-effector is modeled as an omnidirectional gripper, enabling grasps from any angle on the surface of the object. This is done to facilitate the plethora of gripper designs, which range from general two-finger to more specific grippers, such as soft grippers [69][70][71].

Finally, it is assumed that a directed graph of dependency constraints is given as an input. This graph is precomputed based on the geometry of the assembly. The graph encodes the order in which objects can be moved. The generation of the graph also determines the path that an object must follow to be considered free

to move. This assumption is based on established research for sequencing assemblies by generating AND/OR-Graphs [56] and more recent approaches through optimization [58] or simulation [11]. More recent work [59] leverages vision and machine learning to generate information of the assembly at runtime.

4.2 High Level Task and Motion Planning

This section presents the high-level algorithm for this thesis. First the problem is defined. Then, an overview of the algorithm is presented, followed by the implementation. This is followed by a discussion about completeness and optimality of this approach. Finally, the chapter closes on how temporal constraints affects the problem.

4.2.1 Problem Statement

Considering a workspace $\mathcal{W}$ with n robots. Each robot $r_i \in \mathcal{R} = \{r_1, \dots, r_n\}$ has a configuration space $\mathcal{Q}_i$ with d_i degrees of freedom. The compound state space is defined as $\mathcal{X} = \mathcal{Q}_1 \times \dots \times \mathcal{Q}_n$ with a combined dimension $d = \sum d_i$.

It is assumed that an assembly with k rigid objects $\mathcal{O} = \{o_1, \dots, o_k\}$ exists in $\mathcal{W}$. Each object $o_j \in \mathcal{O}$ is defined by a starting pose $T_{\text{start}}^j \in SE(3)$ in the assembled state and a goal pose $T_{\text{goal}}^j \in SE(3)$ in the disassembled state.

The solution to the multi-robot disassembly problem is a set of trajectories $\mathcal{T}$. Each trajectory $\pi_i \in \mathcal{T}$ is a sequence of pairs consisting of a configuration and associated time point. They describe the collision free motion for each robot and object to reach the disassembled state.

The problem is to find a set of trajectories $\mathcal{T}^*$, which minimize the makespan, the total time required for the disassembly process from start to finish.

4.2.2 Object Dependency

Objects often need to be assembled and disassembled in a specific sequence since parts might obstruct each other. A trivial example is stacked boxes, which must be disassembled from top to bottom. This information can be captured using a graph representation.

In a graph data structure, the vertices V represent the objects and the edges E represent the dependencies. The graph is considered to be a directed acyclic graph (DAG).

In Fig. 4.1, five objects are specified in the problem. Object 1 depends on object 4 and object 2 depends on objects 4 and 5. All leaf nodes in the graph have no dependencies on other nodes.

This structure can be queried to identify the leaf nodes that can be moved by the robot.

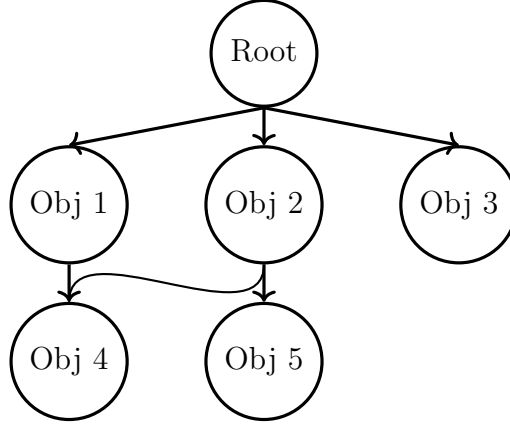


Figure 4.1: Example of a dependency graph

4.2.3 Overview

In order to solve the stated problem, the proposed algorithm takes a dependency graph as input, parses it for movable objects represented by leaf nodes and assigns a task to an available robot.

Using multiple robots to disassemble a structure results in two primary problems. Which robot should move which object and determining how the robot should move the desired object to its goal location. The proposed algorithm uses interleaved task assignment and motion planning in an event-driven loop to coordinate the disassembly.

The approach follows the priority planning paradigm common in multi-robot motion planning. The first robot plans in an empty space, the second robot treats the first robot as a dynamic obstacle and so on. Previously planned trajectories are committed and not changed in later tries. This approach does not guarantee global optimality of the overall makespan, but avoids exponential growth of the

space complexity with an increase in robots used [72]. Alternatives like conflict-based approaches are optimal, but can result in lots of conflict resolution for narrow passages and need a separate strategy to resolve conflicts efficiently.

4.2.4 Algorithm

The pseudocode for the algorithm is described in Alg. 4. The input to the algorithm is a precomputed dependency graph and all available robots. While the dependency graph is not empty (Line 1), the unassigned leaf nodes are obtained, which do not depend on any other vertex (Line 2). Next, all robots that currently do not have an assigned task are selected (Line 3). The algorithm then matches an available robot with a node based on an assignment policy (Line 5). It then creates a task in the robot's task queue (Line 6) and marks the node as assigned to avoid reassignment (Line 7), continuing until all possible pairings of leaf-robot have been exhausted (Line 5). Afterwards, it iterates through every task queue (Line 8), skipping any empty queues since no work needs to be done (Line 9). The first task in the queue is selected (Line 11) and the main run of the of the selected task is invoked (Line 12). Once the task has finished (Line 13), the algorithm checks for failure (Line 14). If the task has failed, the associated node is unmarked in the tree (Line 15) and is ready to be reassigned. If the task is successful, the constraints on the next node are propagated (Line 17) and the node is removed from the dependency tree (Line 18). As there is no more work left for the task, it is removed from the queue (Line 19), freeing the robot for other work.

Algorithm 4: High Level Multi Robot Planning

Input: *depTree*, *robots*
Output:

1. **while** *depTree* $\neq \emptyset$ **do**
2. *leafs* $\leftarrow$ *getUnassignedLeafs*(*depTree*)
3. *freeRobots* $\leftarrow \{r \in \text{robots} \mid \text{taskQueue}(r) = \emptyset\}$
4. **while** *leafs* $\neq \emptyset \wedge \text{freeRobots} \neq \emptyset$ **do**
5. *robot*, *leaf* = *selectRobotAndLeaf*(*freeRobots*, *leafs*)
6. *task* = *createTask*(*robot*, *leaf*)
7. *markLeafAssigned*(*leaf*)
8. **for** (*robot*, *queue*) $\in$ *assignedTasks* **do**
9. **if** *queue* = $\emptyset$ **then**
10. *continue*
11. *currentTask* $\leftarrow$ *queue.front*()
12. *finished* $\leftarrow$ *run*(*currentTask*)
13. **if** *finished* **then**
14. **if** *currentTask.failed* **then**
15. *unmarkLeafAssigned*(*leaf*)
16. **else**
17. *propagateConstraints*(*depTree*, *leaf*)
18. *removeVertex*(*depTree*, *leaf*)
19. *pop*(*taskQueue*(*robot*))

4.2.5 Completeness and Optimality

The sequential approach prioritizes computational feasibility over optimality. However, the quality of the solution depends on the order of assignment and execution. For example, an early robot may occupy regions needed by later robots, forcing longer detours or planning failures. Global makespan optimality is not guaranteed, since each robot independently minimizes its own arrival time. Achieving global optimality would require joint planning or iterative trajectory refinement, both of which are beyond the scope of this work.

Assuming a feasible disassembly exists, this algorithm is not probabilistically complete. This is even the case when the path planning algorithm is probabilistically complete and able to eventually find a solution, if one exists. Since the algorithm has limited abilities to regrasp an object once it has been removed, certain goal configurations are not feasible. This could be an intermediate place-and-pick phase

or a handover to another robot. The scenario in Fig. 4.2 requires a regrasp to guarantee probabilistic completeness. The initial state is displayed in the first step (1), with the screw inserted in an unmovable obstacle. The head of the screw is the only area where the end-effector can attach in the second step (2). The requested goal position of the screw with the current grasp pose, is resulting in collision with the table. Therefore, a regrasp in a temporary position has to be performed in step (3). This step can also be the regrasp by another robot, instead of the regrasp of the same robot. Finally, the new grasp pose allows the requested place configuration.

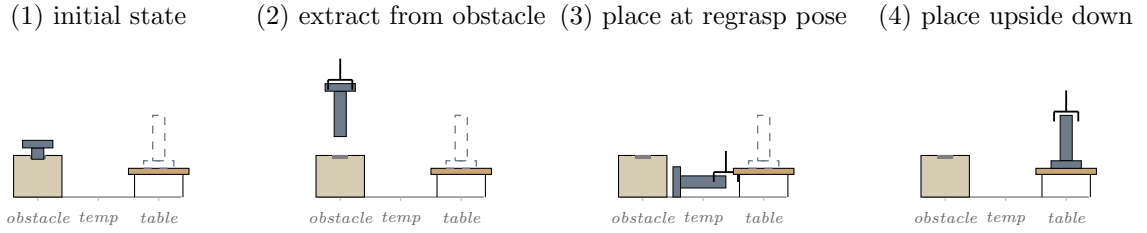


Figure 4.2: Screw regrasp problem for orientation

Furthermore, the algorithm cannot suspend a task, if another task has to be completed first and there are not enough robots present to execute them simultaneously. Fig. 4.3 shows this behavior. In the first step (1), two boxes are stacked on top of each other with only one available robot. The robot has to move box A to a temporary position in the second step (2) in order to get access to box B, assuming it is not allowed to tip the tower over. Then, box B is moved to the desired location in step three (3) and box A is moved to the now reachable goal state in the final step four (4).

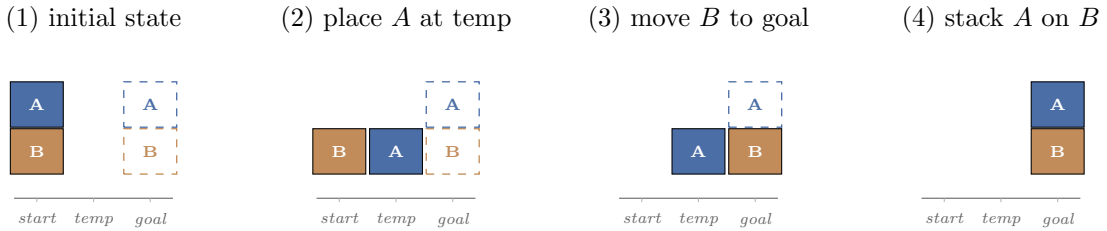


Figure 4.3: Task ordering problem

The framework needs to be able to differentiate between completely and currently unfeasible goal states. If it is assumed that no completely infeasible goal states are provided as input, the algorithm then has to decide on how to resolve this conflict.

Further both scenarios require the concept of a temporary or intermediate state, which the current framework does not provide to allow planning when not enough robots are present.

4.2.6 Temporal Dependency Constraints

The high-level behavior in Alg. 4 removes a dependency vertex after the associated task has been finished. Only then are new leaf nodes available for manipulation. This is a conservative approach, since a successor object can begin its planning before its predecessor has completed its full task sequence. Waiting for the full completion of the predecessor's task wastes time, during which a second robot could already begin approaching the next object.

To exploit this, each vertex in the dependency graph carries a constraint type that controls when temporal bounds are propagated to its successors. Four types are defined:

In Fig. 4.4, no temporal bound is propagated to the next node. The successors simply become available when the vertex is removed from the tree.

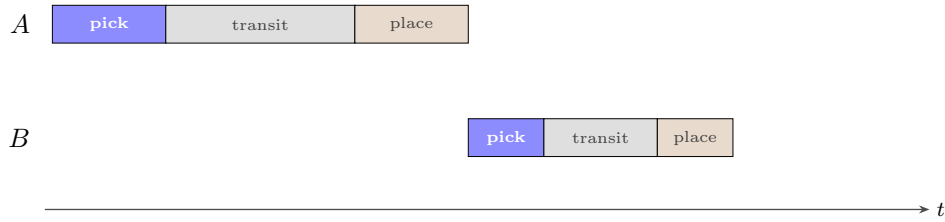


Figure 4.4: Example timeline for no time constraints

With a temporal constraint on the pick time in Fig. 4.5, a successor may begin their own pick phase as soon as the predecessor is attached. When the task finishes, the attach time t_{attach} of this object is propagated as a lower bound on the successor's attach time:

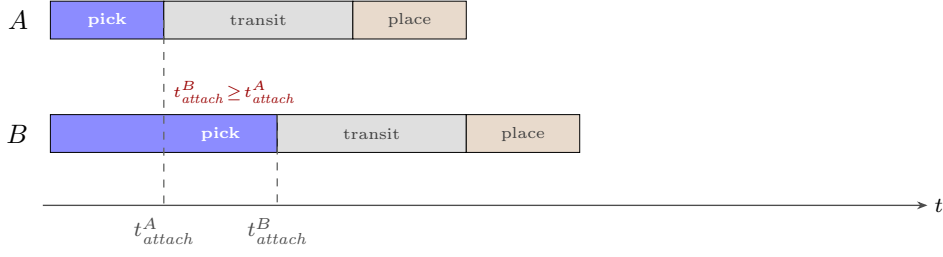


Figure 4.5: Example timeline for time constraint on the pick time

In Fig. 4.5 the place time is constrained, a successor may begin their pick phase at any time, but can only place it after the predecessor has been placed. This is relevant when the successor's goal location is depending on the predecessor's current position. For example, this can happen if objects need to be stacked.

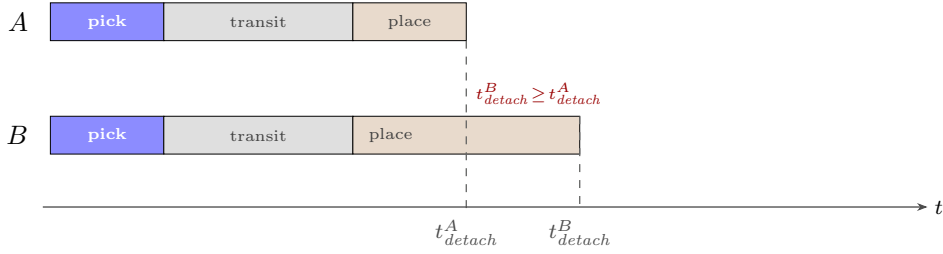


Figure 4.6: Example timeline for time constraint on the place time

Both bounds can also be propagated. In Fig. 4.7 the successor's pick is constrained by the predecessor's attach time and its place by the predecessor's detach time. This can be necessary if you want to stack a tower of cubes upside down. The topmost cube needs to be picked before the second but also has to be placed at the bottom before the second one arrives.

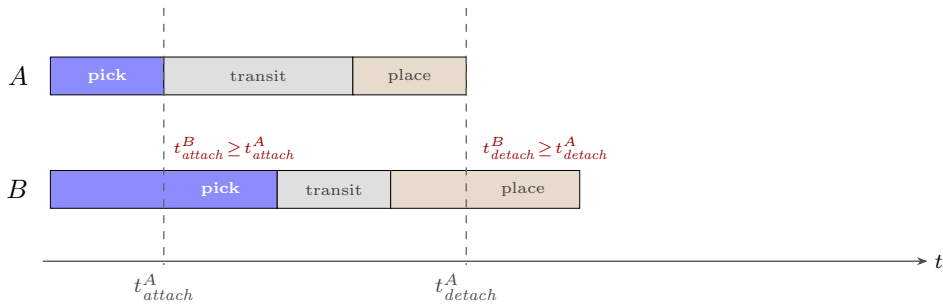


Figure 4.7: Example timeline for time constraint on pick and place time

4.3 Task Representation

The disassembly of a structure consists of m different tasks, which have to be executed in a certain order. Typically each object has to be moved from its initial pose to the desired goal pose. These actions can be decomposed into a sequence of discrete tasks. Each task is associated with a start time $t_{\text{start}} \in \mathbb{R}$ and an end time $t_{\text{end}} \in \mathbb{R}$. These time stamps represent the time frame in simulation time, where the task is active and assigned to the robot. Each task signals to the scheduler when it has finished execution and can be removed.

Task Queue

Centralized storage maintains the relation between each robot and a queue of tasks for each robot. A robot is considered available if and only if its queue is empty. Tasks are executed in first-in-first-out order. The front task is invoked at each simulation step until it signals completion. Completion of the task triggers the removal and the next task begins execution. If the queue is empty, the robot is assumed to be available again and ready for new task assignments.

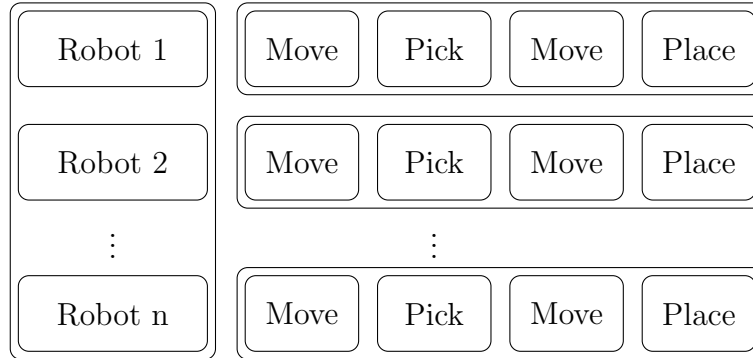


Figure 4.8: Task queue visualization

Primitive Tasks

The following primitive tasks constitute the atomic operations from which higher-level sequences are composed.

Move Task

The move task plans a collision-free path from the robot's current configuration $q_{\text{start}} \in \mathcal{Q}$ to a specified goal pose $T_{\text{goal}} \in \mathcal{W}_{\text{free}}$ in the collision free workspace or a configuration $q_{\text{goal}} \in \mathcal{Q}$ in the robot's configuration space. The robot's current position at the first execution of the task is regarded as its start configuration (Line 1).

The planner is allowed to retry the movement request on failure n_{retry} times (Line 2), since the planner's success is depending on the goal state and configuration samples generated.

If a pose is specified as the goal, a set of goal configurations is generated by solving the corresponding inverse kinematics problem (Line 3). The try finishes early if no goal states are feasible (Line 3–4).

The planner is setup with the required information from the robot and its start and goal conditions (Line 6). The planner is then queried to generate the motion (Line 7). If the planner is successful in finding a solution (Line 8), the result is committed to the robot's path storage (Line 9). Finally, the task returns success (Line 10) or exhausts all attempts and returns failure (Line 11).

Algorithm 5: Move Task

Input: robot, goal, t_{start}
Output: success

1. $\mathbf{q}_{\text{start}} \leftarrow \text{getConfiguration}(\text{robot})$
2. **for** $i \leftarrow 1$ **to** n_{retry} **do**
3. $\text{goalStates} \leftarrow \text{generateGoalStates}(\text{robot}, \text{goal})$
4. **if** $\text{goalStates} = \emptyset$ **then**
5. continue
6. $\text{planner} \leftarrow \text{Planner}(\text{robot}, \mathbf{q}_{\text{start}}, t_{\text{start}}, \text{goalStates})$
7. $\text{path} \leftarrow \text{plan}(\text{planner})$
8. **if** $\text{path} \neq \emptyset$ **then**
9. $\text{appendPath}(\text{robot}, \text{path})$
10. **return true**
11. **return false**

Attach and Detach Task

These tasks represent the grasping of an object by the robot. The attach task saves the homogeneous transformation between end-effector and object at time t_{attach} .

$$T_{\text{EE}}^{\text{obj}} = (T_{\text{world}}^{\text{EE}})^{-1} \cdot T_{\text{world}}^{\text{obj}} \quad (4.1)$$

Saving the homogeneous transform between end-effector and object at the time of execution allows for later reconstruction of the object position based on the robot pose, as described in Sec. 4.6. This task does not move the robot to the object and solely tracks the relationship between robot and object at a specific time.

The tracking is done through the `attachData`, which resembles the association between robot and object at a specific time.

The detach task terminates the kinematic coupling by setting the time t_{detach} of the current `attachData` associated with the robot.

$$T_{\text{world}}^{\text{obj}}(t) = T_{\text{world}}^{\text{EE}}(t) \cdot T_{\text{EE}}^{\text{obj}} \quad \forall \quad t_{\text{attach}} \leq t \leq t_{\text{detach}} \quad (4.2)$$

Therefore, each `attachData` keeps track of:

- o_j : A reference to an object
- r_i : A reference to a robot
- t_{attach} : The attach time
- t_{detach} : The detach time
- $T_{\text{EE}}^{\text{obj}}$: A relative transformation between object and end-effector of the robot

Exit Task

After placing an object, the robot has to vacate the area to avoid obstructing subsequent operations by another robot. The exit task plans a path to a location in a specified safe zone in the workspace, where it does not obstruct any other robot. The configuration is set to the home position of the robot to guarantee a neutral starting position for the next task.

The goal used in the exit task is the final configuration until the robot is reassigned a new task. If the configuration intersects with a committed trajectory of another robot at any future time point, a collision occurs. Therefore, a potential exit configuration has to be verified against all future time points of other trajectories.

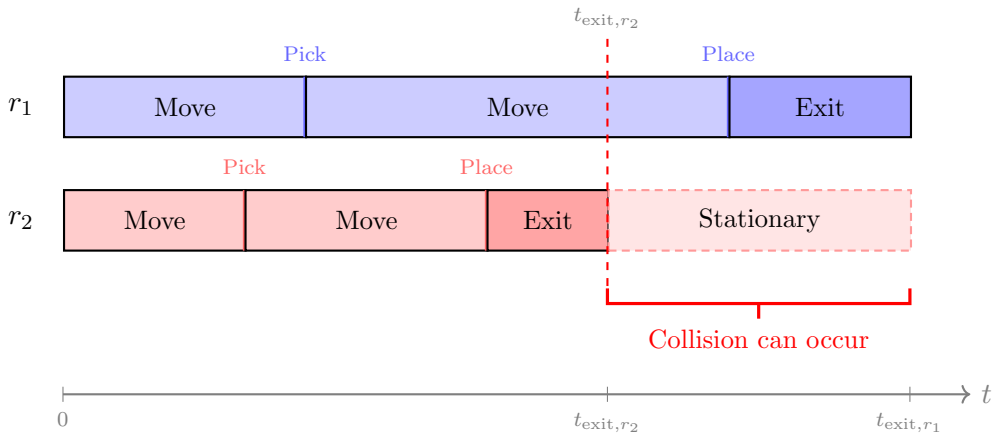


Figure 4.9: Timeline illustration for future collision check

To check the candidate for validity, a linear sweep is performed from the start of the exit task until the largest committed time point of any robot.

The pseudocode to check a configuration for future collisions is described in Alg. 6. First, the lower time point of the time window is determined by the state passed to check for future collision (Line 1). Then, the upper time is determined by largest committed time point of any robot's trajectory (Line 2). While the time of the state is smaller or equal to the maximum time point (Line 3), the state time is increased incrementally by Δt (Line 4 and 5). Each increment, the state of the simulation is checked for collision (Line 6) and returns early on occurrence (Line 7). After the complete time window is passed, the state is validated (Line 8).

Algorithm 6: Check Future Collision

Input: robot, $\mathbf{q}$, t_{at}
Output: validConfiguration

1. $t \leftarrow t_{at}$
2. $t_{max} \leftarrow \text{getMaxTrajectoryTime}()$
3. **while** $t \leq t_{max}$ **do**
4. $t \leftarrow t + \Delta t$
5. $\text{setSimTime}(t)$
6. $\text{setConfiguration}(\text{robot}, \mathbf{q})$
7. **if** $\text{isInCollision}()$ **then**
8. **return** false
9. **return** true

With the introduced algorithms, the complete exit tasks can be formulated in Alg. 7. The robot considers the current position as the start position for the exit task (Line 1). It tries up to a specified number of times (Line 2) to find a exit plan. It first starts with an empty set of possible goal states (Line 3). This set is filled with the specified number of valid exit configurations (Line 4). First a candidate is drawn from the specified exit region (Line 5). Since the robot might stay at the configuration until the simulation ends, the configuration has to be checked for future collisions (Line 6). After the candidate is verified, it is added to the possible goal states (Line 7). If no goal states could be generated a retry is triggered early (Line 8 and 9). The planner then tries to connect the start configuration to one of the possible exit configurations (Line 10). If it succeeds (Line 11), the exit path is committed to the planner's trajectory (Line 12) and returns successfully (Line 13). When all retries are exhausted, the exit task returns failure (Line 14).

Algorithm 7: Exit Task

Input: robot
Output: success

1. $\mathbf{q}_{\text{start}} \leftarrow \text{getPose}(\text{robot})$
2. **for** $i \leftarrow 1$ **to** retryCount **do**
3. $\text{goalStates} \leftarrow \emptyset$
4. **for** $j \leftarrow 1$ **to** $\text{numberOfExitConfigurations}$ **do**
5. $\mathbf{q}_{\text{exit}} \leftarrow \text{sampleExitRegion}()$
6. **if** $\text{checkFutureCollision}(\mathbf{q}_{\text{exit}})$ **then**
7. $\text{append}(\text{goalStates}, \mathbf{q}_{\text{exit}})$
8. **if** $\text{goalStates} = \emptyset$ **then**
9. **continue**
10. $\text{path} \leftarrow \text{plan}(\text{robot}, \mathbf{q}_{\text{start}}, \text{goalStates})$
11. **if** $\text{path} \neq \emptyset$ **then**
12. $\text{appendPath}(\text{robot}, \text{path})$
13. **return** true
14. **return** false

Follow Path

Objects that are geometrically interlocked with adjacent parts are hard to move freely through the workspace after grasping. Instead, they must follow a precomputed path consisting of a sequence of waypoint transforms $T_1, \dots, T_m \in \text{SE}(3)$. These waypoints describe a collision-free trajectory through the surrounding geometry. This procedure is used both for extraction (removing an object from surroundings) and insertion (guiding an object into its goal position).

The follow path task advances through the waypoints using inverse kinematics. For each waypoint T_i , the required end-effector pose is computed from the waypoint and the current grasp transform:

$$T_i^{\text{EE}} = T_i \cdot (T_{\text{EE}}^{\text{obj}})^{-1} \quad (4.3)$$

The IK solver attempts to find a joint configuration $\mathbf{q}_i$ that realizes this end-effector pose.

The inverse kinematics problem for each waypoint can be stated as a constrained

optimization. The full space-time waypoint problem is:

$$\begin{aligned}
 \min_{\mathbf{q} \in \mathcal{Q}, t \in \mathbb{R}} \quad & \alpha t + (1 - \alpha) \|q_i - q_{i-1}\|^2 \quad \text{s.t.} \quad T_{\text{FK}}(\mathbf{q}) = T_i^{\text{EE}} \\
 & \mathbf{q}_l \leq \mathbf{q} \leq \mathbf{q}_u \\
 & t > t_{i-1} + \epsilon \\
 & (\mathbf{q}, t) \in \mathcal{Q}_{\text{free}}(t)
 \end{aligned} \tag{4.4}$$

where $\mathcal{Q}_{\text{free}}(t)$ denotes the free configuration space at time t , determined by the committed trajectories of all other robots and objects.

Solving Eq. 4.4 jointly is intractable in practice, as the collision constraint couples the kinematic and temporal variables through the full simulation state. The problem is therefore decomposed into two sequential subproblems. First, the kinematic subproblem is solved independently

$$\min_{\mathbf{q} \in \mathcal{Q}} \|T_{\text{FK}}(\mathbf{q}) - T_i^{\text{EE}}\|^2 + \lambda \|q_i - q_{i-1}\|^2 \quad \text{s.t.} \quad \mathbf{q}_l \leq \mathbf{q} \leq \mathbf{q}_u \tag{4.5}$$

Second, the temporal subproblem finds the earliest collision-free time for the IK solution:

$$t_i = \min\{t \geq t_{i-1} + \epsilon \mid (\mathbf{q}_i, t) \in \mathcal{Q}_{\text{free}}(t), t - t_{i-1} < t_{\text{max}}\} \tag{4.6}$$

by incrementally advancing in steps of Δt until a valid time is found or the maximum time increase t_{max} is exceeded. This decomposition sacrifices joint optimality for tractability. Configurations can exist that would be valid at an earlier time with a different $\mathbf{q}$ but are not discoverable by the sequential approach. In practice, the waypoint density of the precomputed paths is sufficient to ensure that the decoupled solution reliably finds valid states.

The pseudocode for following a path is described in Alg. 8. The algorithm starts by getting the last committed trajectory time point of the robot (Line 1). It then iterates through each waypoint provided (Line 2). First it calculates the end-effector transformation from the desired object path (Line 3). It then solves the inverse kinematics problem for that transformation (Line 5) to find the next configuration. If no valid configuration is found (Line 6), a regrasp can be attempted if allowed (Line 7–9). When the object does not permit regrasps, a failure is returned (Line 8). After the regrasp is successful, the desired end-effector transformation is updated with the new relative transformation between end-effector and object (Line 10). This procedure is repeated until a valid configuration is found (Line 11).

After a possible configuration is found. The last committed trajectory time point is stored (Line 12). This time point is then used to iterate over a time window,

which is limited by an upper bound (Line 13). The solution from the inverse kinematics is set to the relevant time point (Line 14) and validated for collision (Line 15). If no collision occurs, the state is committed to the robot’s trajectory (Line 16) and continues to the next inverse kinematics problem (Line 17). In case of collision, the time point is increased by a time delta Δt (Line 19), until either a valid time point is found or the time window is exhausted.

After the time window is over (Line 20) and no solution was found, a regrasp is performed (Line 23) and the current waypoint is retried again (Line 24). If no regrasp is permitted, the planner instead fails (Line 22) after no valid solution was found.

When all waypoints find a suitable configuration and timepoint, the algorithm returns success (Line 25).

Algorithm 8: Follow Path

Input: robot, object, path = $[T_1, \dots, T_m]$
Output: success

1. $t \leftarrow \text{getLastStateTime}(\text{robot})$
2. **for** $T_i \in \text{path}$ **do**
3. $T_i^{\text{EE}} \leftarrow T_i \cdot (T_{\text{EE}}^{\text{obj}})^{-1}$
 // Solve IK, regrasping on failure
4. **repeat**
5. $\mathbf{q}_i \leftarrow \text{solveIK}(\text{robot}, T_i^{\text{EE}})$
6. **if** $\mathbf{q}_i = \emptyset$ **then**
7. **if** *not* $\text{object.allowRegrasp}$ **then**
8. **return** failure
9. $\text{regrasp}(\text{robot}, \text{object}, t)$
10. $T_i^{\text{EE}} \leftarrow T_i \cdot (T_{\text{EE}}^{\text{obj}})^{-1}$ // Recompute with new grasp
11. **until** $\mathbf{q}_i \neq \emptyset$
 // Find earliest valid time
12. $t_{\text{start}} \leftarrow t$
13. **while** $t - t_{\text{start}} < t_{\text{max}}$ **do**
14. $s \leftarrow \text{createState}(\mathbf{q}_i, t)$
15. **if** $\text{isValid}(s)$ **then**
16. $\text{robot.states.append}(s)$
17. **break**
18. **else**
19. $t \leftarrow t + \Delta t$
20. **if** $t - t_{\text{start}} \geq t_{\text{max}}$ **then**
21. **if** *not* $\text{object.allowRegrasp}$ **then**
22. **return** failure
23. $\text{regrasp}(\text{robot}, \text{object}, t)$
24. **redo** current waypoint T_i
25. **return** success

4.4 Compound Task: Pick and Place

The primitive tasks can be composed into higher-level sequences to encapsulate the complete manipulation of an object.

The compound task wraps multiple primitive tasks together to allow for planning at the same simulation time step. This can be necessary if it is required for a robot to plan later tasks before the termination of its previously scheduled tasks.

All variants share the same rollback mechanism. If any phase fails, all states and attachment data created during the task are removed and the object is released back to the dependency tree for future reassignment.

Depending on the geometric constraints of the object, three variants of increasing complexity arise.

Variant 1: Unconstrained placement If the object can be freely moved through the workspace after grasping, the compound task consists of:

1. **Move to object** Plan a path to a grasp configuration on the object surface with unconstrained orientation
2. **Attach:** Establish the kinematic coupling between end-effector and object
3. **Move to goal** Plan a path to transport the object to its goal pose with $T_{\text{goal}}^{\text{EE}} = T_{\text{goal}}^{\text{obj}} \cdot (T_{\text{EE}}^{\text{obj}})^{-1}$
4. **Detach:** Terminate the kinematic coupling
5. **Exit** Move the robot outside the active workspace

This variant applies to objects that are not geometrically interlocked with their surroundings.

Variant 2: Constrained extraction Objects that are interlocked with adjacent parts must first follow a precomputed removal path before they can be freely transported. The compound task extends Variant 1 by inserting a follow path phase, Alg. 8, between attach and move-to-goal task:

1. Move to object
2. Attach
3. **Follow removal path** (Alg. 8): Advance through $T_1, \dots, T_m$ using iterative IK with time scaling.
4. Move to goal
5. Detach
6. Exit

Variante 3: Constrained extraction and insertion In certain configurations, the goal position is itself enclosed by surrounding geometry, requiring the object to follow a specific insertion trajectory to reach its final pose. The insertion path $T'_1, \dots, T'_k$ is the geometric counterpart of the removal path at the goal location. The compound task replaces the move-to-goal phase with a combination of free-space planning and constrained path following:

1. Move to object
2. Attach
3. Follow removal path $T_1, \dots, T_m$
4. **Move to insertion start** (Alg. 5): Plan a path to the configuration corresponding to T'_1 .
5. **Follow insertion path** (Alg. 8): Advance through $T'_1, \dots, T'_k$ to guide the object into its goal position.
6. Detach
7. Exit

Both the removal and insertion paths are precomputed from the assembly geometry and provided as input. The follow path procedure is identical for extraction and insertion. The directionality is encoded entirely in the waypoint sequence.

The pseudocode for the compound pick-and-place task is described in Alg. 9. First, the current number of robot states (Line 1) and attachData (Line 2) are stored in case a task fails and the initial state has to be restored. This rollback and return failure behavior is repeated after every task is ran (Line 4–5, 9–10, 14–15, 17–18, 22–23, 26–27).

The path to approach the object for the robot is calculated (Line 3). After the robot plans to the grasp position, the attachData is stored (Line 6). If the object has an associated removal path (Line 7), the robot follows the path specified (Line 8). If the object has an insertion associated (Line 11), the desired end-effector transformation is calculated from the start of the path and the relative transformation between end-effector and object (Line 12) instead of the specified target (Line 14). The robot then moves to the calculated position (Line 15). If an insertion path is specified the robot follows the waypoints (Line 19). After the goal transformation is achieved, the robot detaches the object (Line 22) and leaves the area to make room for other robots (Line 23).

The algorithm signals success after all subtasks are completed (Line 26).

Algorithm 9: Pick and Place Task

Input: robot, object, T_{goal} , t_{pick} , t_{place}
Output: success

1. $n_s \leftarrow |\text{robot.states}|$ // Rollback checkpoint
2. $n_a \leftarrow |\text{object.attachData}|$
3. **if** *not* $\text{moveToObject}(\text{robot}, \text{object}, t_{\text{pick}})$ **then**
4. rollback(n_s , n_a)
5. **return** failure
6. attach(robot, object)
7. **if** $\text{object.removalPath} \neq \emptyset$ **then**
8. **if** *not* $\text{followPath}(\text{robot}, \text{object.removalPath})$ **then**
9. rollback(n_s , n_a)
10. **return** failure
11. **if** $\text{object.insertionPath} \neq \emptyset$ **then**
12. $T_{\text{goal}} \leftarrow T_1' \cdot (T_{\text{EE}}^{\text{obj}})^{-1}$
13. **else**
14. $T_{\text{goal}} \leftarrow T_{\text{goal}} \cdot (T_{\text{EE}}^{\text{obj}})^{-1}$
15. **if** *not* $\text{moveToGoal}(\text{robot}, T_{\text{goal}}, t_{\text{place}})$ **then**
16. rollback(n_s , n_a)
17. **return** failure
18. **if** $\text{object.insertionPath} \neq \emptyset$ **then**
19. **if** *not* $\text{followPath}(\text{robot}, \text{object.insertionPath})$ **then**
20. rollback(n_s , n_a)
21. **return** failure
22. detach(robot, object)
23. **if** *not* $\text{exit}(\text{robot})$ **then**
24. rollback(n_s , n_a)
25. **return** failure
26. **return** success

Variants 1 and 2 are special cases of variant 3 with an empty removal and insertion path. The algorithm pseudocode handles all three cases with conditional branches on `removalPath` and `insertionPath`.

4.5 Grasp Pose Generation

A grasp can occur on any point on the surface of the object since the end-effector is modeled as an omnidirectional gripper, as described in the third assumption in Sec. 4.1. The grasp pose generation produces a set of possible candidates in $\mathcal{Q}$ for the motion planner. These candidates are generated by sampling contact points on the surface of the object and then used as the target for inverse kinematics.

4.5.1 Surface Sampling

A point on the object surface serves as the target position for the end-effector.

The object surface is considered to be described as a mesh. Each mesh consists of vertices, edges and faces. Vertices describe positions in the space. Edges are connections between vertices. A closed set of edges is called a face. In this case, the faces are considered to be triangle faces, resulting in a triangle mesh.

For mesh objects, a face is selected with probability proportional to its area to ensure a uniform distribution of sample points across the surface. The cumulative area distribution over all faces is computed and a face is selected via inverse transform sampling.

$$A_i = \frac{1}{2} \|(v_1 - v_0) \times (v_2 - v_0)\| \quad (4.7)$$

The probability of selecting a face i from a mesh with N faces

$$P(i) = \frac{A_i}{\sum_{j=1}^N A_j} \quad (4.8)$$

Then a uniform sample from the distribution is used to select a face.

Within the selected face, a point is sampled using barycentric coordinates with two uniform random variables $r_1, r_2 \in [0, 1]$:

$$\mathbf{p} = (1 - r_1)\mathbf{v}_0 + r_1(1 - r_2)\mathbf{v}_1 + r_1r_2\mathbf{v}_2 \quad (4.9)$$

where $\mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_2$ are the faces' vertices. The result is transformed to world coordinates using the object's current pose.

4.5.2 Goal State Generation

Each surface sample provides a target position for the inverse kinematic problem of the end-effector. Since the vacuum gripper can approach from any orientation, the orientation is not constrained during solving and a random orientation is sampled for each IK query. The inverse kinematics solver uses different restarts with random initial guesses sampled uniformly within the robot's configuration space $\mathcal{Q}$.

Not every IK solution represents a valid grasp configuration. A candidate is only accepted if it passes a future collision check with Alg. 6. This prevents selecting grasp poses that are feasible but invalidated by other robots' trajectories at different time points.

The goal state generation pseudocode is described in Alg. 10. First, an empty goalStates set is initialized (Line 1). Then, the algorithm tries up to a certain number of tries (Line 2) to generate at most a specified number of possible goal states (Line 3) and return early if the amount is met (Line 4). A point on the object is sampled (Line 5) and used as the target position for the inverse kinematics problem (Line 6). If no valid configuration can be found (Line 7), the next iteration begins (Line 8). When a valid configuration is found, it is only kinematically feasible and has to be checked to be free in the future (Line 9). After the state was validated it is added to the set of possible goal states for the planner (Line 11). The algorithm returns the set of all found goal state candidates (Line 12).

Algorithm 10: Goal State Generation

Input: robot, object
Output: goalStates

1. goalStates $\leftarrow \emptyset$
2. **for** $i \leftarrow 0$ **to** *maxTryNumber* **do**
3. **if** $|goalStates| \geq maxNumGoals$ **then**
4. **return** goalStates
5. surfacePoint $\leftarrow$ sampleSurface(object)
6. $q \leftarrow$ solveIK(robot, surfacePoint)
7. **if** $q = \emptyset$ **then**
8. **continue**
9. **if not** checkFutureCollision(q) **then**
10. **continue**
11. append(goalStates, q)
12. **return** goalStates

4.6 Setting the Simulation State

It is required to set the simulation state to query the mandatory information, like collision occurrences, object distances etc. Moreover, the state of each robot and object has to be queryable at a continuous time point. However, the robot's path is stored as a trajectory, which are discrete configurations at specific time points.

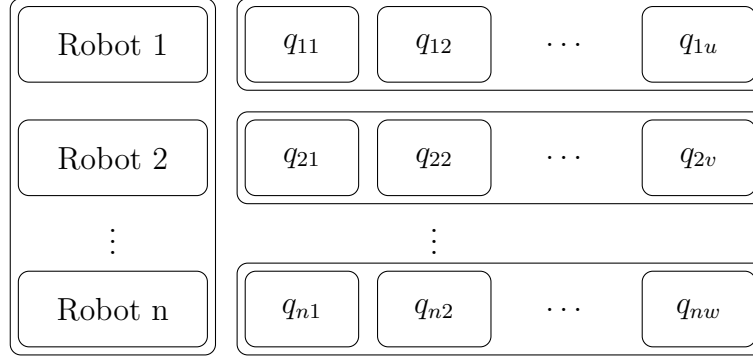


Figure 4.10: Storage representation of robot paths

Since objects can be attached to different robots at different time points, information about the attachment time, detachment time and relative transform between end-effector and object has to be saved to restore the complete state at any time point. Attachment data is stored in chronological order of occurrence.

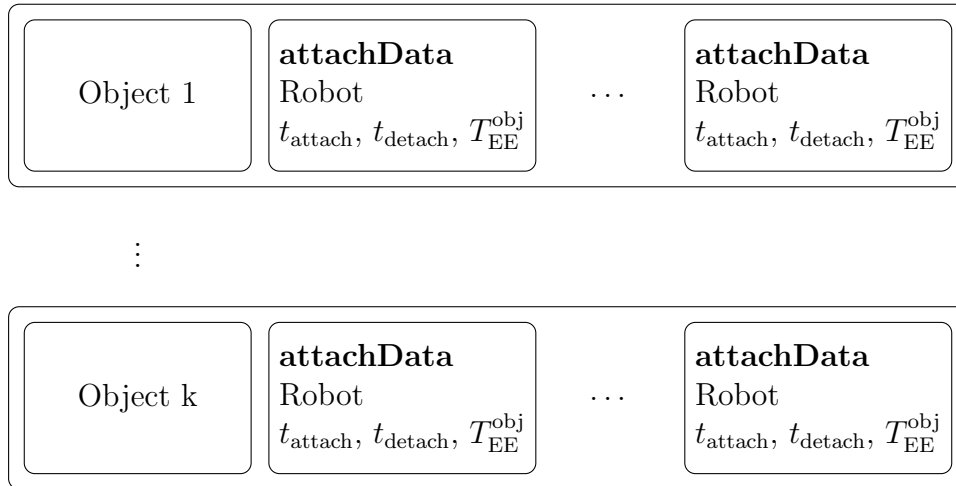


Figure 4.11: Storage representation of attachment data

The pseudocode to find the attachData at a specific time point is described in Alg. 11. First, the associated data storage is selected (Line 1). It then verifies that the object was actually moved (Line 2). Since the data is stored in chronological order, the first attach time is used as an early return to indicate that the object is still in its initial pose (Line 3–5). The same behavior is true for every time point after the last detach time point. The last detach time (Line 6) is compared against the timepoint (Line 7) to determine if an early return (Line 8) is feasible. Both of the early returns return their attachData as a comparison against the timepoint to allow to check if the object should be placed in the start or goal pose.

If neither condition is met, a linear sweep across the data (Line 9) is combined with a range check on the time point with a time window, consisting of the attach and detach time (Line 10). The relevant attachData is then finally returned (Line 11).

Algorithm 11: Find AttachData

Input: object, t_{at}
Output: relevantAttachData

1. $attachData \leftarrow \text{getAssociatedAttachData}(\text{object})$
2. **assert** $attachData \neq \emptyset$
3. $firstAttachTime \leftarrow attachData.front().attachTime$
4. **if** $t_{at} < firstAttachTime$ **then**
5. $\lfloor$ **return** $attachData.front()$
6. $lastDetachTime \leftarrow attachData.back().detachTime$
7. **if** $t_{at} > lastDetachTime$ **then**
8. $\lfloor$ **return** $attachData.back()$
9. **foreach** $data \in attachData$ **do**
10. **if** $t_{at} > t_{attach} \wedge t_{at} < t_{detach}$ **then**
11. $\lfloor$ **return** data

The position of the object in the world is calculated by the end-effector world transform and object end-effector transform.

$$T_{obj} = T_{EE} \cdot T_{EE}^{obj} \quad (4.10)$$

This is done to avoid tracking the position of the object directly without drastically increasing the time step at which it is stored. The simulation has to be able to be set on a continuous timescale. If the object is only tracked at discrete time steps and interpolated between them, a discretization error will occur. Therefore, only the object to end-effector transformation is saved when a robot attaches to the object. To get the end-effector world transform forward kinematics are performed.

The pseudocode to set the simulation state is described in Alg. 12. First, the algorithm iterates over each object to set their pose (Line 1). The relevant attachData for each object is located (Line 2), as outlined in Alg. 11. The time step is categorized as either prior to, within or subsequent to the attachment data (Line 4). If the time step is prior to the first attachment data, the object has not yet been moved and remains at the starting position. If the time step falls within the time period of an attachment, the time itself is utilized to set the pose of the robot. Conversely, if the time step is larger than the last time period, the detach time of the last attachment data is used for the pose. The robot's states which lie right before and after the time point are identified (Line 5). Then, the state is interpolated (Line 6) and the robot's configuration is set (Line 7) for the time point. This is done, so the end-effector transformation can be acquired (Line 8). This is used with the relative end-effector-object transformation (Line 9) to reconstruct the object's world transformation (Line 10). The object is then set to that pose (Line 11).

After each object is placed to the correct transformation, each robot can be set (Line 12). Now the input time is used to determine the bounding states (Line 13) and interpolation (Line 14) like before. Then the robot is set to the interpolated configuration (Line 15).

Algorithm 12: Set Simulation State

Input: t_{at}

1. **for** $obj \in objects$ **do**
2. $data \leftarrow \text{findAttachData}(obj, t_{at})$
3. $robot \leftarrow \text{getRobot}(data)$
4. $time \leftarrow \text{determineTimePoint}(data, t_{at})$
5. $q_{lower}, q_{upper} \leftarrow \text{findBoundingStates}(robot, time)$
6. $q_{inter} \leftarrow \text{interpolate}(q_{lower}, q_{upper}, time)$
7. $\text{setConfiguration}(robot, q_{inter})$
8. $eeTF \leftarrow robot.eeTF$
9. $eeToObjTF \leftarrow data.eeToObjTF$
10. $objTF \leftarrow eeTF * eeToObjTF$
11. $\text{setPose}(obj, objTF)$
12. **for** $robot \in robots$ **do**
13. $q_1, q_2 \leftarrow \text{findBoundingStates}(robot, t_{at})$
14. $q_{inter} \leftarrow \text{interpolate}(q_1, q_2, time)$
15. $\text{setConfiguration}(robot, q_{inter})$

After the simulation state is set, the desired information, such as collision detection, distance queries etc., can be queried.

4.7 Implementation Details

This section discloses implementation details of the approach for the experimental section of this thesis. First, the simulation setup is disclosed, including software and robots used. Afterwards, the format of the input to the algorithm is specified. Then, algorithm implementation details regarding assignment policy, motion planning and collision checking are explained.

4.7.1 Simulation Environment

As a base for the thesis, the *Dynamic Animation and Robotics Toolkit* (DART) is used for visualization, inverse kinematics and collision checking [73].

The robot model is a Franka Emika Panda manipulator mounted on a holonomic mobile base. The Franka Emika Panda is a collaborative robot with seven degrees of freedom, which is an $\mathbb{R}^7$ state space. The mobile base is an unspecified, generic base with an SE(2) state space. This robot has the compound state

$$\mathbf{q} = [x \ y \ \theta \ q_1 \ \dots \ q_7]^\top \quad (4.11)$$

where $q_1, \dots, q_7$ represent the joint angles of the arm and x, y, θ the Cartesian coordinates and rotation of the base.

Time t is added to the state since the planner will be using space-time planning. Resulting in the final space

$$\mathcal{Q} = \text{SE}(2) \times \mathbb{R}^7 \times \mathbb{R}_{\geq 0} \quad (4.12)$$

with the state

$$\mathbf{q} = [x \ y \ \theta \ q_1 \ \dots \ q_7 \ t]^\top \quad (4.13)$$

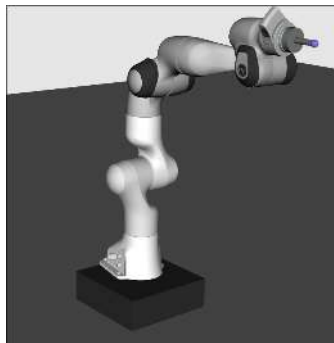


Figure 4.12: Franka Emika Panda with SE2 base in DART simulation

4.7.2 Input File Format

As the input to the algorithm a YAML configuration file is passed. Lst. 4.1 shows an example configuration file. All entries in the configuration have a name, start pose/configuration and a file path to their **Universal Robot Description Format** (URDF). First, are the available robots defined with their type. This thesis only implements a homogeneous set of robots, consisting of n Franka Emika Pandas of the same kinematic model, but any heterogeneous set of different robots can be implemented. This is because the algorithm does not assume anything about the robot except that it has a definition of an end-effector and has an IK solver for its pose. Then all objects are further defined by their dependency list and desired goal pose.

Listing 4.1: Example scenario configuration.

```

1 robots:
2   - type: panda_mobile
3     name: Robot1
4     path: "path_to_my_panda.urdf"
5     start_config: [0.5, -1.5, 0, 0, 0, 0, -1.57, 0, 3.14, 0]
6 objects:
7   - name: "obj1"
8     dependency: ["obj4"]
9     path: "path_to_my.urdf"
10    start_pos: [0.1, 0.1, 0.1, 0, 0, 0]
11    end_pos: [1.0, 0, 0, 1.57, 0, 0]
12   - name: "obj2"
13     dependency: ["obj4", "obj5"]
14     path: "path_to_my.urdf"
15     start_pos: [0.2, 0.1, 0.1, 0, 0, 0]
16     end_pos: [2.0, 0, 0, 1.57, 0, 0]
17   - name: "obj3"
18     dependency: []
19     path: "path_to_my.urdf"
20     start_pos: [0.3, 0.1, 0.1, 0, 0, 0]
21     end_pos: [3.0, 0, 0, 1.57, 0, 0]
22   - name: "obj4"
23     dependency: []
24     path: "path_to_my.urdf"
25     start_pos: [0.4, 0.1, 0.1, 0, 0, 0]
26     end_pos: [4.0, 0, 0, 1.57, 0, 0]

```

4.7.3 Assignment Policy

The assignment policy in Alg. 4 is responsible for pairing the available robots with the current leaf nodes of the dependency graph.

This decision can be purely random, based on a heuristic, like distance between robot and object, or based on a more complex algorithm. While more advanced strategies like load balancing or spatial proximity can lead to lower planning cost, the random assignment combined with the failure retry mechanism provides a simple but reliable strategy. Heuristics without randomness might get stuck since they will reselect the same robot and object pairing each time.

This work implements a purely random selection. This acts as a baseline for further exploration of different policies without influencing the evaluation of the high-level algorithm.

4.7.4 Motion Planning

Three different tasks need to plan a trajectory: move, follow path and exit.

The follow path task uses DARTs implementation for inverse kinematics to implement its algorithm as described in Alg. 8.

The move and exit task can be regarded as the nearly the same problem. The main difference is that the exit task has no fixed goal pose, rather a safe region it has to reach. For all experiments the workspace $\mathcal{W}$ for the mobile base is defined as an Axis Aligned Bounding Box (AABB). Both pick and place regions are surrounded by an unsafe exit region to avoid obstruction of other robots and keep the direct path between both regions free.

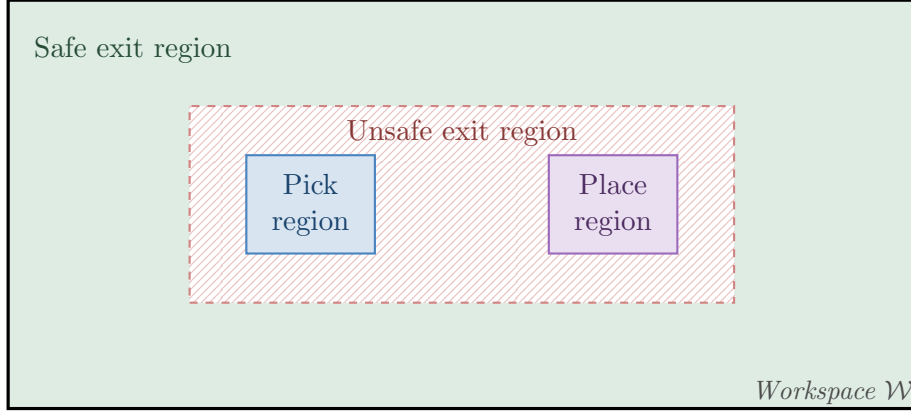


Figure 4.13: Visualization of exit regions

Both tasks are implemented as sampling-based planners from the Open Motion Planning Library (OMPL) [74] for solving this problem. Both RRT* [17] and ST-RRT* [28] are used with an optimization objective to minimize the arrival time. The arrival time is the last time point of a trajectory for one planner query and is not the same as the makespan, which is the total time needed to complete the disassembly process, introduced in the problem statement in Sec. 4.2.1. A maximum computation time of $t_{\text{timelimit}}$ is used as a termination condition for the planner. After a solution is found shortcutting and B-spline smoothing are applied to the path.

The planner used in the experiments in Ch. 5 is ST-RRT*. The planner was selected to leverage the additional information provided from the time dimension. This includes the exploration of different time windows to find a better solution for the makespan by exploiting the provided trajectories of other robots and objects. Additionally, all problems have no defined upper time bound since the problem difficulty and expected solution depends strongly on the previously planned trajectories' solution quality. Furthermore, the time bound depends on the number of robots and objects. This leads to excessive hyperparameter tuning for each scenario, with even more tuning for different robot counts within the scenario, making it infeasible in practice. ST-RRT* avoids this problem due to its dynamic time bound, which allows for flexibility in the time domain, while keeping the time bound as tight as possible. Another benefit of ST-RRT* comes from the handling of the temporal dependency constraint propagation from Sec. 4.2.6. These constraints can be used as upper or lower time bounds in the dynamic expansion of ST-RRT* goal time selection.

The performance of ST-RRT* is compared against RRT*, which was adapted to work in space-time in Sec. 5.9.

4.7.5 Collision Checking

All planners need to check if a certain configuration is collision free at specific time points in the simulation. The pseudocode for collision checking is described in Alg. 13. First, the associated time with the state is acquired (Line 1). Then, the simulation state is set with Alg. 12 (Line 2). Then the new configuration is set (Line 3). Since it is unknown if an object is attached at that time point to the robot, each object (Line 4) is checked if it is attached to the desired robot. First, the relevant attachment data for that time point is searched with Alg. 11 (Line 5). If the robot associated with the attachData matches the selected robot (Line 6), the current end-effector pose is saved (Line 7) and combined with the saved relative transform (Line 8) between object and robot from the attachData to the actual pose of the object at that time point (Line 9). Then the pose of the object is set (Line 10) and the loop exits early (Line 11). After each object and robot is set, the algorithm returns if a collision is detected by DARTs collision detector (Line 12).

Algorithm 13: Collision Check

Input: $q_{\text{new}}, robot_{\text{selected}}$
Output: `simInCollision`

1. $t_{\text{at}} \leftarrow \text{getTime}(q_{\text{new}})$
2. `setSimulationState( $t_{\text{at}}$ )`
3. `setConfiguration( $robot_{\text{selected}}, q_{\text{new}}$ )`
4. **for** $o_j \in \mathcal{O}$ **do**
5. $\text{data} \leftarrow \text{findAttachData}(o_j, t_{\text{at}})$
6. **if** $o_j.\text{robot} == robot_{\text{selected}}$ **then**
7. $\text{eeTF} \leftarrow \text{robot.eeTF}$
8. $\text{eeToObjTF} \leftarrow \text{data.eeToObjTF}$
9. $\text{objTF} \leftarrow \text{eeTF} * \text{eeToObjTF}$
10. `setPose(obj, objTF)`
11. **break**
12. **return** `simInCollision()`

Alg. 12 needs to be able to interpolate between two discrete states at a continuous level.

The interpolation in Alg. 14 takes the two bounding states of the time point and the time point itself as an input.

First, the time components of the state are saved (Line 1–2). Then a safety check for small time deltas is performed (Line 3). If the time delta is small enough, the

first state is returned (Line 4). This is done to avoid the explosion of the denominator of c (Line 6). After the safety check, the time point is verified to lie between the time component of the states (Line 5). After the verification, the constant c is calculated, which describes the time distance between q_1 and q_2 on a scale from 0–1 (Line 6). The new interpolated x and y component is calculated by the first state q_1 and the scaled difference between both states (Line 7–8). The rotation difference of the mobile base has to be handled separately to handle the non-euclidean space (Line 9–10). Afterwards, it can calculate the new rotations with the same approach as before (Line 11). Each joint of the robot arm (Line 12) can be interpolated by adding the difference between the joint states and scaling them with the constant c to the base joint state (Line 13). Finally the time component is set to the time point (Line 14) and the interpolated state is returned (Line 15).

Algorithm 14: Interpolation

Input: q_1, q_2, t_{at}
Output: q_{inter}

1. $t_1 \leftarrow \text{GetTime}(q_1)$
2. $t_2 \leftarrow \text{GetTime}(q_2)$
3. **if** $|t_2 - t_1| < \varepsilon$ **then**
4. $\quad$ **return** q_1
5. **assert** $t_1 \leq t_{\text{at}} \leq t_2$
6. $c \leftarrow \frac{t_{\text{at}} - t_1}{t_2 - t_1}$
7. $q_{\text{inter}}.x \leftarrow q_1.x + (q_2.x - q_1.x) \cdot c$
8. $q_{\text{inter}}.y \leftarrow q_1.y + (q_2.y - q_1.y) \cdot c$
9. $\Delta\theta \leftarrow q_2.\theta - q_1.\theta$
10. $\Delta\theta \leftarrow \Delta\theta - 2\pi \cdot \text{round}\left(\frac{\Delta\theta}{2\pi}\right)$
11. $q_{\text{inter}}.\theta \leftarrow q_1.\theta + \Delta\theta \cdot c$
12. **for** $i = 1, \dots, 7$ **do**
13. $\quad$ $q_{\text{inter}}.\text{joint}_i \leftarrow q_1.\text{joint}_i + (q_2.\text{joint}_i - q_1.\text{joint}_i) \cdot c$
14. $q_{\text{inter}}.t \leftarrow t_{\text{at}}$
15. **return** q_{inter}

5 Results

This chapter evaluates the algorithm proposed in Ch. 4. First, benchmarking is performed on six different disassemblies. Then a failure analysis is performed in Sec. 5.8 to identify which subtask limits the algorithm’s performance. Finally, a comparison between ST-RRT* and RRT* is conducted in Sec. 5.9.

5.1 Benchmark Setup

The algorithm is evaluated on six scenarios, which are introduced in Sec. 5.2 to 5.7. Each scenario stresses a different combination of dependency depth, execution order and geometric complexity. Ranging from the strictly sequential tower problem to the deeply interleaved gearbox disassembly. The scenarios are ordered by increasing geometric and dependency complexity, starting from synthetic tests and progressing toward real-world inspired disassembly problems with interlocking geometry.

For every scenario, the number of robots is varied to assess scalability. For each combination of scenario and robot count, independent runs are performed with different random seeds to account for the stochastic nature of the sampling-based planner and the random robot-object assignment in Alg. 4. All robots use the kinematic model introduced in Sec. 4.7.1 and are initialized in distinct, collision-free home configurations distributed around the workspace.

For each scenario, the same set of graphs is reported to allow comparison across benchmarks. Makespan, planning time, makespan speedup, calculation time increase and the Pareto front are reported for different numbers of robots. The planner performance is evaluated with a success rate and cost decay plot over all objects and robot counts. Lastly an example timeline plot is reported.

Subsequent scenarios refer to these conventions without repeating them.

Execution and Planning Time

The execution and planning time comparison reports the associated time per run over a number of robots. The makespan captures the total time required to complete the planned disassembly sequence. The planning time captures the cumulative time consumed across all queries needed to create the sequence. The median is reported with the shaded bands denoting the interquartile range across runs. The makespan is expected to decrease, while the execution time per task is expected to grow as individual assignments take place in an increasingly populated workspace. The calculation time is expected to grow with the number of planner queries that have to be coordinated.

Makespan Speedup and Computation Time Increase

The makespan speed up graph displays the relationship between the different number of robots compared to the one robot baseline. An ideal linear scaling line is added for comparison. The computation time increase compares the calculation time of the different number of robots in relation to the one robot baseline. The solid line is the median, while the shaded bands denote the interquartile range across runs.

Pareto Front

The Pareto front characterizes the trade-off between execution performance and planning effort across robot configurations. The horizontal axis denotes the median makespan in seconds, and the vertical axis denotes the cumulative calculation time in seconds. Each individual run is rendered as a faint marker. The median across runs for each robot count is rendered as a labelled dot, with the label indicating the number of robots used in that configuration. The Pareto front connects the dominant configurations and traces the boundary along which a reduction in makespan can only be achieved by accepting an increase in calculation time.

Success Rate

The success rate plot reports the cumulative fraction of planner queries that have produced a solution at that time point. Each pick-and-place task is separated into its motion phases. Dashed markers on each curve indicate the elapsed time at which 90%, 95%, 99%, and 100% of queries have found their initial solution. Since

the per-task planning time limit is set to ten seconds, all curves are bounded by this limit.

Cost Decay

The cost decay plot reports the median cost with a shaded interquartile range over the planning time. The three different planning phases include different objects and therefore converge to different values, resulting in a wide interquartile band. The green dashed marker indicates the elapsed time at which 50% of queries have produced an initial solution. The red dashed marker indicates the elapsed time at which the median cost has converged to less than 1% improvement. The absence of a red marker indicates that the planner does not exhibit a measurable decay over time.

Timeline

The example timeline shows a single representative run of the scenario at the maximum robot count. The horizontal axis denotes simulation time in seconds and each row corresponds to one robot’s schedule. Each assignment is rendered as a horizontal bar segmented into the three motion phases. The label above each bar identifies the object handled by that assignment. The timeline is intended as a qualitative complement to the previous figures. It showcases how individual robots interleave along the time axis, how phases overlap across robots and where idle gaps or sequential dependencies become visible in the schedule. A single representative run is picked at random.

5.1.1 Benchmark Hardware

All runs were performed on a Dell XPS 15 9530 laptop, equipped with an Intel Core i7-13700H processor. Each trial is pinned on performance core 0 of the CPU with a stable clock speed between 4.7 and 4.8 GHz under sustained load. Runs were executed sequentially since the CPU frequency varied for each core under load if multiple instances ran in parallel. This ensures comparable results within the benchmarks.

5.2 Tower Problem

The tower scenario consists of ten identical cubes stacked vertically. During the placement of the objects, the order of the cubes has to be reverted, while the pick and place phases of the parts have to be performed in strictly sequential order. The resulting dependency graph is a single chain, with each cube depending exclusively on the one above it. The tower is used to analyze the contribution of the temporal constraint propagation introduced in Sec. 4.2.6. A successor robot without time constraint propagation must wait until its predecessor has fully placed its cube and exited the workspace. This reduces the system's performance to that of a single robot regardless of the number of robots. However, with propagation, a successor can begin planning its approach as soon as the predecessor has finished planning the attachment or detachment. This allows assignments to be interleaved along the time axis. This scenario highlights the importance of spatial coordination among the robots, as they all share the same vertical column and a small goal region. When multiple robots converge on a confined workspace, they have to rely on the space-time collision checker to coordinate their trajectories rather than spreading out spatially to avoid collision.

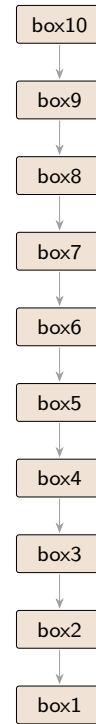
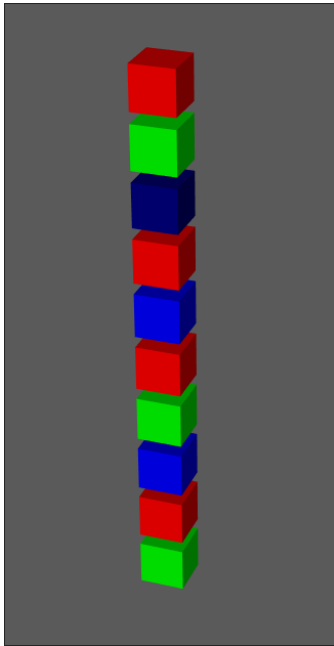


Figure 5.1: Tower scenario and its dependency structure

Table 5.1: Object composition of the tower scenario

Component Group	Geometry	Count
Cube	Primitive	10
Total		10

Execution and Planning Time

Fig. 5.2 reports the median makespan and cumulative calculation time for different numbers of robots. The single robot baseline for the tower scenario calculates a makespan of approximately 94 seconds with a calculation time of 320 seconds. Adding robots reduces this monotonically, reaching a minimum of 19 seconds with ten robots. This corresponds to an 80% reduction, or fivefold improvement compared to the baseline. The reduction saturates at around five robots with a makespan of 25 seconds, after which the graph flattens. Increasing the number of robots further reduces the makespan by approximately 1 second per robot.

The calculation time remains consistent at around 320 seconds and starts to increase after 4 robots to a calculation time increase factor of up to 1.15. Due to the limited sample size, the median fluctuates significantly.

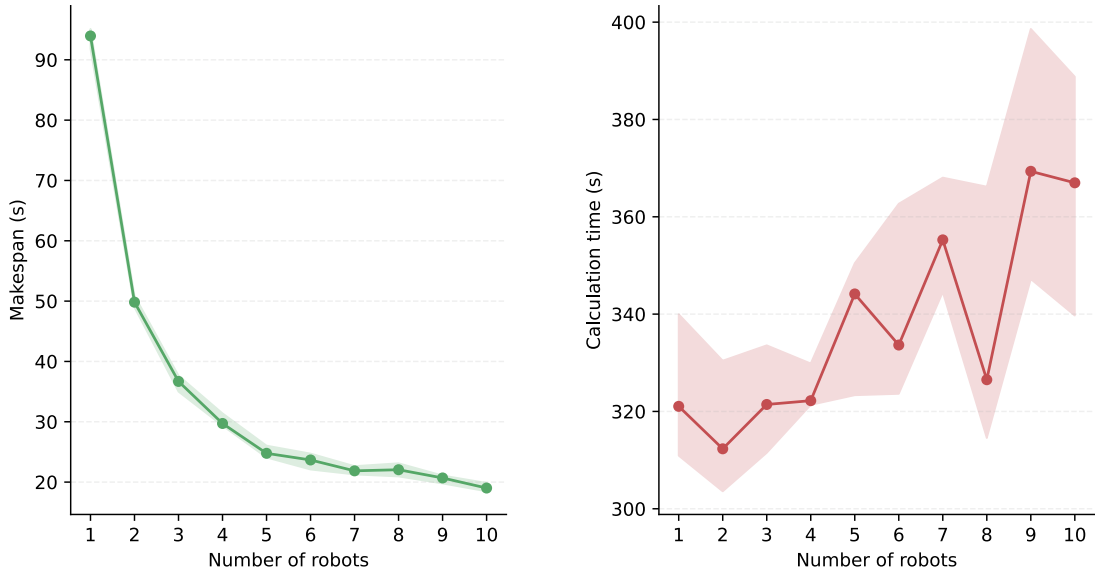


Figure 5.2: Execution and planning time comparison for the tower scenario

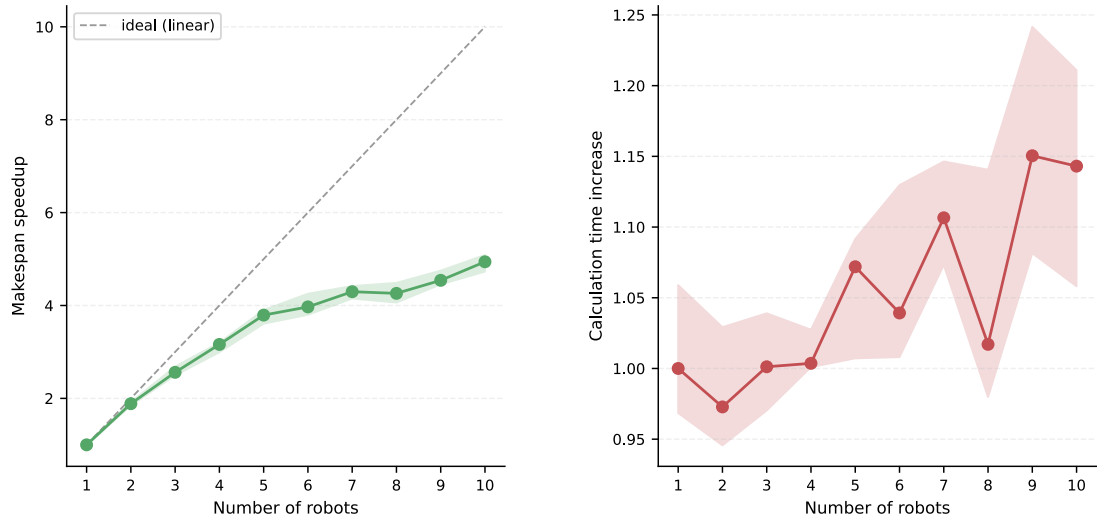


Figure 5.3: Speedup factor and calculation time increase for the tower scenario

Fig. 5.4 places the different number of robots in the makespan-calculation time plane. The Pareto front consists of two, three, four, eight, seven and ten robots. Both dimensions feature a long, shallow decline. The makespan drops rapidly from 50 to 20 seconds and the calculation time from 375 to 320 seconds.

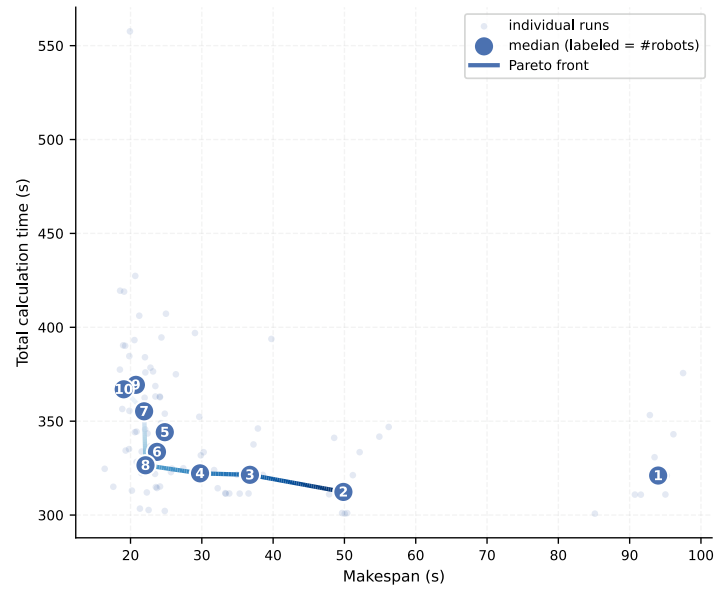


Figure 5.4: Pareto front of the makespan and calculation time for the tower scenario

Success Rate and Cost Decay

Fig. 5.5 shows the cumulative success rate and cost decay of all runs across different numbers of robots over time. It shows that the pick phase is the slowest of the three phases. It reaches 90% in 0.69 seconds, 95% in 1.83 seconds, 99% in 6.33 seconds and 100% in 9.14 seconds. The place phase reaches 90% in 0.47 seconds, 95% in 0.81 seconds, 99% in 2.93 seconds and 100% in 9.13 seconds. The exit phase converges almost immediately, reaching 90% in 0.07 seconds, 95% in 0.10 seconds, 99% in 0.18 seconds and 100% in 0.46 seconds.

All three phases reach 100% within the 10-second timeframe.

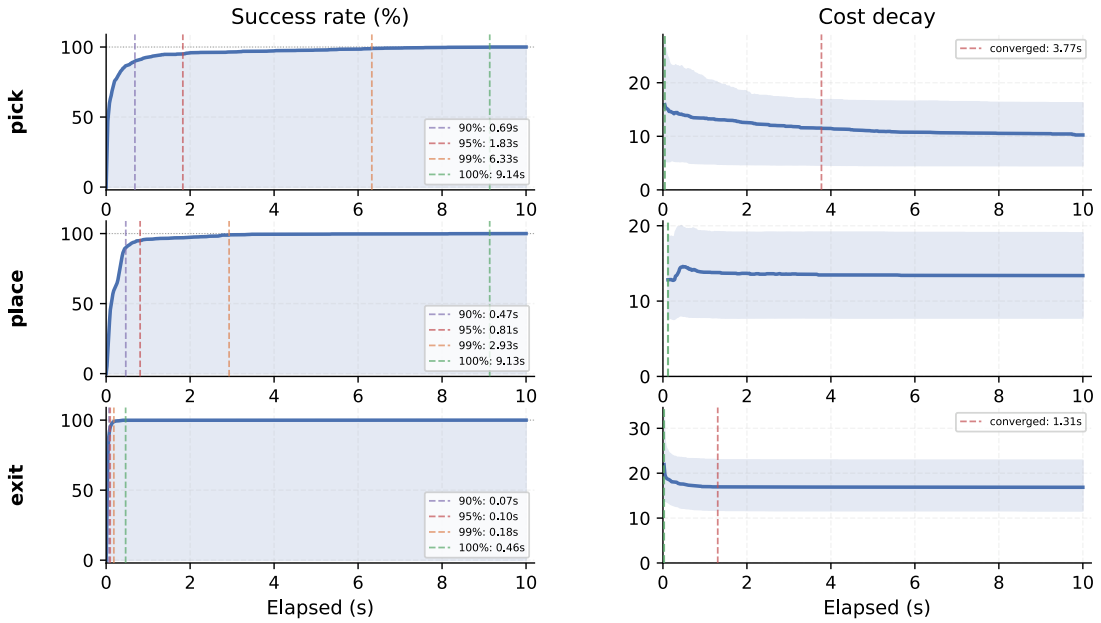


Figure 5.5: Planner success rate for the tower scenario

The pick phase exhibits the most pronounced decrease in cost. The median cost falls from around 16 at the first feasible solution to approximately 10 at the convergence point of 6.05 seconds. This results in an improvement of around 35%. Most of this gain occurs within the first two seconds, with the remaining time contributing only marginally to further reductions.

By contrast, the place phase does not produce a convergence marker. The median cost stays essentially constant at around 13 for most of the time. A sudden increase in solution cost occurs at the same time as a stronger increase in solution rate, as shown in Fig. 5.5.

The exit phase converges much earlier, at 1.57 seconds. The median drops from around 22 to 17 within the first second, resulting in an improvement of approximately 23%.

Therefore, the bulk of the per-task planning budget on the tower is spent refining the pick phase, while the place and exit phases either converge quickly or do not measurably benefit from additional time.

Timeline

Fig. 5.6 shows a representative execution timeline for the tower scenario with ten robots. The total makespan of this run is approximately 16.5 seconds, which is consistent with the median reported earlier. The temporal constraint propagation introduced in Sec. 4.2.6 can be seen in the schedule, since the dependency tree is a strict chain in which each cube depends on the cube above it. Successor robots begin their pick phase well before their predecessors have completed their placement, producing the heavily interleaved pattern observed across the rows. The pick segments are longer for robots that begin later in the run (e.g. Robot4, Robot5 and Robot9). The exit phases are of comparable in length for all robots, which is consistent with the rapid convergence observed in the success rate and cost decay plots for the exit phase. There are no large idle gaps in the schedule, indicating that the prioritized spacetime planner keeps the robots occupied throughout the run, despite the spatial bottleneck imposed by the shared vertical space.

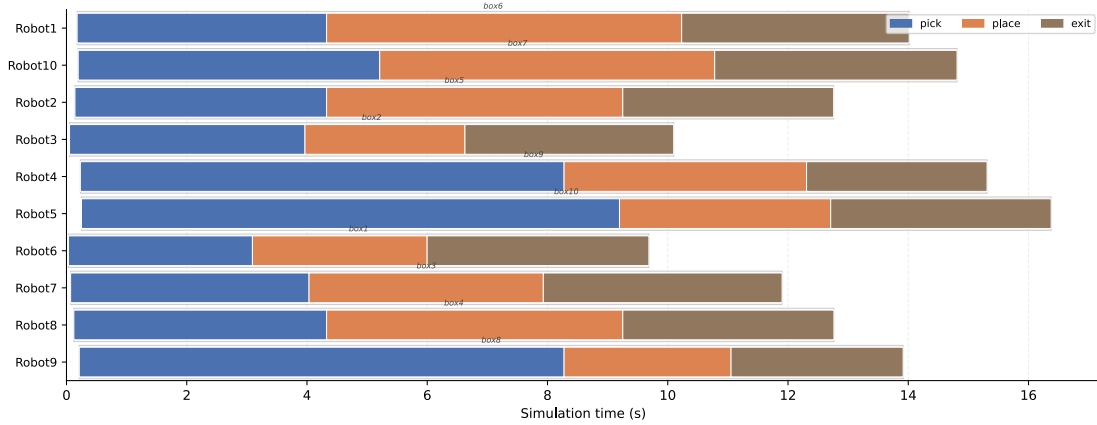


Figure 5.6: Example timeline of tower scenario with ten robots

Interpretation

The results confirm that the temporal constraint propagation works as intended in the sequential case, achieving a fivefold increase in performance compared to the baseline. Without propagation, the chain dependency would force the system to behave in such a way that new plans could only be started after the predecessor's placement and exit were finished in order to guarantee the correct pick-and-place order.

Fig. 5.6 explicitly shows the successful coordination within the time domain. Boxes 1 to 7 can be picked up in quick succession, while boxes 8 to 10 are picked up at a later time. This is due to the spatial congestion around the tower's starting position. Once the initial batch of robots have left the area to place their object, the area becomes less occupied and the remaining robots can pick theirs.

This behavior also explains why the makespan flattens in Fig. 5.2 since the first robots start to become available again at around the same time. By the time the newly free robots want to pick up an object, the other occupied robots will have moved out of the starting area. Therefore, it is a simpler motion planning problem with fewer obstacles. However, the planner's ability to move closer to the cube while waiting still outperforms them consistently, albeit with an increasing path cost and calculation time.

The congested space also explains the longer calculation time before an initial solution can be found in Fig. 5.5. Although the placement phase has the same temporal constraints as the picking phase, it does not exhibit a slow down. This is because the picking phase acts as a buffer for the robots and acts like a temporal narrow passage, forcing the robots' movements to temporarily align with each other. Additionally, robots that have finished exit to a wide area of possible configurations and therefore do not obstruct the placement area as much.

The difference between the picking and placement phases of the planning problem can also be seen in Fig. 5.5, where the later initial solutions around the 0.5 second mark quickly converge.

5.3 Wall Problem

In the wall scenario, 40 bricks are arranged in a wall-like pattern, with each cube depending on the cubes directly above it. Unlike the tower, the dependency tree contains multiple branches. Therefore, several leaves are accessible at any time during the disassembly. The wall is used to analyze how well a task can be parallelized when the dependency structure permits it. While multiple cubes can be processed concurrently, the wall surface is a long, planar region and the robots may need to plan trajectories through overlapping regions in front of it. This scenario therefore quantifies how effectively the prioritized space-time planner exploits the temporal dimension to coordinate along this surface.

This scenario complements to the tower scenario, in which dependency depth is tested, rather than parallelization.

Table 5.2: Object composition of the wall scenario

Component Group	Geometry	Count
Brick	Primitive	40
Total		40

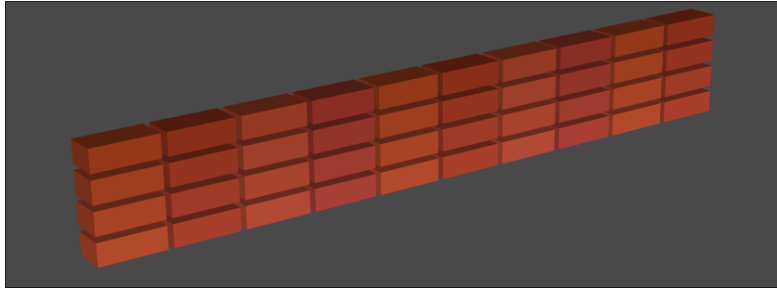


Figure 5.7: Wall scenario in simulation

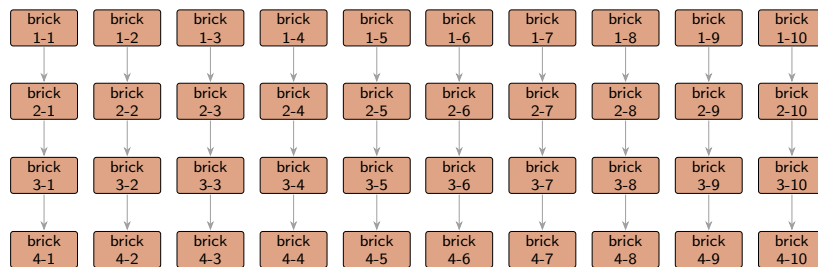


Figure 5.8: Wall dependency graph

Execution and Planning Time

The baseline for the makespan is a single robot in 470 seconds with a calculation time of 1290 seconds. Adding robots reduces the makespan monotonically to around 65 seconds with ten robots. This corresponds to an 86% reduction in makespan or a sevenfold speedup compared to the baseline. However, the calculation time increases to 1720 seconds with ten robots, which is an increase of 1.33.

The interquartile range of the makespan remains narrow throughout, whereas that of the calculation time fluctuates considerably.

The speedup factor in Fig. 5.10 tracks the ideal linear speedup factor closely until six robots, after which it starts to slow down.

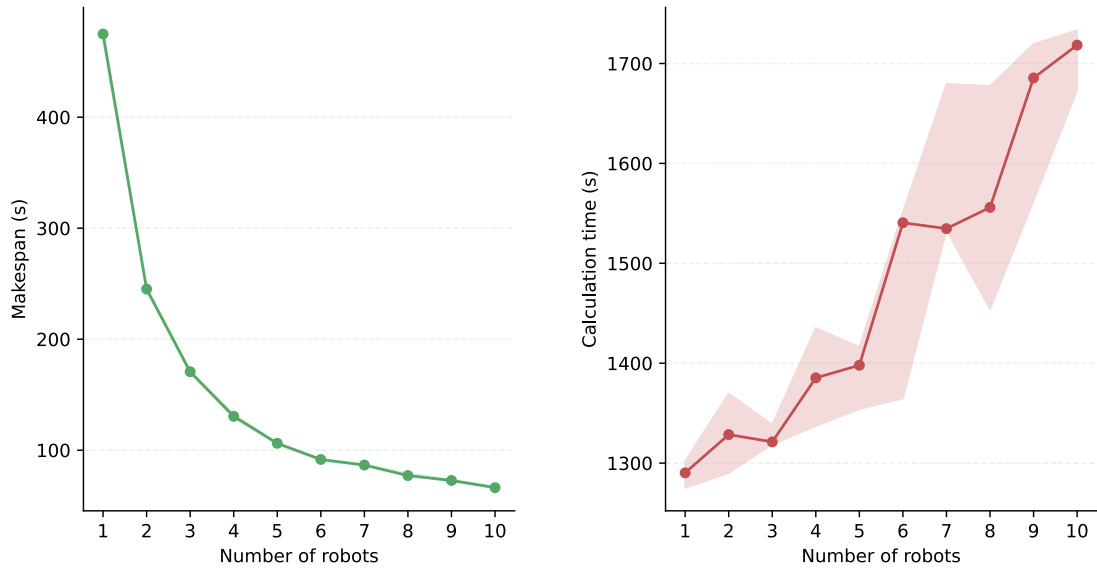


Figure 5.9: Makespan and calculation time for the wall scenario

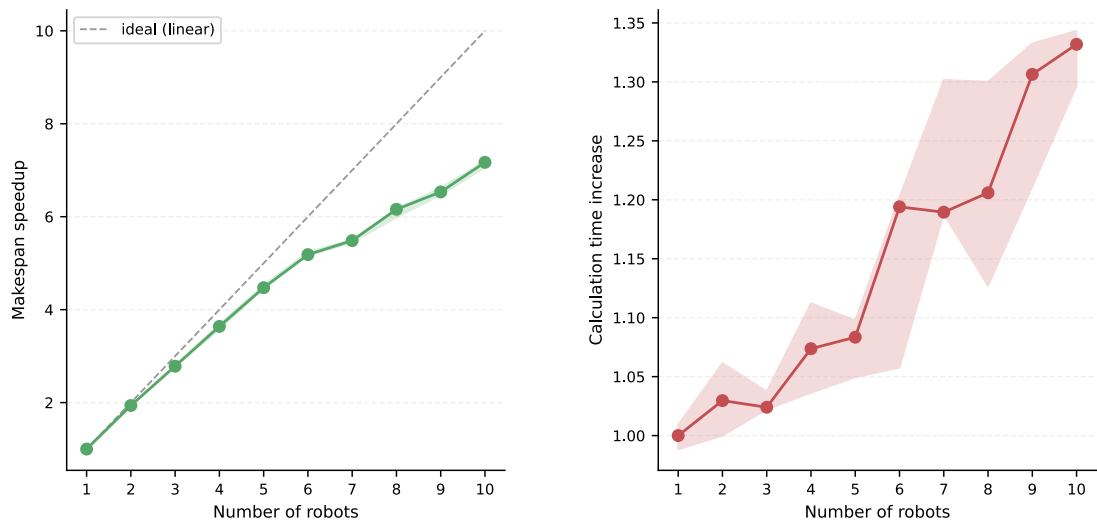


Figure 5.10: Execution speedup and calculation time increase for the wall scenario

The Pareto front in Fig. 5.11 consists of all but two and six robots. However, these two robot counts are close to the front.

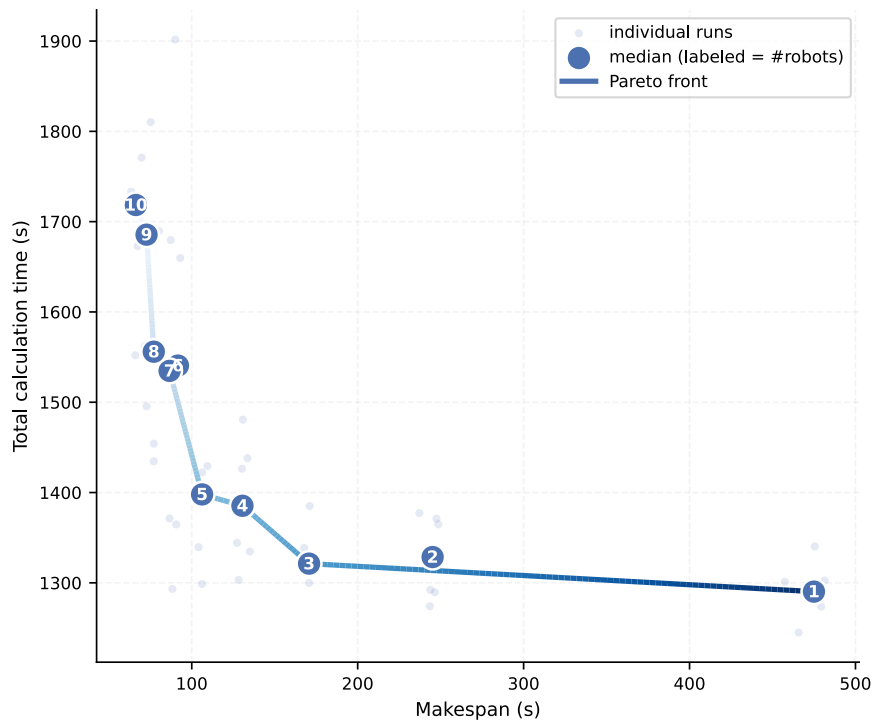


Figure 5.11: Pareto front for the wall scenario

Success Rate and Cost Decay

Fig. 5.12 shows that the pick phase takes the longest to find an initial solution of the three phases. It reaches 90% in 2.05 seconds, 95% in 3.83 seconds, 99% in 7.90 seconds and 100% in 10.04 seconds. The place phase is slightly faster, reaching 90% in 1.45 seconds, 95% in 2.68 seconds, 99% in 6.30 seconds and 100% in 9.73 seconds. The exit phase converges almost immediately, reaching 90% in 0.21 seconds, 95% in 0.27 seconds and 99% in 0.73 seconds, with 100% reached in 8.36 seconds.

The cost decay improves in the across all three phases.

The median of the pick phase falls from approximately 70 to 51 after 1.65 seconds, yielding an improvement of around 27%. The 75th percentile falls from approximately 130 to 85 in a long tail. The 25th percentile stays mostly at 35 with only marginal improvements.

The place phase median falls from around 65 to 56 in 1.85 seconds, with the 75th percentile decreasing from 130 to 90 in the slow decay similar to the pick phase, while the 25th percentile stays consistent throughout.

The exit phase median stays close to 60, with the 75th percentile decreasing to 95 after two seconds and 25th percentile consistent at 40 throughout.

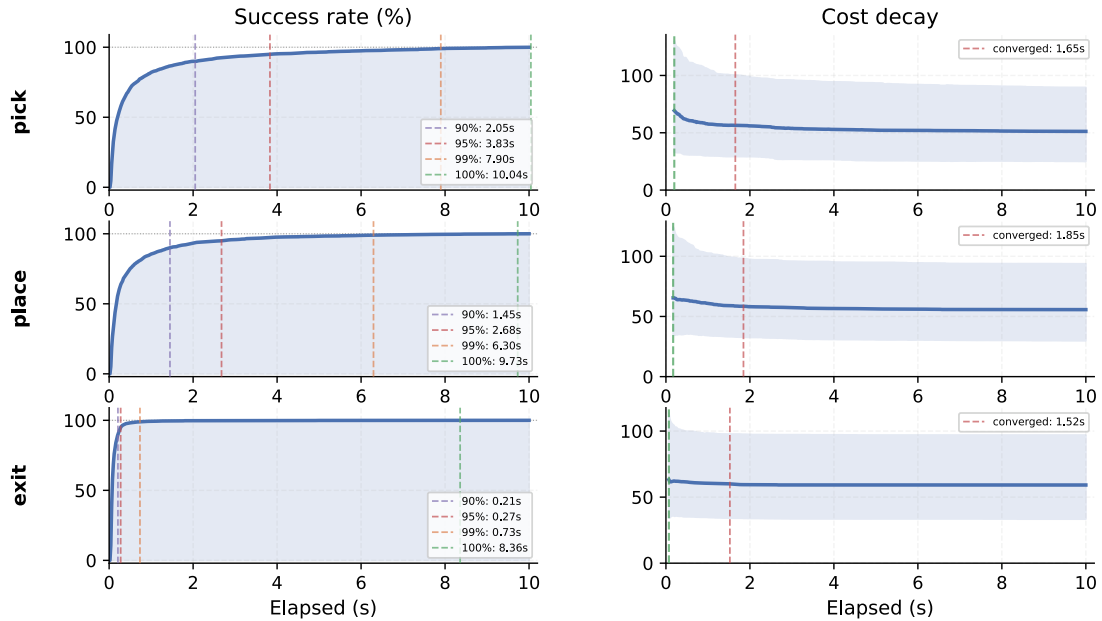


Figure 5.12: Success rate and cost decay for the wall scenario

to avoid other robots. Beyond five robots, the space begins to become congested and the speedup slows down. This is also evident in the Pareto front, which shows a sharp increase in computation time after five robots.

Overall, the wall scenario demonstrates that the algorithm scales well on problems where the dependency tree exposes wide parallelism, provided the workspace is large enough to accommodate the full robot count without inducing congestion. The sevenfold speedup can be considered as an upper bound for all scenarios in which the robots must plan around each other but are not restricted by each other.

5.4 Crate Problem

The crate scenario models a beverage crate populated with a grid of ten bottles. Each bottle resides in its own slot and does not depend on any other object. This results in a flat dependency tree in which all bottles are leaves from the start of the disassembly. The bottles are represented as triangle meshes rather than primitive shapes, which exercises the mesh-based surface sampling strategy described in section 4.5.2. Although the dependency tree is flat, the slots are tight and the bottles must be extracted vertically along a short axial removal path. Afterward, they can be moved freely through the workspace. Because the dependency structure permits unrestricted parallelism while the workspace around the crate remains spatially confined, the crate is used to study the point at which spatial congestion becomes the limiting factor for the achievable speedup.

Table 5.3: Object composition of the crate scenario.

Component Group	Geometry	Count	Extraction
Bottle	Mesh	10	Constrained
Total		10	

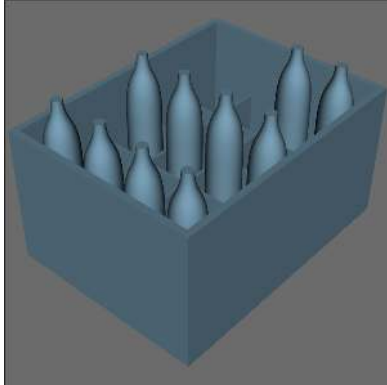


Figure 5.14: Crate scenario in simulation



Figure 5.15: Dependency graph for the crate scenario

Execution and Planning Time

Fig. 5.16 shows that the single robot baseline produces a makespan of around 122 seconds and a calculation time of approximately 370 seconds. Increasing the number of robots reduces the makespan monotonically, reaching a minimum of 28 seconds with eight robots. After this, the median reaches an inflection point and increases again, reaching 32 seconds with ten robots. The interquartile range begins to grow after seven robots, with the 25th percentile dropping to 29 seconds with nine robots.

There is no notable increase in calculation time with one to three robots, while the makespan drops from 122 seconds to 48 seconds. After this, growth is approximately linear with the number of robots, while the interquartile range widens.

Fig. 5.17 shows the corresponding speed-up factor of 4.3 with eight robots, which decreases to 3.8 with ten robots. Calculation time increases by a factor of 1.7 with ten robots.

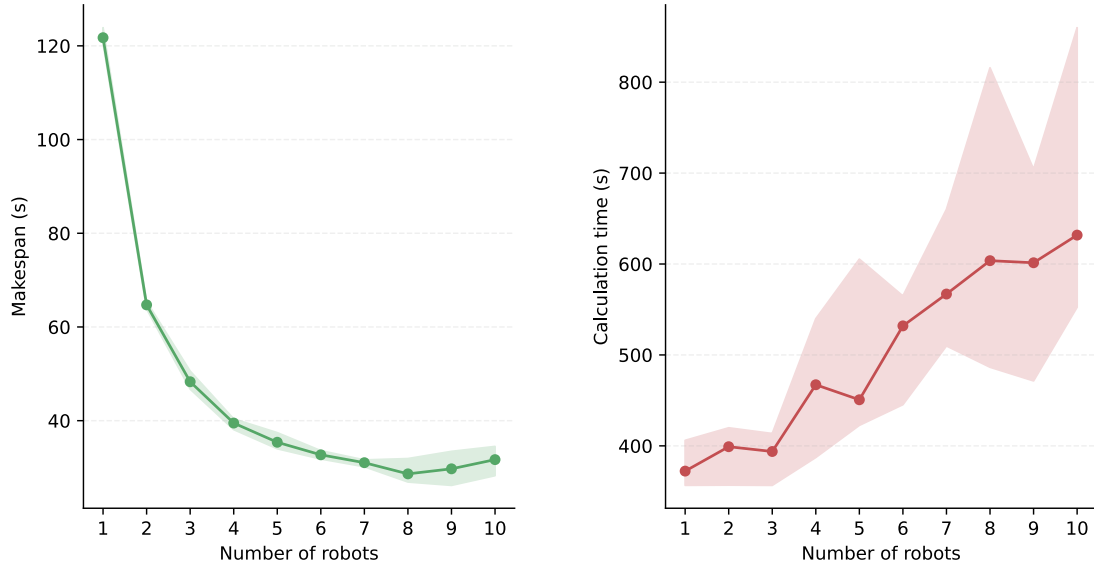


Figure 5.16: Makespan and calculation time for the crate scenario

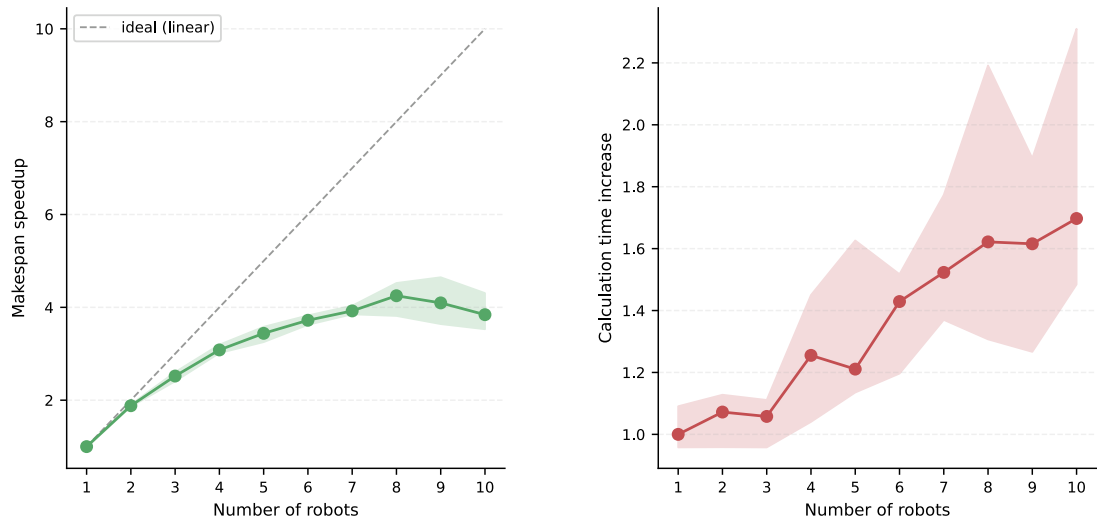


Figure 5.17: Execution speedup and calculation time increase for the crate

The non-dominant numbers of robots in Fig. 5.18 are two, four and ten. These are outperformed by the other dominant solutions on the Pareto front. While one robot has the shortest calculation time, the median for three robots has a considerable decrease in makespan for a slight increase in calculation time. The dominant makespan medians lie on a line ranging from eight to five robots.

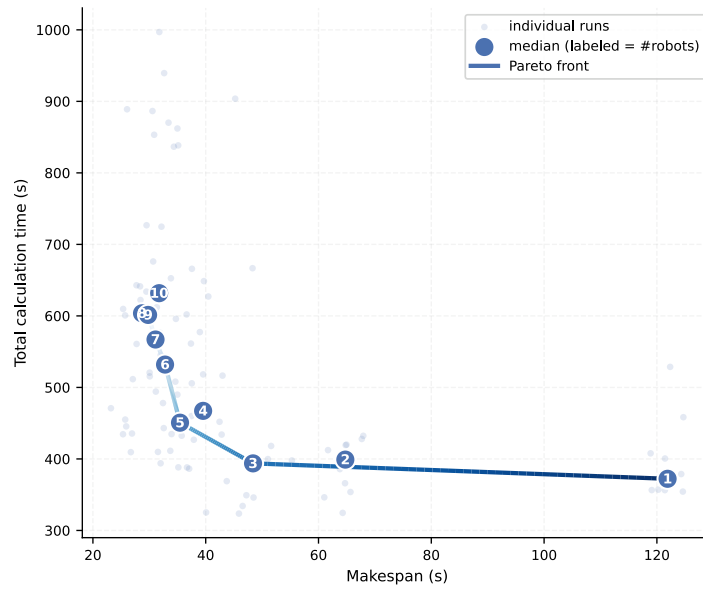


Figure 5.18: Pareto front for the crate scenario

Success Rate and Cost Decay

Fig. 5.19 shows that the pick phase is the slowest of the three phases in finding an initial solution. It reaches 90% in 2.35 seconds, 95% in 4.29 seconds, 99% in 7.56 seconds and 100% in 9.74 seconds. The associated loss plot shows a slight increase in median cost from 16 to 18 after 0.65 seconds, before decaying to a median of 14 after four seconds. The 25th percentile remains consistent throughout, while the 75th percentile decreases from around 30 to approximately 23.

The placement phase reaches 90% in 1.12 seconds, 95% in 1.86 seconds, 99% in 5.02 seconds and 100% in 8.85 seconds. The success rate increases sharply after approximately 0.5 seconds. The median and the 25th percentile of the loss plot do not change significantly over time. The 75th percentile, on the other hand, does converge from around 30 to approximately 27.

The exit phase is the fastest of the three, reaching 90% in 0.13 seconds, 95% in 0.20 seconds, 99% in 0.54 seconds and 100% in 2.27 seconds. The interquartile range and the median converge aligned after 1.13 seconds.

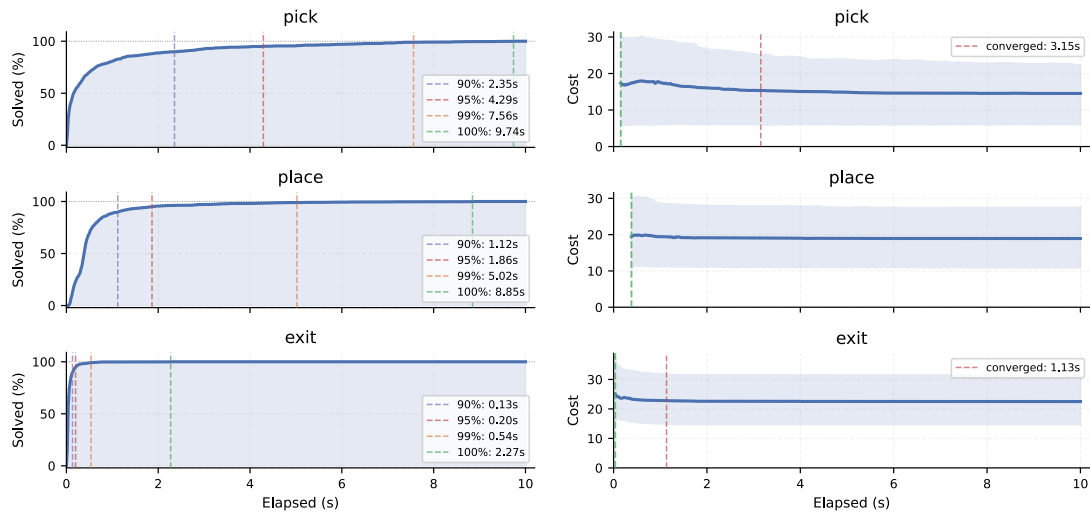


Figure 5.19: Success rate and cost decay for the crate scenario

Timeline

The representative run with ten robots has a makespan of 35 seconds. The pick phase varies from a 4.7 to 25.0 seconds, with the earlier trajectories being shorter and the later ones being longer. The robots are planned in the following order for this run: 7, 8, 1, 4, 10, 6, 2, 3, 5, 9. This is evident from the start time of the trajectories, which increases as no viable inverse kinematics solution can be found.

The pull segments are consistently around two seconds long. Around two to three robots are able to extract a bottle simultaneously. Notably, the last three robots start their extraction after previous bottles have already been placed.

The exit phases are comparable to each other in length at around four to five seconds.

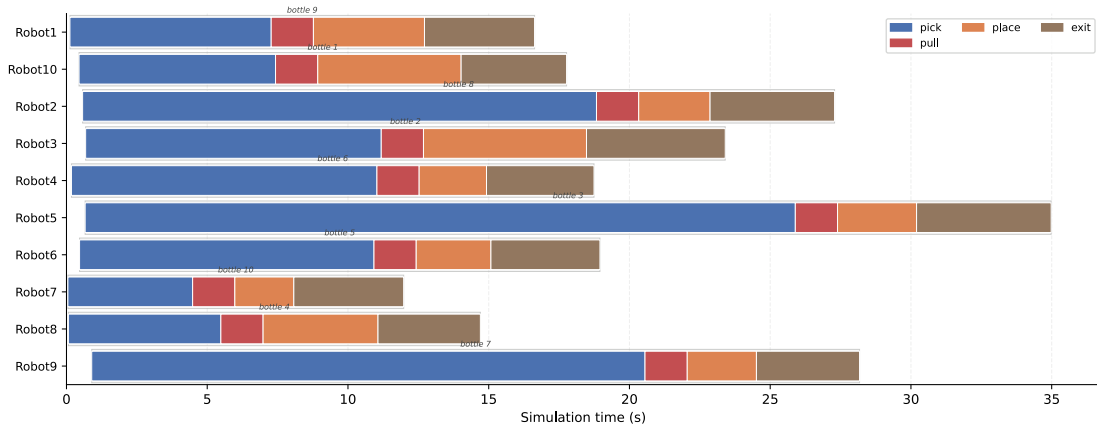


Figure 5.20: Example timeline for the crate scenario

Interpretation

In the crate scenario, the spatial restrictions are isolated from the coupling with the temporal ones of the tower. Since the dependency tree is flat, it imposes no temporal constraints on the disassembly.

Yet, the median speedup factor saturates at around 4.3 due to different robots impeding each other. The workspace above the crate can easily be blocked by one or two robots due to the size of the end-effector. The configuration of the pick position, and therefore the linear extraction, is only concerned with minimizing the arrival time and feasibility of the plan. This can lead to heavy obstruction of the other objects.

However, the algorithm does manage to find solutions for later planned trajectories, if the previously planned trajectories leave enough room. This can be seen in Fig. 5.20, where robot10 finds a solution before robot4. While this is a desirable property of the algorithm, it relies on the random nature of the previously planned trajectories rather than ensuring optimal space usage by design.

Due to the tight spatial constraints the planner must explore more and more as the number of planned trajectories increases. In the absence of temporal constraints, it has to consider the entire time-space for possible solutions. This leads to a degradation in solution quality as the number of robots increases.

Poorer solutions result in later trajectories being less interleaved than earlier ones, which increases the total makespan. This is likely the cause of the inflection point in the makespan with eight robots. When earlier trajectories finish, they start their timespace at the point at which they finished their previous task and became available again. This leads to a much tighter time-space bound, comparable to that from the earliest tasks.

Since the pull sequences have a similar duration, close to the minimum, it is likely that the temporal scaling, discussed in Sec. 14, does not contribute to the success rate of the extraction. This is probably caused by not taking the proximity of obstacles into account when solving the problem.

5.5 Motor Disassembly

The motor scenario involves the disassembly of a generic electric drive comprising a housing with two end covers, six fastening screws, a clamp, two bearings, a rotor, and a stator. With 13 components, the motor is the smallest of the realistic scenarios in terms of number of components. It features a combination of sequential and parallel dependencies, as well as constrained removal paths for the fasteners and bearings. The end caps must be released by removing three screws each before they can be detached. The bearings can then be pulled axially off the rotor shaft. Lastly, the rotor itself can be extracted from the stator. The fasteners are modeled as pins rather than threaded screws, so they can be removed by pulling them along the bolt axis. The motor is used to demonstrate the algorithm on a realistic disassembly problem involving mixed extraction modes in order to evaluate the robustness of the entire extraction pipeline.

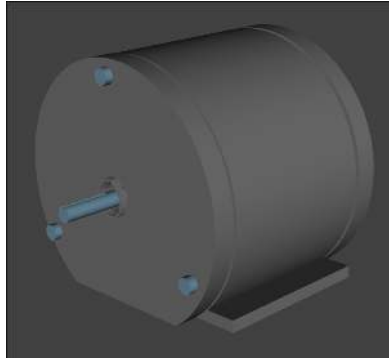


Figure 5.21: Motor scenario in simulation

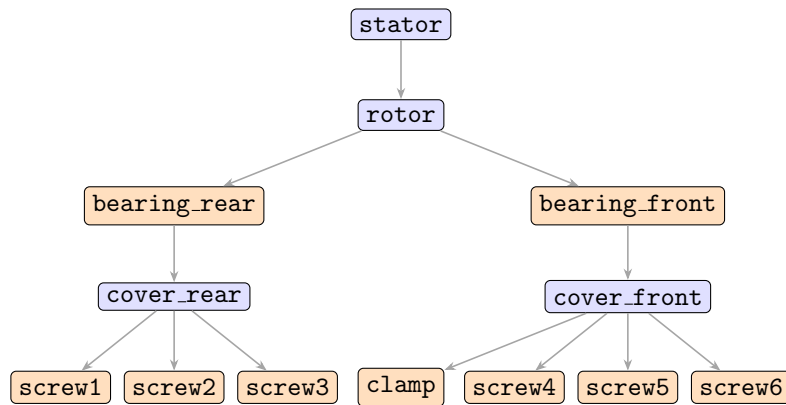


Figure 5.22: Motor dependency graph

Table 5.4: Object composition of the motor scenario

Component Group	Geometry	Count	Extraction
Screws	Mesh	6	Constrained
Clamp	Mesh	1	Constrained
Cover	Mesh	2	Free
Bearings	Mesh	2	Constrained
Rotor	Mesh	1	Free
Stator	Mesh	1	Free
Total		13	

Execution and Planning Time

The makespan baseline for a single robot is around 148 seconds, with a calculation time of 470 seconds. Increasing the number of robots monotonically reduces the makespan to 32 seconds, with a calculation time of 810 seconds. This is almost reached with seven robots in 34 seconds with a calculation time of 610 seconds. The interquartile range of the makespan remains narrow throughout, while the one of the calculation time fluctuates considerably and increases approximately linearly, although seven robots are an outlier with a lower calculation time than anticipated.

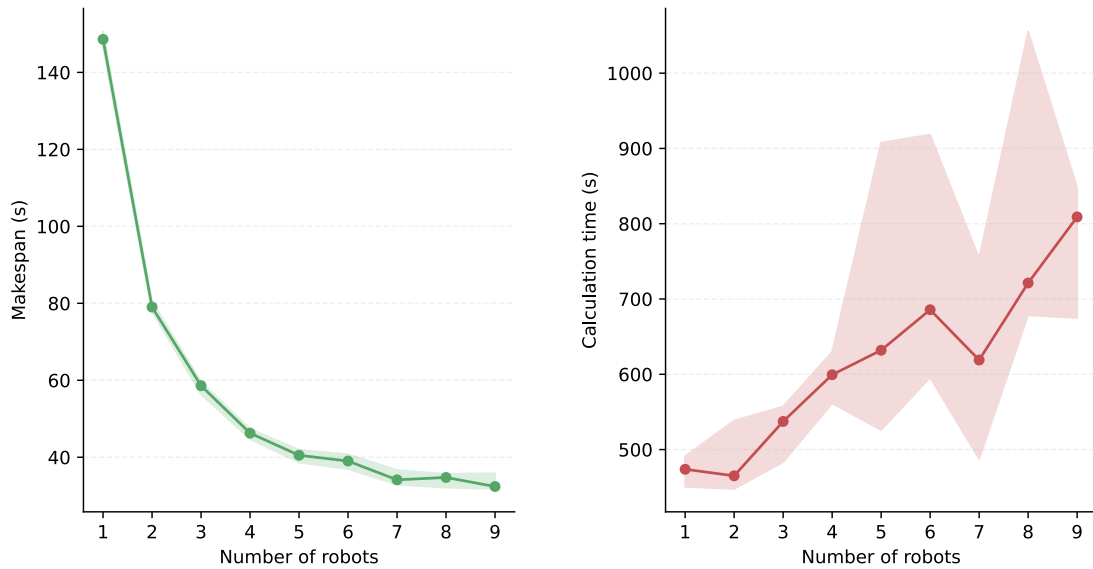


Figure 5.23: Makespan and calculation time for the motor scenario

The corresponding speedup factor in Fig. 5.24 reaches 4.6 with nine robots, while the calculation time grows by a factor of 1.7.

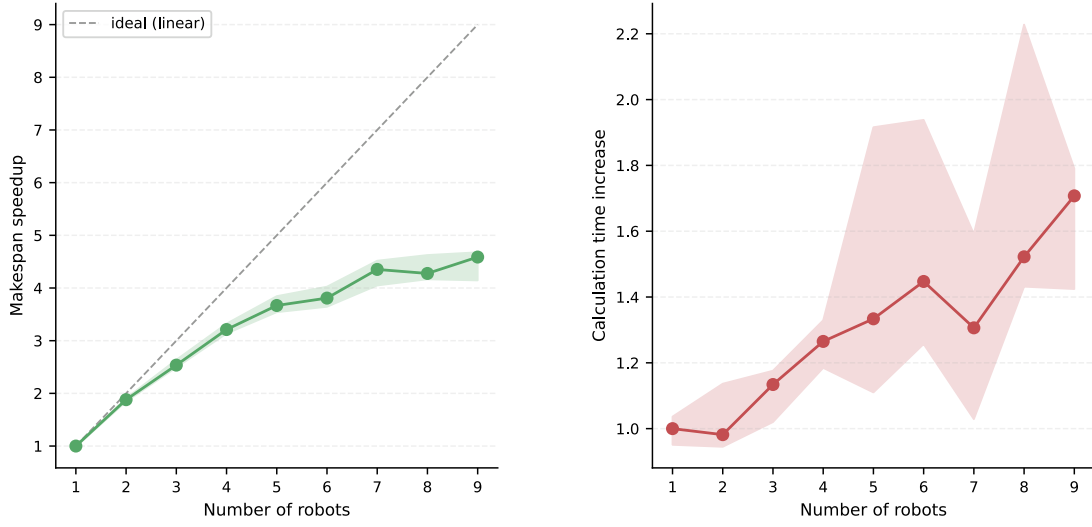


Figure 5.24: Execution speedup and calculation time increase for the motor scenario

The Pareto front in Fig. 5.25 consists of robots: 2, 3, 4, 7 and 9. The non-dominant medians are: 1, 5, 6 and 8. Most medians lie on a line between eight and two robots. Notable exceptions are one robot, which is dominated by two robots and reduces the makespan by 70 seconds without an increase in calculation time and nine robots, which have a large increase in calculation time without a significant decrease in makespan. The seven robots are also not on the line, but are assumed to be outliers due to the low sample count.

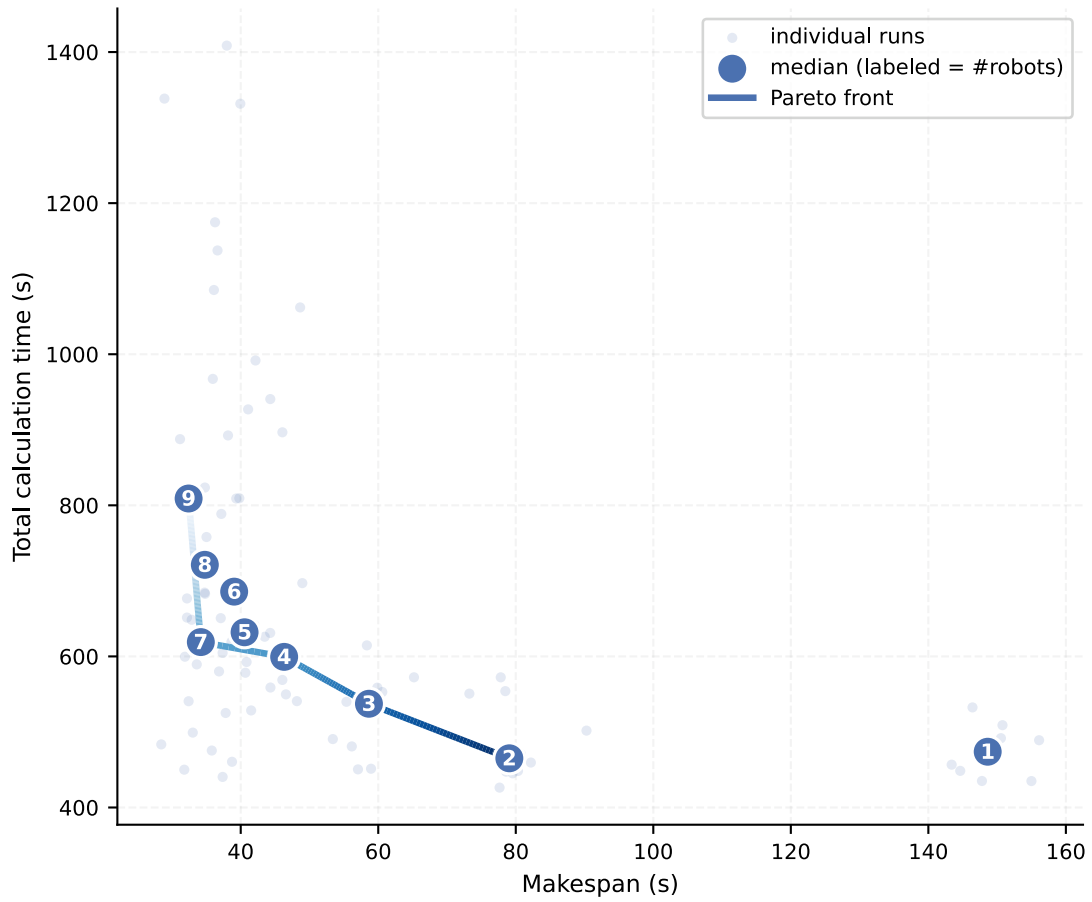


Figure 5.25: Pareto front for the motor scenario

Success Rate and Cost Decay

Fig. 5.26 shows that the pick and place phases require similar planning times to find an initial solution. The pick phase reaches 90% in 1.50 seconds, 95% in 3.20 seconds, 99% in 7.46 seconds and 100% in 9.83 seconds. The place phase reaches 90% in 1.90 seconds, 95% in 3.18 seconds, 99% in 7.55 seconds and 100% in 9.70 seconds. The exit phase converges almost immediately, reaching 90% in 0.13 seconds, 95% in 0.19 seconds, 99% in 0.59 seconds and 100% in 3.02 seconds.

The cost decay shows only a modest improvement across all phases. The pick phase median falls from approximately 22 to 19 between 0.60 seconds and 4.00 seconds, remaining essentially flat afterwards and yielding an improvement of around 14%.

The 75th percentile shows a sizeable decrease from 44 to 32, while the 25th percentile remains consistent throughout.

The place phase shows no significant improvement in the median, which remains close to 23 throughout. The 75th percentile shows a sizeable decrease from 42 to 38, while the 25th percentile stays consistent throughout.

The median for the exit phase drops from around 30 to 26 within the first second and converges at 1.50 seconds, corresponding to an improvement of around 13%.

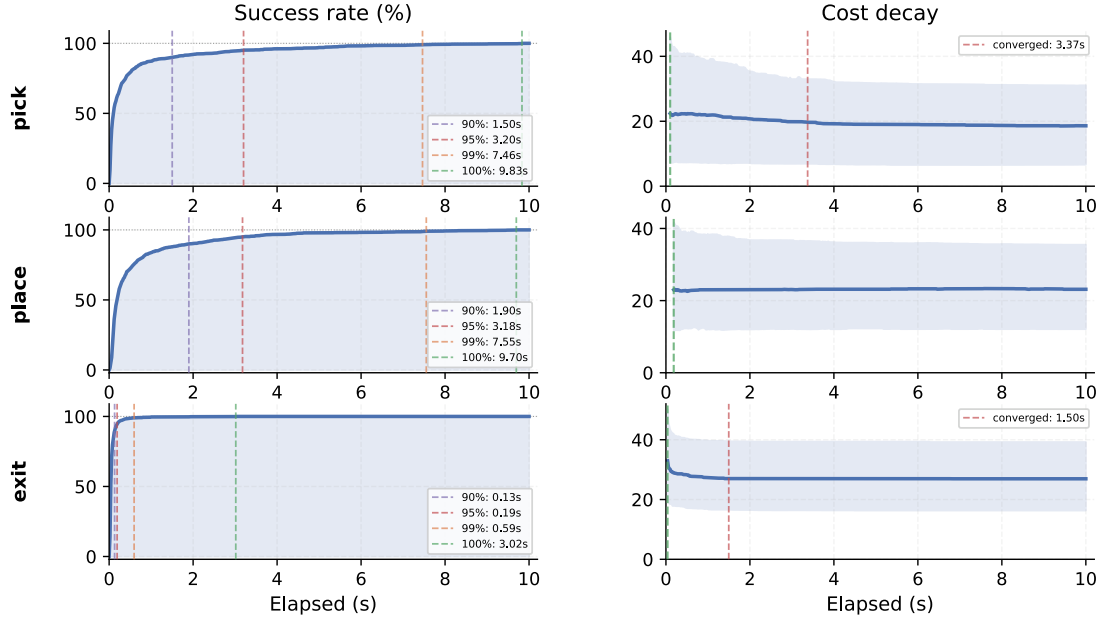


Figure 5.26: Success rate and cost decay for the motor scenario

Timeline

Fig. 5.27 shows a representative execution timeline for the motor scenario involving nine robots. The total makespan of this run is approximately 39 seconds. The dependency tree structure introduced in Sec. 5.22 is visible in the timeline. The original leaf objects, consisting of six screws and the clamp, are picked up by the robots first. This is followed by the front cover and bearing. Due to the slow extraction of the back screws, the back cover can be picked up by robots that have already finished. The front cover and bearing have a long pick phase with loose temporal bounds on the temporal pick constraint by their dependencies.

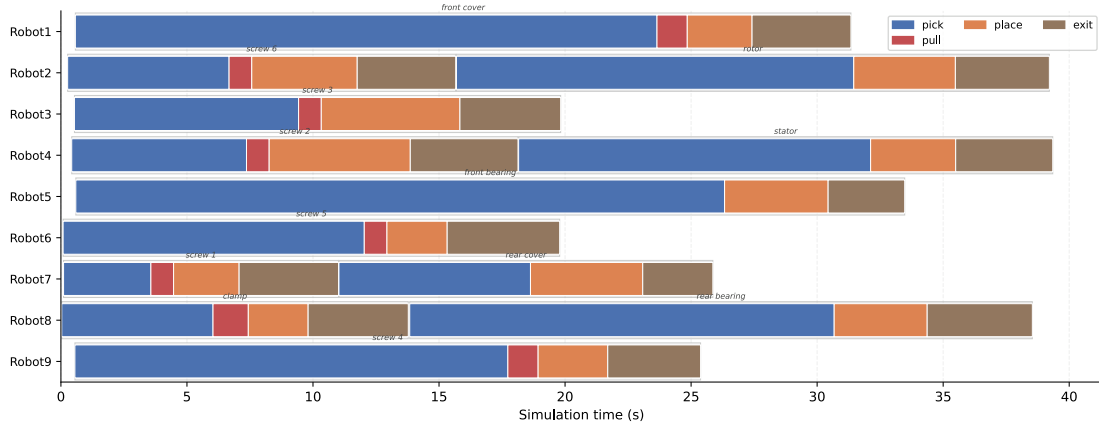


Figure 5.27: Example timeline for the motor scenario

Interpretation

The motor scenario combines the temporal coupling of the tower with the spatial congestion during extractions, like the crate. This is layered on top of a hierarchical dependency tree of depth five and a mixed set of free and constrained extraction modes.

Consequently, the full disassembly pipeline is exercised, including temporal constraint propagation, mesh-based surface sampling, constrained pull phases for fasteners and bearings and prioritized planning under geometric congestion.

Two factors reduce the achievable speedup. First, the dependency tree narrows from seven leaves at its base, with six screws and one clamp, to two covers, two bearings and finally the rotor and stator. Each side of the motor must be removed in sequence. This places an upper bound on the speedup that cannot be increased by any number of additional robots, since the additional capacity cannot be applied to the sequential portion of the disassembly.

The second effect is a coordination failure on the parallelizable portion of the tree. Even when the dependency graph allows for simultaneous extraction, grasp positions from previously planned trajectories hinder subsequent ones, just as in the crate. This can be seen in Fig. 5.27, where the screws and clamp are independent leaves, yet are extracted in a staggered sequence rather than simultaneously.

This behavior occurs even though enough space is available for the extraction of multiple objects at the same time. The result is a median speedup of 4.6 with nine robots, sitting between the tower’s 4.9 and the crate’s 4.3. However, the crate

lacks the workspace to accommodate all robots simultaneously. Therefore, the motor’s gap to the tower is attributable to the inability of the planner to coordinate extraction of multiple objects in proximity to each other without impeding each other.

The assignment of the next object to a free robot is depth-agnostic, therefore, it does not prioritize shallow leaves over objects whose predecessors have just been cleared. This is visible in the front and back halves of the motor. The dependency tree is symmetric, with the front and rear cover sitting at the same depth as the front and rear bearing. In the run shown in Fig. 5.27, however, the front pair is assigned to robots at the start of the schedule, while the back pair is assigned as the second task to robots that have already removed a screw.

Despite the symmetry, the front cover and bearing take roughly as long to extract as the back cover and bearing. The robots incur a high approach cost because they idle through the dependency wait, whereas the other robots arrive at a later stage after performing other tasks. The depth-agnostic assignment therefore pairs robots to tasks in a way that systematically wastes the available parallel capacity.

5.6 Battery Disassembly

The battery scenario is inspired by the modular structure of an automotive battery pack, consisting of 49 components in total. Once the outer housing has been removed, a busbar and 14 bus connectors become accessible. Each connector restricts a subset of four battery cells. The connectors can be moved freely once detached. However, the 32 battery cells must be pulled vertically along a straight extraction path. This battery scenario is used to demonstrate the scalability of the algorithm in scenarios involving many objects that require path-following motion.

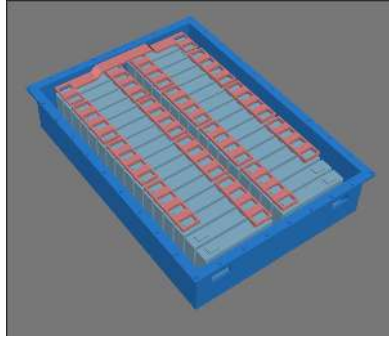


Figure 5.28: Battery simulation without top housing in simulation

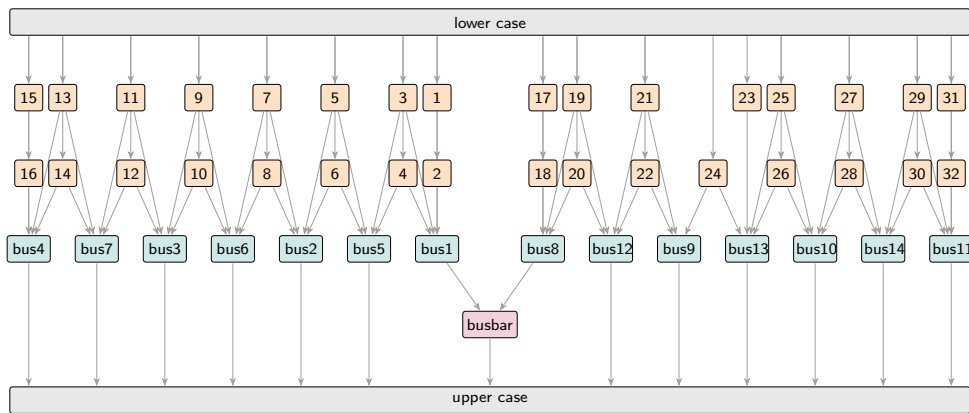


Figure 5.29: Battery dependency graph

Table 5.5: Object composition of the battery scenario

Component Group	Geometry	Count	Extraction
Housing	Mesh	2	Free
Busbar	Mesh	1	Free
Bus connector	Mesh	14	Free
Battery cells	Mesh	32	Constrained
Total		49	

Execution and Planning Time

Fig. 5.30 shows the median makespan and calculation time for different numbers of robots. The makespan baseline for a single robot is around 550 seconds, with a calculation time of 1800 seconds. Adding robots reduces the makespan monotonically, dropping to 91 seconds with nine robots. This corresponds to an 83% reduction or a sixfold speedup. The interquartile range remains narrow throughout.

The median of the calculation time remains consistent at around 1800 for one to four robots. After that, it increases up to around 3100 seconds, resulting in a 1.7-fold increase in computation time. The interquartile range widens after four robots but remains consistent until nine robots are added, at which point it widens further.

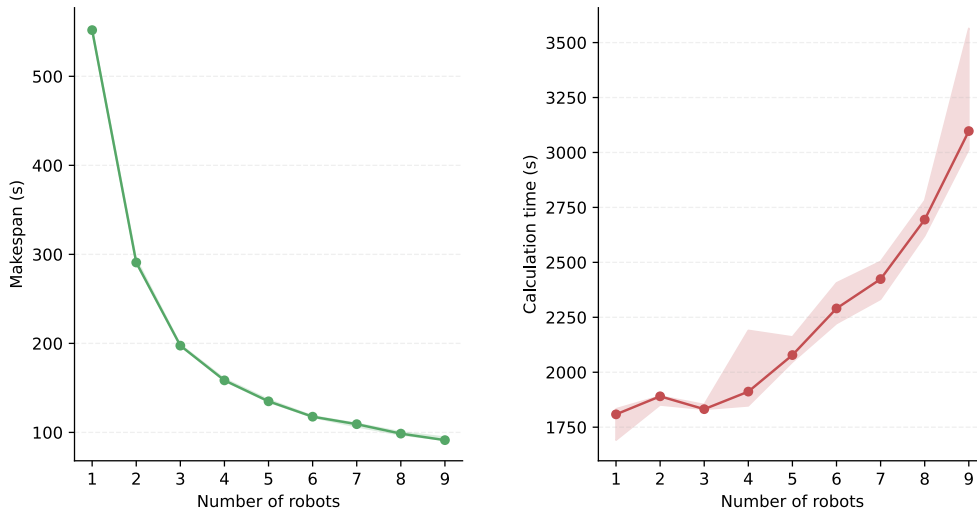


Figure 5.30: Makespan and calculation time for the battery scenario

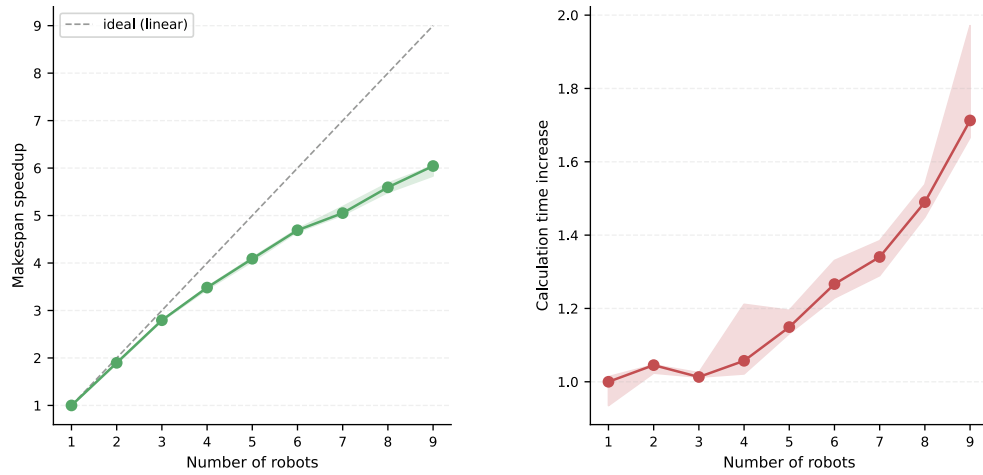


Figure 5.31: Makespan speedup and calculation time increase for the battery scenario

The Pareto front in Fig. 5.32 identifies all robot counts as dominant, except for two robots, which are dominated by one and three robots. The front exhibits a steep decline from one to three robots. All other numbers of robots are a trade off between calculation time and makespan.

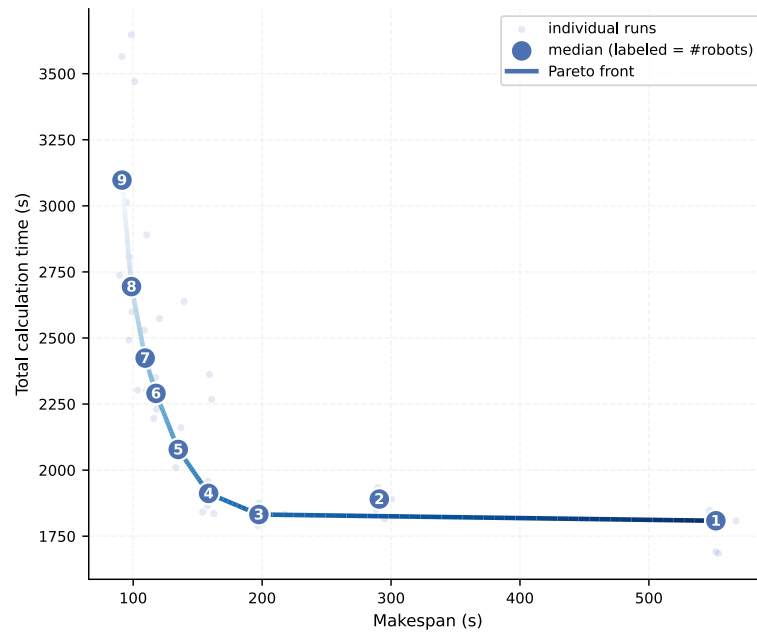


Figure 5.32: Pareto front for the battery scenario

Success Rate and Cost Decay

Fig. 5.33 shows that the pick phase reaches 90% in 2.51 seconds, 95% in 4.54 seconds, 99% in 7.79 seconds and 100% in 9.95 seconds. The place phase is slightly faster, reaching 90% in 1.79 seconds, 95% in 2.96 seconds, 99% in 6.41 seconds and 100% in 9.61 seconds. The exit phase is the quickest of the three, with 90% in 0.48 seconds, 95% in 0.78 seconds, 99% in 2.36 seconds and 100% in 9.50 seconds.

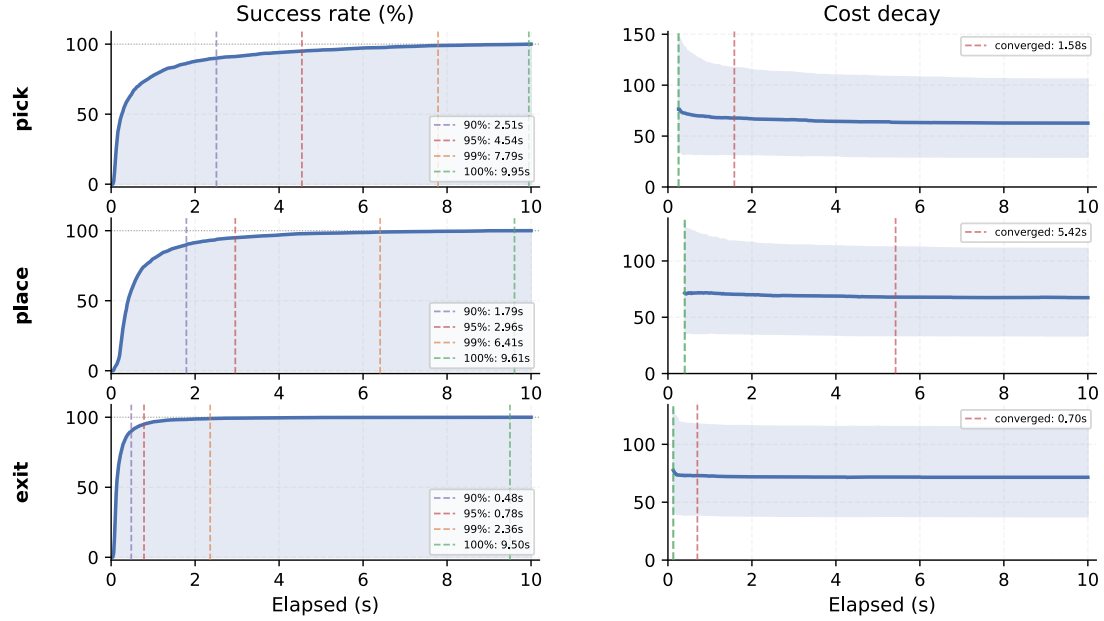


Figure 5.33: Success rate and cost decay for the battery scenario

The cost decay shows modest to no improvement in the median across all three phases. However, the 75th percentile of the pick phase does show an increase, from around 150 to around 120, after approximately 1.50 seconds. The same behavior is observable in the place phase, which drops its 75th percentile from around 120 to 110 in about two seconds and the exit phase, which decreases from 120 to 115 in 0.3 seconds.

Timeline

Fig. 5.34 shows a representative timeline for a run involving nine robots. The total makespan is around 90 seconds. Each robot completes between five or six tasks during the run.

The pull segments, associated with the battery cell removal, range from one to two seconds. This is due to the time scaling during the extraction. The length of the pick, place and exit phases varies widely across the schedule.

The timeline is tightly packed, but has occasional short gaps between pick-and-place tasks.

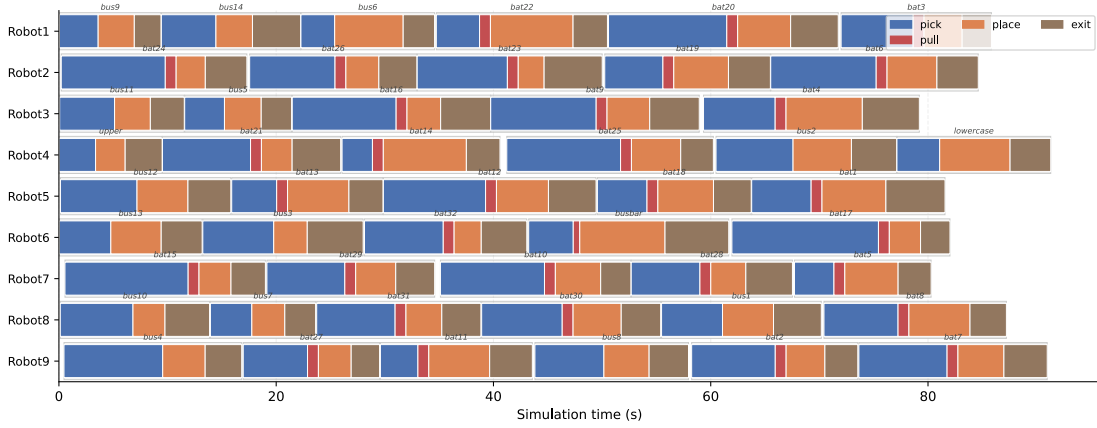


Figure 5.34: Example timeline for the battery scenario

Interpretation

The battery scenario puts the algorithm under stress at scale. With its 49 objects, it is around five times larger than the comparable crate example in Sec. 5.4 or roughly four times larger than the motor scenario from Sec. 5.5.

The sixfold speedup with nine robots is second only to the wall example. This is considerably higher than the 4.3 for the crate or 4.6 for the motor. However, it is still smaller than the sevenfold increase of the wall example. This is likely due to the increased complexity introduced by the pull sequence, since each pull is more likely to fail due poor inverse kinematics solutions or obstruction by other robots than a regular move, as in the wall example.

Further difficulties compared to the wall scenario are introduced by the housing, which forces most batteries to be picked up from the top until enough battery cells have been removed to allow access from the side.

The difference in speedup between the crate and battery scenarios is significant. Due to the high leaf count of the dependency graph in the battery scenario, it is less likely that multiple robots will be active in the same area. In the crate example, robots are forced to pull in a confined space surrounded by many other robots. This behavior does not occur as strongly here, since the objects are spaced out over a larger area due to their size. This is also seen in the inflection point in the makespan in Fig. 5.16, which does not occur in the battery scenario. The Pareto front in Fig. 5.32 contains almost every number of robots.

Overall, the battery scenario demonstrates that the algorithm’s performance characteristics scale favourably with problem size when the dependency tree exposes sufficient parallelism. The sixfold speedup is achieved because the bottlenecks identified in the smaller scenarios — the depth of the tower’s chain, the spatial congestion of the crate and the depth-agnostic assignment in the motor — do not apply to this example or are remedied by the number of objects and lack of dependency depth.

The scenario thus validates the algorithm on a problem of a realistic industrial scale. It suggests that the algorithm’s limitations are most pronounced in small, tightly constrained problems rather than on large, structured ones. However, improvements to tightly constrained problems could also benefit larger examples like this one, since it contains smaller subproblems, such as a bank of battery cells, which can still occur.

5.7 Gearbox Disassembly

The gearbox scenario is the most geometrically demanding benchmark and models a four-stage planetary gearbox comprising 33 objects in total. Each stage contains a sun gear in the middle, three planet gears surrounding the sun gear and an outer ring gear. Two carrier plates on each side of the stage fixate the assembly around a central shaft. The dependency tree is both deeper and more interleaved than in the previous scenarios. The process switches regularly between sequential and parallel constraints since each stage can be seen as a subassembly. Gears can be extracted in parallel, while carrier plates have to be done sequentially.

Stage 1 is the input stage, which is independent from the output stages 2, 3 and 4. Therefore, the gearbox can be disassembled simultaneously from both sides.

The gearbox is used to demonstrate the algorithm on a complex industrial disassembly problem with deep, interleaved dependencies and tight geometric clearances, where many components are interlocked and require constrained extraction paths.

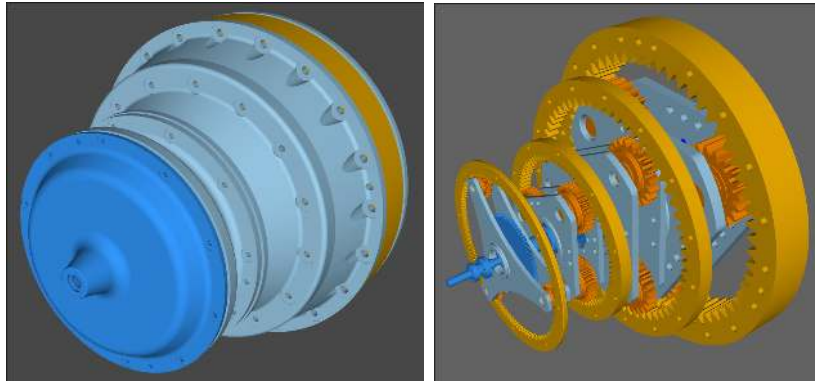


Figure 5.35: Gearbox example [75] with (left) and without (right) casing in simulation

Table 5.6: Object composition of the gearbox scenario

Component Group	Geometry	Count	Extraction
Housing	Mesh	4	Mixed
Sun gear	Mesh	4	Free
Planet gear	Mesh	12	Free
Outer ring gear	Mesh	4	Mixed
Carrier plate	Mesh	8	Constrained
Shaft	Mesh	1	Free
Total		33	

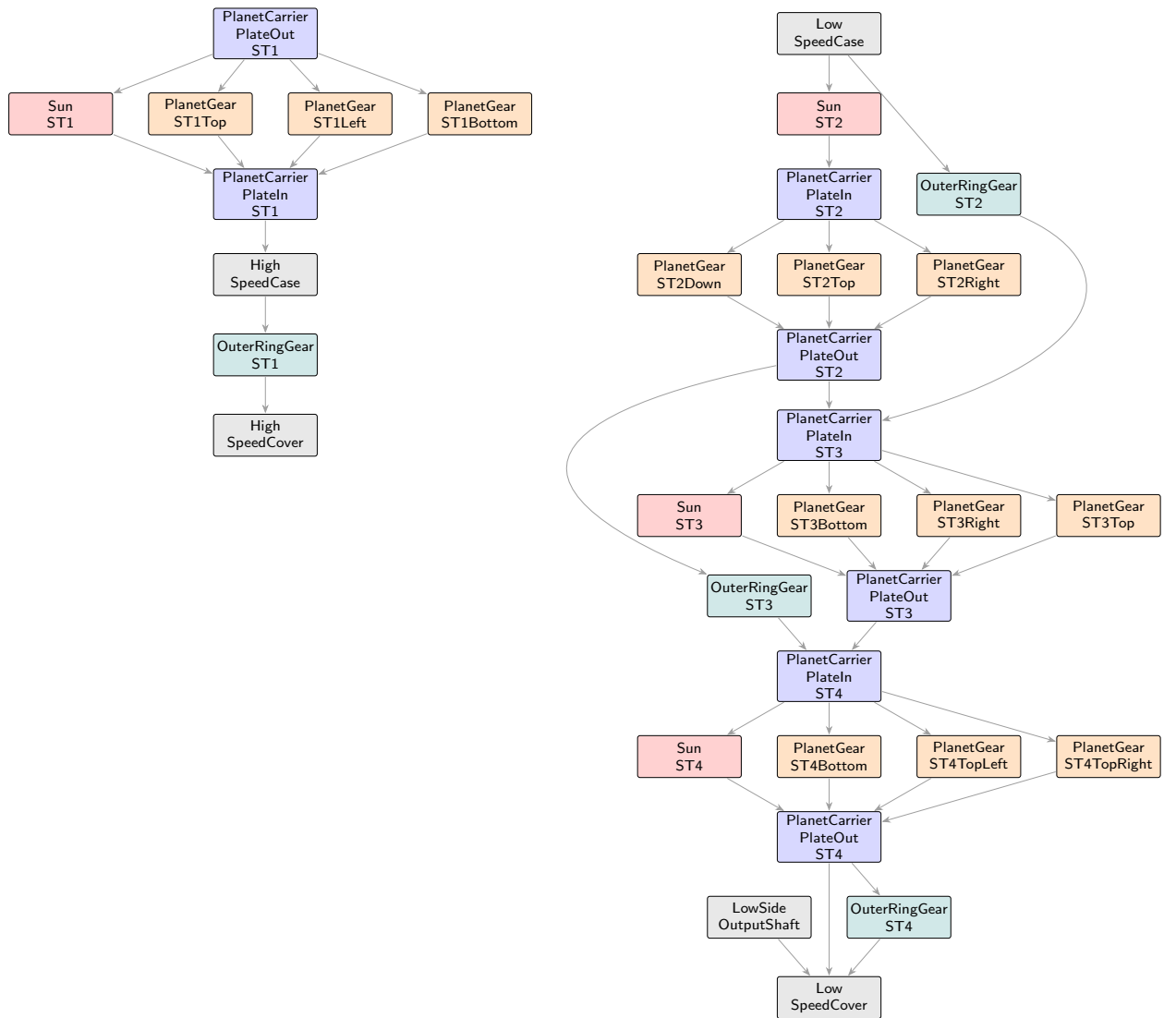


Figure 5.36: Gearbox dependency graph

Execution and Planning Time

The single robot baseline produces a makespan of around 390 seconds with a calculation time of approximately 1010 seconds. Adding robots reduces the makespan monotonically to around 81 seconds. The decline starts to saturate with six robots at 88 seconds. This corresponds to an 80% reduction in makespan. The interquartile range of the makespan stays consistently narrow throughout and only widens slightly with eight and nine robots.

The calculation time does show a linear trend with an increase in the number of robots, but fluctuates significantly due to the low sample count of five. This can also be seen in the rapid changes in the interquartile range.

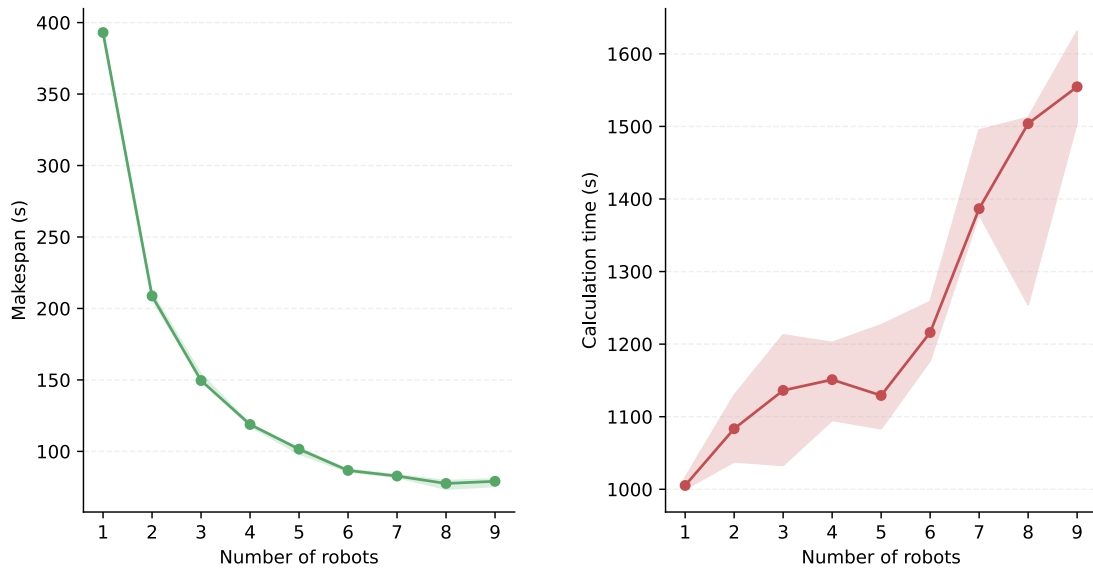


Figure 5.37: Makespan and calculation time for the gearbox scenario

Fig. 5.37 shows the corresponding speedup factor of up to approximately 5.0 with nine robots, while the calculation time grows by a factor of 1.55.

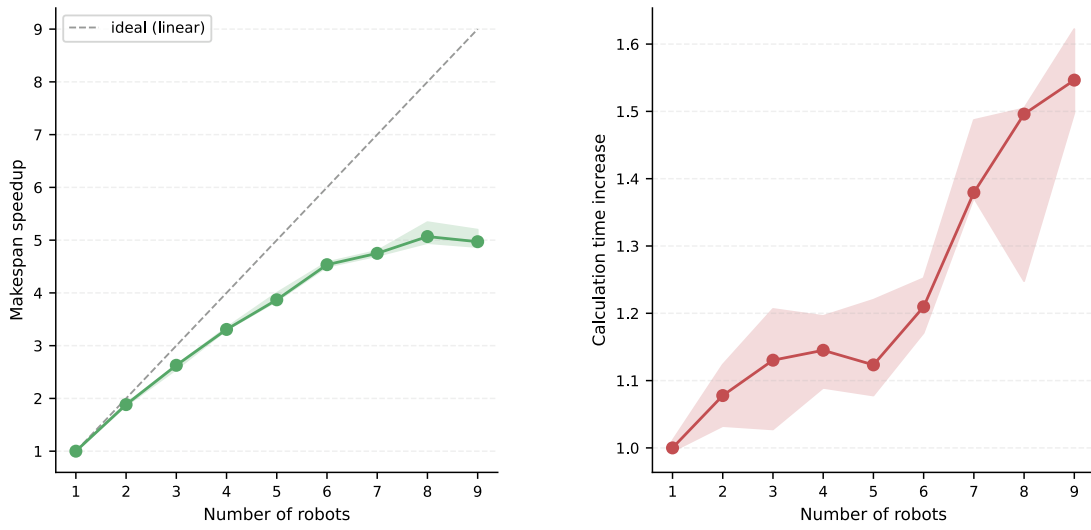


Figure 5.38: Execution speedup and calculation time increase for the gearbox scenario

The Pareto front in Fig. 5.39 identifies all robot counts as dominant except for 3, 4 and 9. Robot counts six through nine lie almost on a line, with only a slight improvement in makespan and a notable increase in calculation time.

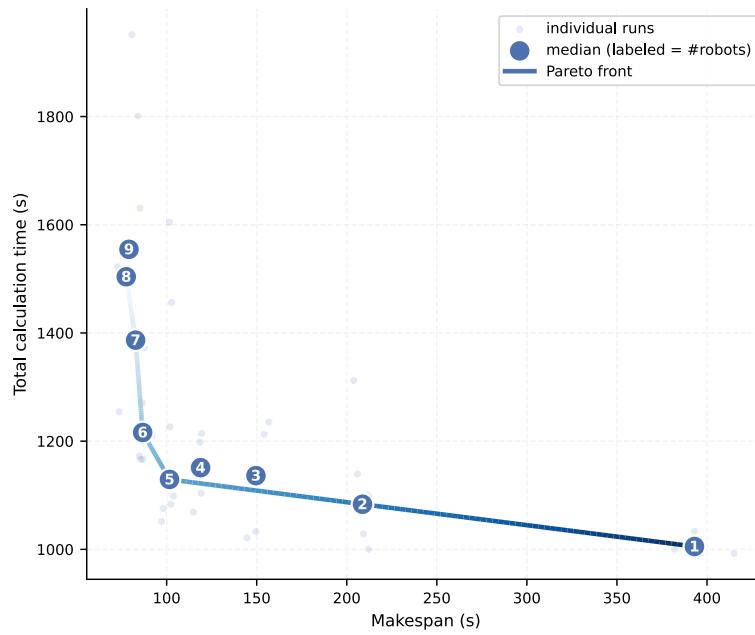


Figure 5.39: Pareto front for the gearbox scenario

Success Rate and Cost Decay

Fig. 5.40 shows that the pick and place phases require similar planning budgets to find an initial solution, with the place phase slightly faster than the pick phase.

The pick phase reaches 90% in 1.71 seconds, 95% in 2.98 seconds, 99% in 7.10 seconds and 100% in 9.95 seconds. The place phase reaches 90% in 1.40 seconds, 95% in 2.25 seconds, 99% in 6.15 seconds and 100% in 9.74 seconds.

The exit phase converges much quicker than the previous two, reaching 90% in 0.17 seconds, 95% in 0.28 seconds and 99% in 0.83 seconds.

The median cost decay shows a modest improvement for the place and exit phase, while consistent for the pick phase. The 75th percentile drops for all phases, while the 25th percentile only drops for the place and exit phase.

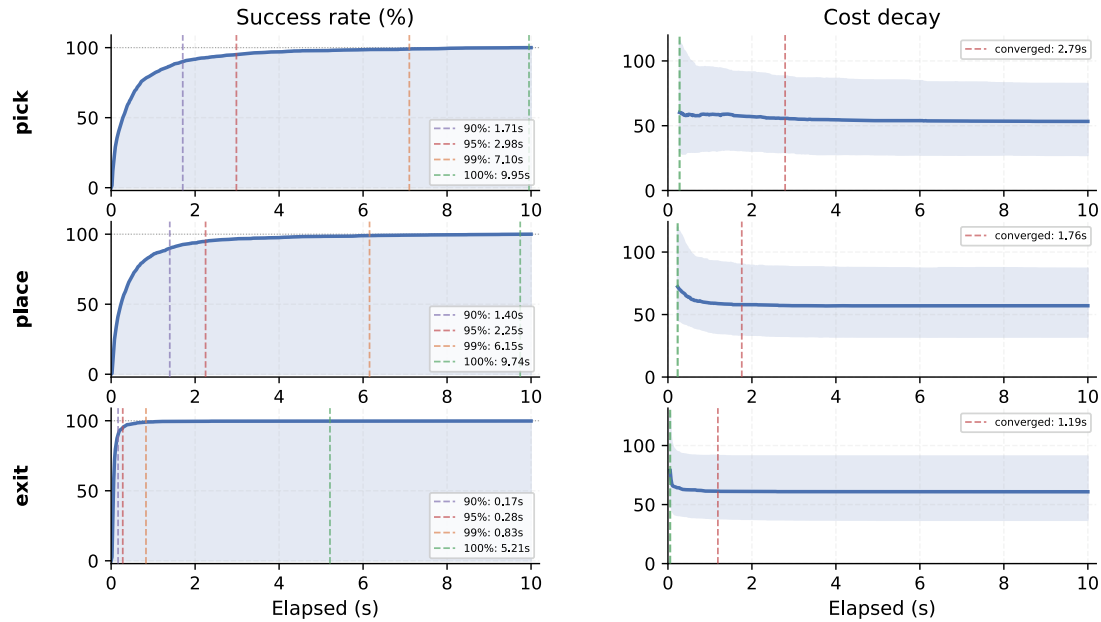


Figure 5.40: Success rate and cost decay for the gearbox scenario

Timeline

Fig. 5.41 shows a representative execution timeline for the gearbox scenario with nine robots. Each robot completes three or four tasks across the run.

The example run disassembles the high speed side of the gearbox first with long pick phases. The place and exit segments remain similar in length throughout.

The pull segments vary in size due to the temporal scaling ranging from one to three seconds.

The schedule is tightly packed and does not show any significant gaps.

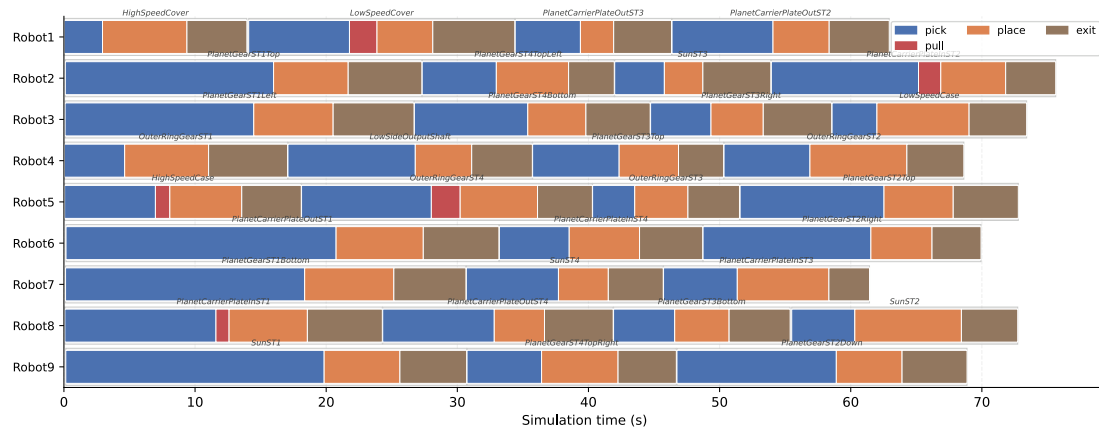


Figure 5.41: Example timeline for the gearbox scenario

Interpretation

The gearbox scenario brings together features that were tested individually in the preceding scenarios. Dependency propagation is deeper than in the tower, while the number of parallel branches can match the wall at certain times.

As with the crate and motor, constrained extraction is needed, during different stages of the disassembly.

With 33 objects of mixed modes arranged across four sequentially coupled stages, it is the most geometrically demanding scenario. It serves as a stress test for the algorithm on a problem that most accurately resembles a real industrial disassembly task.

The fivefold speedup is comparable to the 4.9 speedup of the tower and slightly higher than the 4.6 speedup of the motor. However, it is significantly lower than the sixfold speedup of the battery and the sevenfold speedup of the wall. This is likely due to the selection of nodes from the dependency graph, since the algorithm tends to disassemble nodes deeper in the graph, rather than selecting shallow nodes.

This results in long wait times for the robots instead of disassembling the gearbox from both sides simultaneously. Consequently, the gearbox can degrade into two sequential tower examples, with only slight parallelism.

This demonstrates the same problem discussed in Sec. 5.5, where a lack of awareness of the subsequent planning problems can lead to a degradation of parallel tasks into sequential ones. Both scenarios demonstrate this with the screws from the motor and the gears from the gearbox.

Overall, the gearbox scenario shows that the algorithm can handle realistic, geometrically demanding industrial disassembly problems with deep, interleaved dependencies. However, a lack of coordination when removing objects from the same confined space will lead to performance degradation.

5.8 Failure Analysis

This section analyzes the various reasons for failure across the tower, crate, motor and battery scenario. These scenarios were selected for their display of different behavior during the disassembly, providing deeper insight into the algorithm.

The planner pipeline, as described in Ch. 4, decomposes the disassembly into pick-and-place tasks, which are then broken down into subtasks. All of these tasks share the same stages: generation of inverse kinematics, motion planning to the object, motion planning to the place pose and motion planning to the exit pose. Additionally, some objects require a pull phase for path-following behavior in narrow passages.

When a stage fails, parts of the pipeline that have already been executed become unusable, resulting in wasted computation time. Therefore, it is important to analyze where difficulties arise in order to eliminate bottlenecks, which increase the computation time.

Tower

Fig. 5.42 shows the failure reasons in the tower scenario. The success rate remains between 70% and 86% throughout, decreasing from 79% with one robot to 62% with ten robots. The most common failure reason is failing to plan to the goal, accounting for 14% to 30%. An outlier is the failure to plan to the grasp pose with ten robots at 12%, which does not occur at all, except for 3% with six robots. A slight increase of one to two percent in failure to exit the area occurs for nine and ten robots.

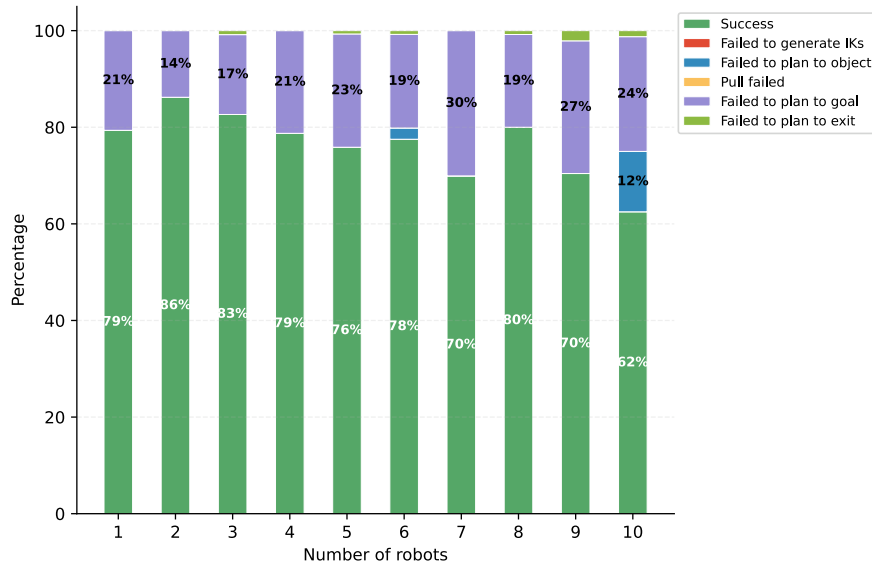


Figure 5.42: Failure reasons for the tower scenario

Crate

The crates' failure reasons in Fig. 5.43 show a decline in the success rate, from 56% with an increase in number of robots to 24% with ten robots. Failure to plan to the goal pose is the dominant failure reason for a lower number of robots, starting at 38%, but steadily decreasing to 17% with ten robots. In contrast, the pull failures start at 6% for one robot and increase to 48% at ten robots. Failure to plan to the object grasp pose starts to increase from 0% to 1% until five robots, reaching a maximum of 10% at ten robots. Failure to plan to the exit configuration ranges from 0 to 4%, peaking at six robots. Lower and higher robot counts taper off from the peak.

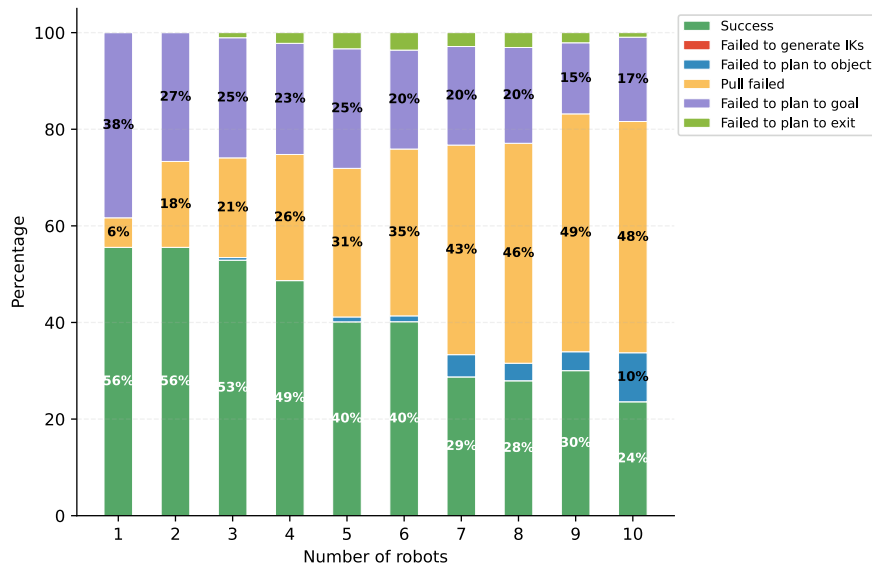


Figure 5.43: Failure reasons for the crate scenario

Motor

Fig. 5.44 shows the failure reasons for the motor scenario. The success rate decreases from 64% with one robot to 36% with nine robots, with dips down to 29% at eight robots. The dominant failure reason is the place phase, accounting for 28% to 37% of failures. The pull phase starts at 4% for a low robot count and increases with the number of robots to between 18% and 26%. From three robots onwards, the pick phase starts to fail more frequently, but stays at or under 4% for all robot numbers. The exit phase peaks at 7% for five, six and seven robots, tapering off for lower and higher robot counts.

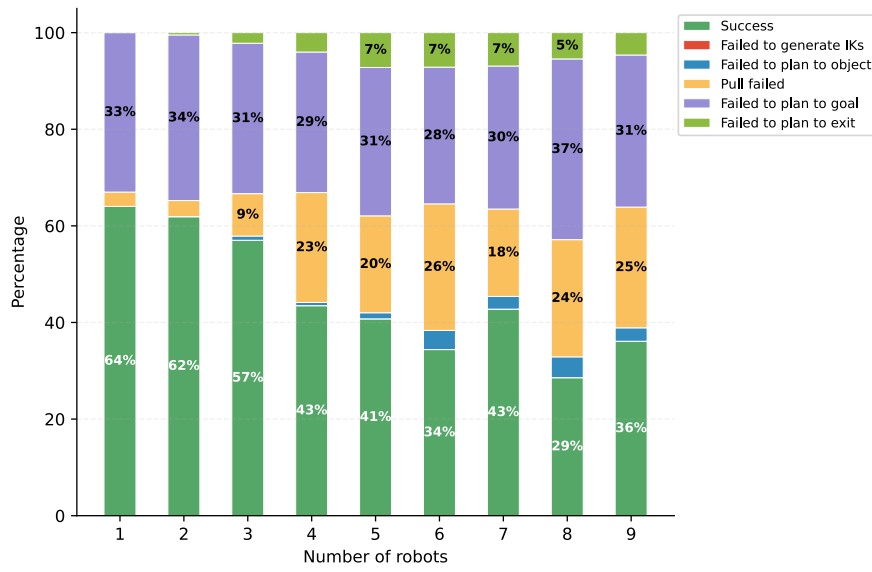


Figure 5.44: Failure reasons for the motor scenario

Battery

In Fig. 5.45, the batteries' success rate starts at 66% with one robot, decreasing to 26% with nine robots. The pick phase remains consistent and dominant throughout, with a failure rate between 31% and 36%. The pull segment starts at 4% for a low number of robots, increasing up to 24% with nine robots. The pick phase begins to regularly fail at five robots with a failure rate of 4%, increasing to 14% with nine robots. Exiting the area fails at most with 4% with less failures for lower number of robots.

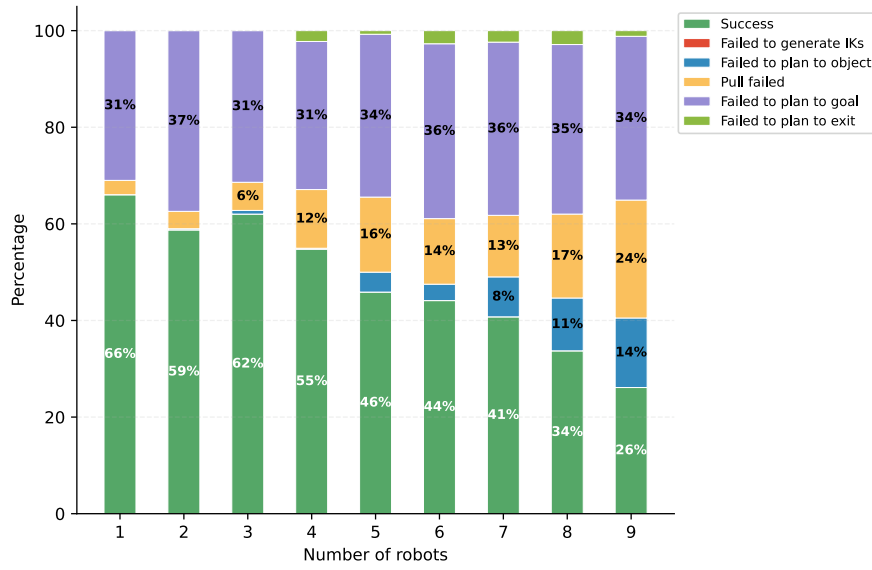


Figure 5.45: Failure reasons for the battery scenario

Interpretation

The failure distributions reveal the bottlenecks in the disassembly pipeline and their severity in relation to scaling up to more robots.

In the tower scenario, the parts are arranged in an open vertical stack, which imposes few geometric constraints on the pick-and-place behavior. Consequently, the planner can pick the objects at a high rate, only occasionally failing at ten robots, presumably due to high congestion around the tower. The dominant reason for failure is the planning for the placement configuration. The slight increase in failure rate with a higher number of robots can be attributed to increased congestion around the place area. However, the high failure rate for the small numbers of robots indicates a problem generating the pick-and-place configurations. Notably, the failure rate of 21% for one robot is abnormally high. This is in stark contrast to the motion planning for the pick configuration, which fails at a much lower rate. Since the generation of pick-and-place configurations is tracked as a separate failure reason and does not show up at all, further compounds the evidence that the generated place configurations are not approachable by the robot with an attached object. The planner and configuration generation are using the same validity checker, therefore, it is highly likely that it does not identify certain unreachable configurations.

In the crate scenario, the parts are tightly packed with a restrictive pull movement. The growth in the failure rate for the pull clearly demonstrates this, increasing from 6% to 48%. The more the robots have to move and plan around the crate, the more likely the simple pull algorithm is to encounter an obstacle or another robot. The previously high failure rate for the motion planning in the place segments can be seen again with only one robot, at an even higher rate of 34%, whereas the failure rate for the pull is only 6%. However, this problem becomes less noticeable for a higher number of robots because fewer attempts even advance to the placement stage of the pipeline without terminating early at the pull stage. Movement to the pick configuration of the object also starts to fail earlier compared to the tower. This is likely due to the more confined area of the crate, forcing the robot's end-effector into much narrower areas.

The motor and battery scenarios have a similar profile, with a dominant failure of the motion planning for the place configuration. This is likely due to the same reasons stated previously in the tower section. However, due to the easier pull segments, this issue is not as significant as in the crate example. This is especially noteworthy in the battery example, which has the same ratio of place and pull tasks as the crate.

Most examples demonstrate similar behavior during the exit phase, which worsens with an increase number of robots. This is likely due to poor sampling, since it does not increase in line with the number of robots, unlike the pick movement. Alternatively, it may be the result of occasional congestion due to unfavorable previous trajectories of other robots.

The failure analysis clearly shows the shortcomings of the current implementation of the algorithm. Most noteworthy is the high failure rate for the motion planning to the place configuration. It is highly likely that a failure rate this high for a single robot indicates a faulty implementation. The expected behavior is much closer to that for motion planning for the pick configuration. It starts at a low failure rate and increases with the number of robots due to congestion. Second, the approach to the pull phase does not scale well in a congested workspace. This matches expectations, since it is a naive implementation of the problem and warrants further investigation to find a better approach.

5.9 Comparison Against RRT*

This section compares RRT* and ST-RRT* in the context of the tower scenario introduced in Sec. 5.2. It compares their performance with regard to makespan, calculation time and success rate. Each scenario attempt has a timeout of 1000 seconds to prevent premature termination. The individual planner call for a sub-task has a calculation time of ten seconds. The number of robots is varies from one to ten in order to test the algorithms’ scalability. Each algorithm is evaluated with ten runs for each number of robots.

Since RRT* can only operate within a fixed time space, two time windows were tested. The five-second window is used as a tight bound with little room for detours, but easier to optimize due to the smaller state space. Whereas the ten-second window allows for greater flexibility in space-time, enabling detours to be taken to avoid robots. However, this results in a larger state space, leading to a less dense planning tree with the same number of valid samples as the five-second time-space state space.

Both time windows were empirically selected based on the convergence behavior of ST-RRT* in Sec. 5.2. These bounds are not known a priori and are subject to hyperparameter tuning.

The comparison in Fig. 5.46 compares the scaling of the makespan and calculation time for the three implementations. With a five-second time window, the makespan for RRT* is the slowest of the three at the start, with 168 seconds with one robot, decreasing to 26 seconds with five robots. It does not improve further with more robots. With four robots, the makespan increases again to 90 seconds, compared to the 82 seconds with three robots. After five robots, the makespan matches or outperforms that of the ten-second time window. Starting at 99 seconds with one robot, the ten-second window improves monotonically up to 27 seconds with five robots. Afterwards, the makespan increases again, fluctuating between 31 and 36 seconds for six to ten robots.

ST-RRT* consistently outperforms the makespan of the other two. It starts at 94 seconds with one robot and falls monotonically to 19 seconds with ten robots. The calculation time sits between 320 and 370 seconds across the range, with a slight upward trend as the number of robots increases.

The calculation times of both RRT* variations are primarily close to the minimum calculation time of 300 seconds with occasional outliers spiking to 380 to 430 seconds. ST-RRT* shows a steady increase in calculation time as the number of robots increases.

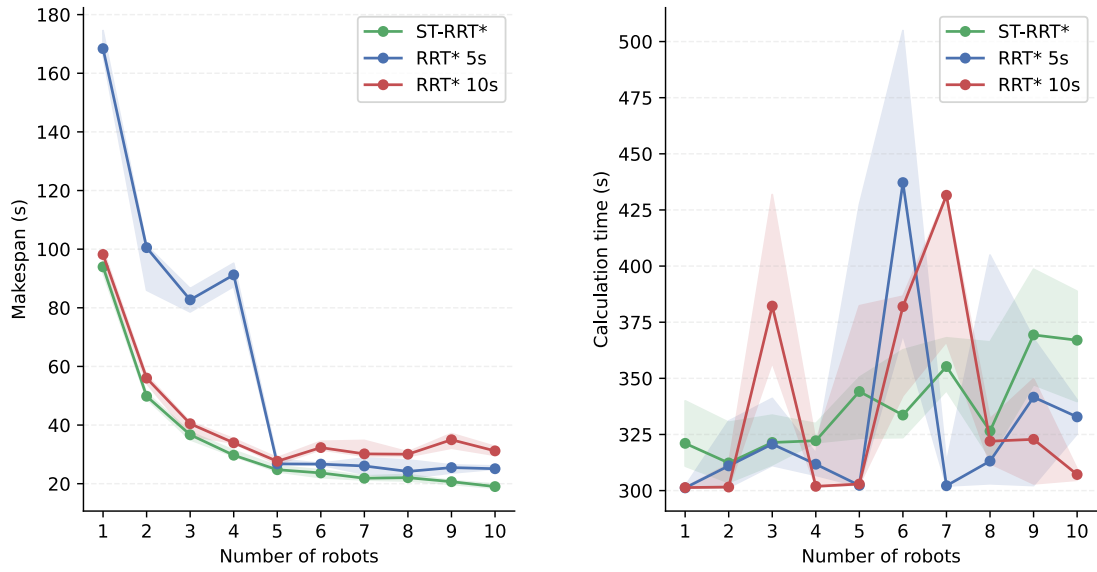


Figure 5.46: Makespan and calculation time comparison between ST-RRT* and RRT*

Fig. 5.47 shows the success rate of the three planners over ten runs of the complete scenario. RRT* with a ten-second window starts at 100% for one robot, but drops rapidly to around 30% for three or more robots. The five-second window for RRT* has a success rate of around 50% for one to three and eight to ten robots. With four to seven robots, it drops to approximately 30%. ST-RRT* achieves a success rate of 100% throughout.

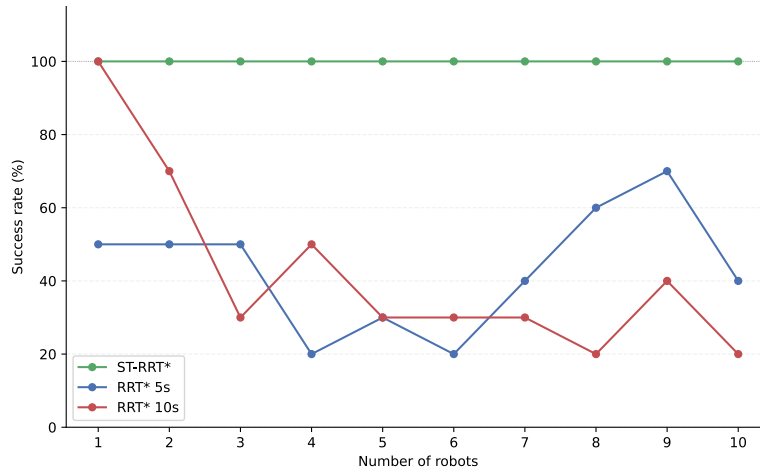


Figure 5.47: Success rate comparison of ST-RRT* and RRT*

Interpretation

The comparison demonstrates the importance of designing a planner to operate directly in space-time, rather than adapting an existing planner to incorporate this feature.

RRT* with a ten second window achieves a comparable makespan to ST-RRT* for one to five robots, but struggles to find competitive solutions beyond this. While the wider time window provides flexibility for detours, the resulting state space becomes too large to explore and optimize once the workspace becomes crowded with other robots. The decline in success rate with an increasing number of robots reflects the same limitation.

The five second window performs worse for a small number of robots due to the inflexible nature of a small time budget. The planner cannot plan a detours to avoid robots, even when the surrounding space is unoccupied. The narrow window concentrates sample density along the time dimension, which becomes advantageous once the workspace is crowded enough that detours would rarely be useful anyway.

This explains the crossover at five robots, where the narrower window begins to outperform the wider one. The underlying trade-off for a fixed sample budget is that the sampling density in time and the volume of the state space are inversely related. No single fixed window can be optimal across the entire range of robot counts.

ST-RRT* avoids this issue by allowing the time bound to shrink or grow on demand. It combines the makespan quality of the wider window at low robot counts with the sampling density of the narrower window at high robot counts, outperforming both across the full range. The adaptive bound comes at a modest computation cost, since additional work is required to determine the bounds at runtime.

The success rate provides further evidence for the inflexibility of the fixed time window. Both RRT* variants exhaust the full calculation budget of 1000 seconds on unsuccessful runs and still fail to find a solution. This suggests that failures are not due to an insufficient sample count, but rather to the time bound itself being infeasible for the given scenario, which a fixed window planner cannot address at runtime.

6 Discussion

This chapter will interpret and discuss the work as a whole. The first section will evaluate the results from Ch. 5. The second section will consider alternative approaches that could have been taken, both as competing methods and variations within the algorithm. The third section consolidates the insights gained across scenarios.

6.1 Results

The approach from Ch. 4 is evaluated on the six benchmark scenarios in Ch. 5, depicting a variety of different assemblies ranging from synthetic to industrial disassemblies. These scenarios were designed to identify and test individual bottlenecks, both individually and in combination with each other.

The achieved makespan results by the algorithm across all scenarios range from a fourfold to a sevenfold improvement compared to the one-robot baseline.

The wall reaches a sevenfold improvement and serves as an upper bound for all other problems, displaying the expected slowdown due to the shared workspace in the absence of strong geometric or spatial restrictions imposed by the scenario. The wall's speedup sits in stark contrast to the speedup of the crate with 4.3. Although both scenarios provide enough objects to allow for a high degree of parallelism, the crate scenario has the worst speedup, showcasing the importance of spatial congestion during disassembly.

The battery scenario, with its 49 components, is the most realistic parallel disassembly. With a sixfold improvement, it sits in the middle of the previous scenarios. The lower speedup from the battery compared to the wall scenario is attributed to the introduction of the pull phase, which fails more often with an increase in the number of robots. Compared with the crate scenario, the battery does not suffer as much from spatial congestion due to the larger physical size of the battery leading to a larger workspace.

Despite having a strictly sequential dependency chain, the tower achieves a fivefold improvement. This demonstrates the effectiveness of temporal constraint propagation with a deep dependency chain. Furthermore, it provides a lower bound on the degradation that can occur when a mixture of sequential and parallel dependencies is poorly planned.

This is evident in the motor and gearbox scenario, where the speedups of 4.6 and 5.0 do not exceed the purely sequential performance of the tower. However, the increase in geometric complexity of the gearbox does not slow down the algorithm beyond the sequential case. In contrast, the motor has a worse speedup despite being geometrically simpler than the gearbox. This is likely due to a lack of coordination when removing the bolts, leading to a behavior similar to that observed in the crate example.

The computation time across all scenarios increases with the number of robots. While the median of the computation time typically grows somewhat linearly with the number of robots, it suffers from strong fluctuations across the board. This is a stark contrast to the small deviation in the makespan, which has little spread throughout all scenarios and number of robots. It only starts to widen its interquartile range for the motor and crate scenario with larger numbers of robots.

While a portion of the fluctuation in the computation time can be attributed to the small sample count, the dominant cause for them is the flaky implementation and therefore high failure of the place phase.

The failure analysis does show that every other phase increases its likelihood to fail with an increasing number of robots. This is the expected behavior, since the workspace gets increasingly more crowded and difficult to plan. Solely the place phase retains a high failure rate across all scenarios in the failure analysis. This is especially alarming for low robot counts, which should not struggle to find a place trajectory due to the lack of obstruction from other robots.

Furthermore, a failure in the place phase is quite costly since it often invalidates previously planned trajectories from the pick and pull phases. Consequently, requiring them to also be replanned. With the tested planning time of ten seconds, a failure results in approximately 20 seconds of wasted computation time each time. With an unreliable place phase, failing due to random chance and a high cost for failure, the computation time varies quite drastically without impacting the makespan.

The comparison of vanilla RRT* [17] and ST-RRT* [28] shows a clear advantage in solving with a planner, which was designed to plan directly in space-time. RRT* can produce similar quality solutions to ST-RRT*, but has to be tuned for the scenario and number of robots, which is infeasible in practice. Furthermore, it

does not solve the problem reliably due to the lack of flexibility in the time domain. Adapting space-time onto a standard planner does not result in competitive solutions.

6.2 Limitations and Alternative Approaches

There are several alternative approaches that could have been taken at different points in the design.

The closest existing work to this is Hartmann et al. [53], which targets multi-robot task and motion planning work for large-scale construction. Although the application domain differs from this work, both approaches solve a similar subset of problems. Hartmann et al. introduce ST-RRT* [28], which is used extensively in this work. However, it uses LGP [48] to formalize the TAMP problem, rather than to rely on a precedence graph. LGP is primarily used for tool usage and sequential manipulation [50] and it is still an open question how well it performs in disassembly work in particular. The precedence graph in this work was inspired by the work of Tian et al. [11], which was designed to work directly on assembled meshes to find a path through physical simulation. It is important to note, that the generation of the precedence graph from the assembly was not included in the calculation time in this work.

The makespan speedup performance of both approaches is comparable. This work seemingly outperforms the makespan speedup slightly in the tower and wall scenario, which both works share. However, this comes at the cost of calculation time, which is drastically higher due to a lack of hyperparameter tuning per scenario and no early convergence termination.

While this work’s approach can compete in terms of makespan, the LGP integration allows for greater flexibility thanks to its symbolic abstraction capabilities. This makes it easier to integrate tool usage or handover sequences, which the current framework does not support. Augmenting the current framework with LGP to include these capabilities is a suitable area for future work.

The most consequential alternative within the prioritized framework itself would be a depth-aware assignment policy. Currently, the framework assigns the next available object to a robot without considering the nodes currently executing in the dependency graph. This would balance the assembly more evenly throughout the workspace. In the motor and gearbox scenario, a better assignment policy could enable disassembly from two sides rather than relying on pure chance. This would prevent dependency chains with long pick phases, which slow down the

makespan. One approach could be to consider the depth in the dependency graph, the movement of other robots around the object or the available workspace into account.

Further alternatives concern the extraction itself. The follow-path decomposition described in Sec. 14 plans each pull as a constrained motion scaled in time to satisfy the velocity bounds. The pull phase does not consider the proximity of obstacles or the configurations of other robots planned earlier or later. A more sophisticated approach incorporating a clearance cost would likely reduce pull failure. Incorporating avoidance of critical areas for subsequent tasks could also improve performance, since the robot would proactively avoid obstructions for future tasks while solving for the current pull. This would facilitate the extraction of objects in congested areas.

As this would represent a significant departure from the current implementation, a more modest improvement would be to plan the potential pick-and-place configurations of objects in close proximity or dependency depth together. For example, this could apply to the screws of the motor or the gears of the gearbox. This could also be extended to problems such as the battery or crate, where the IK solver would try to fit as many manipulators as possible into the workspace simultaneously before dispatching to the planning phase. Online updates should be implemented when new trajectories are planned to maximize the availability of good approach configurations.

Another bottleneck is the division into three different planning problems. The algorithm treats each problem separately and is unable to apply an overarching planning objective. This can result in an unfavorable starting position in subsequent planning problems. The planner should be able to sacrifice local optimality for global optimality.

Another approach would be to incorporate centralized planning methods, such as dRRT* [34], which can handle high-dimensional planning problems. However, solving the complete problem in a combined state space of up to ten mobile manipulators, resulting in up to 100 degrees of freedom, is presumably still infeasible. Decomposing coordination-heavy tasks into a subproblem in a combined state space might be feasible, while guaranteeing optimality. The dRRT* algorithm has been shown to efficiently handle four seven degree-of-freedom manipulators in a shared workspace.

The most consequential improvements, without requiring too many changes, would be to design a better assignment policy, a more advanced path-following algorithm and a smarter pick configuration generation algorithm. The depth-agnostic assignment was a deliberate choice at the start of the project, intended to avoid repetitive

robot-object assignment using simple heuristics such as Euclidean distance on failure. The path-following approach was deliberately kept simple, since it was anticipated that the movement would be a small with not too much movement needed. However, the workspace quickly becomes congested with an increasing number of robots, resulting in more collisions than expected. Failing to consider better grasp selection was an oversight, as problems often require not only optimization of the current trajectory, but also an awareness of the total disassembly.

6.3 Insights

The experimental work reveals several insights about the proposed algorithm and disassembly planning in general.

Speedup is governed by the dominant bottleneck. The ceiling of the speedup across the six benchmarks correlates strongly with the structural properties of the scenario.

Scenarios with shallow, wide dependency trees and unconfined workspaces approach linear speedup. However, even in these workspaces, achieving an ideal linear speedup for a large number of robots is not possible due to the overhead of avoiding collisions. This disparity between real and ideal speedups will always exist for every disassembly due to the nature of the problem. Scenarios with deep, varied dependency chains can easily deteriorate to a sequential speedup if the removal of the object is poorly coordinated. This is due to the obstruction of parallel removals by a previously planned trajectory. Scenarios with tight spatial restrictions reach a saturation point at the number of robots that can physically operate in the contested region.

Disassembly algorithms cannot be evaluated in isolation from the scenario structure. Speedup figures can be misleading without their context. Therefore, a collection of scenarios should therefore be used to evaluate the different performances. While there are general benchmarks for task and motion planning problems [76], no universal dataset and uniform comparison suite is currently available for disassembly work. A good starting point for a collection of problems is Tian et al. [11] and its parent dataset [77]. For contact-rich assemblies, Sliwowski et al. [78] provide a dataset based around the NIST Assembly Task Board 1 [79].

Depth-agnostic assignment leads to long approach phases. The assignment policy selects the next available object regardless of the graph’s structure. This has little effect on wide, shallow graphs because all leaves are roughly equivalent. However, on deep, hierarchical trees with mixed branch depths, it can lead to a loss of parallelism. This is the most actionable negative result of the thesis. Making a small change to the assignment heuristic, without changing the underlying planner, would likely improve the performance on scenarios such as the gearbox. A more refined assignment policy could also improve scenarios such as the battery, where a more spread-out assignment might lead to less congestion and further speedup.

Spatial congestion scales with object density, not object count. The achievable speedup is not related to the number of objects in a scene, but rather the spatial congestion caused by the shared workspace. The crate has the slowest speedup due to spatial congestion because all extractions occur in a small region. Despite having five times as many objects, the battery does not have the same slowdown. This is because the objects are distributed across a larger workspace. Therefore, spatial congestion is not a function of object count, but of object density.

Future scenario design should treat density rather than count as the primary scaling variable.

Constrained extraction is more sensitive to upstream planning than to the extraction itself. The high failure rates observed in the pull phase of the crate and motor are not caused by the pull phase being intrinsically difficult. The pull motion itself is short and follows a fixed path. Failures often occur because the grasp configuration was chosen by the pick planner based on path cost rather than favorable downstream conditions for the subsequent task. Improvements to the extraction quality would therefore come from changes to either the extraction algorithm or the decision-making process that selects the configuration for the extraction.

This highlights the need for an algorithm that can optimize the entire pipeline across different phases, rather than optimizing each subsection individually.

7 Conclusion

This thesis presented a prioritized space-time motion planner for multi-robot disassembly. The planner combines ST-RRT* with temporal dependency constraint propagation and an optimization-based extraction for object assemblies. The algorithm uses a pregenerated precedence graph as an input, which is used to determine the order of execution. The graph is also used to pass additional planning information to subsequent planning problems. These constraints impose time bounds on the dynamic time state space expansion of ST-RRT*, thereby enabling temporal coordination between robots.

The approach was evaluated using six assembly scenarios, each of which stressed different characteristics of the algorithm during disassembly. The tower scenario requires deep, sequential temporal constraint propagation. The wall scenario tests scaling for unconstrained, heavily parallizable object access, whereas the crate scenario stresses spatial congestion arising from constrained extraction during simultaneous object access. The motor tests a mixture of sequential and parallel access. The battery scenario is the largest, stressing heavy parallization during constraint extraction. The gearbox is the most geometrically complex scenario, testing spatial congestion and a mixture of sequential and parallel extraction.

The comparison of ST-RRT* and RRT* clearly demonstrated the advantages of the adaptive time bound of ST-RRT*. While the makespan achieved with RRT* can be competitive with extensive hyperparameter tuning, it lacks the flexibility to perform well across the whole range of robot counts. Furthermore, ST-RRT* was able to solve the problem with a 100% success rate across the comparison, whereas both RRT* time window variations only reached around 40%.

The achieved makespan speedup is between 4.3 and 7.0 compared to the one-robot baseline. The tower scenario provides a makespan speedup of 4.9, acting as a lower bound if parallizable scenarios decay into purely sequential ones. This is undercut by the crate scenario makespan speedup of 4.3 and the motor's 4.6, identifying significant bottlenecks in the proposed algorithm.

The lower makespan speedup in these scenarios is attributed to three main factors. First, the depth-agnostic assignment policy causes parallel problems to degrade to

sequential ones. Second, a lack of coordination during the generation of grasp configurations leads to the blocking of parallel tasks. Third, the current approach to the pull phase struggles to solve problems reliably in congested workspaces. Possible solutions to these bottlenecks include smarter robot-object assignment policies, such as dependency depth-aware assignment, more advanced path following algorithms and dependency graph aware grasp configuration generation.

Despite limitations, this work provides groundwork for general-purpose multi-robot disassembly, providing insight in bottlenecks and key challenges to solve, in order to advance towards a circular economy by automating industries from recycling and upcycling all the way through remanufacturing.

Bibliography

- [1] H. Bruyninckx, “Bend the trend, pathways to a liveable planet as resource use spikes,” *Global Resources Outlook 2024*, 2024.
- [2] Circle Economy Foundation, “The circularity gap report 2024, A circular economy to live within the safe limits of the planet,” Circle Economy Foundation, Amsterdam, Report, 2024. Accessed: May 28, 2026. [Online]. Available: <https://www.circularity-gap.world/2024>.
- [3] F. Larsson, P. Andersson, P. Blomqvist, and B.-E. Mellander, “Toxic fluoride gas emissions from lithium-ion battery fires,” *Scientific Reports*, vol. 7, no. 1, p. 10 018, 2017.
- [4] B. Schwaebe, H. He, C. Glaubenskle, O. A. Ogunseitan, and J. M. Schoenung, “Chemical hazard assessment toward safer electrolytes for lithium-ion batteries,” *Integrated Environmental Assessment and Management*, vol. 20, no. 6, pp. 2231–2244, Nov. 2024.
- [5] X. Feng, M. Ouyang, X. Liu, L. Lu, Y. Xia, and X. He, “Thermal runaway mechanism of lithium ion battery for electric vehicles: A review,” *Energy storage materials*, vol. 10, pp. 246–267, 2018.
- [6] K. Grant et al., “Health consequences of exposure to e-waste: A systematic review,” *The Lancet Global Health*, vol. 1, no. 6, pp. 350–361, 2013.
- [7] O. Causa, M. Abendschein, N. Luu, E. Soldani, and C. Soriolo, “The post-covid-19 rise in labour shortages,” 2022.
- [8] G. Formentini et al., “A review of disassembly systems for circular product design,” no. 506, 2025.
- [9] G. Foo, S. Kara, and M. Pagnucco, “Challenges of robotic disassembly in practice,” *Procedia CIRP*, vol. 105, pp. 513–518, 2022.
- [10] E. Kongar and S. M. Gupta, “Disassembly sequencing using genetic algorithm,” *The International Journal of Advanced Manufacturing Technology*, vol. 30, no. 5-6, pp. 497–506, 2006.
- [11] Y. Tian et al., “Assemble them all: Physics-based planning for generalizable assembly by disassembly,” *ACM Transactions on Graphics*, vol. 41, no. 6, 2022.

- [12] G. Boothroyd, P. Dewhurst, and W. A. Knight, *Product design for manufacture and assembly*. CRC press, 2010.
- [13] S. Smith, G. Smith, and W.-H. Chen, “Disassembly sequence structure graphs: An optimal approach for multiple-target selective disassembly sequence planning,” *Advanced Engineering Informatics*, vol. 26, no. 2, pp. 306–316, 2012.
- [14] L. E. Kavraki, P. Švestka, J.-C. Latombe, and M. H. Overmars, “Probabilistic roadmaps for path planning in high-dimensional configuration spaces,” *IEEE Transactions on Robotics and Automation*, vol. 12, no. 4, pp. 566–580, 1996.
- [15] S. M. LaValle, “Rapidly-exploring random trees: A new tool for path planning,” Iowa State University, Tech. Rep. TR 98-11, 1998.
- [16] J. J. Kuffner and S. M. LaValle, “Rrt-connect: An efficient approach to single-query path planning,” in *Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings (Cat. No.00CH37065)*, vol. 2, 2000, 995–1001 vol.2.
- [17] S. Karaman and E. Frazzoli, “Sampling-based algorithms for optimal motion planning,” *The International Journal of Robotics Research*, vol. 30, no. 7, pp. 846–894, 2011.
- [18] J. D. Gammell, S. S. Srinivasa, and T. D. Barfoot, “Informed rrt*: Optimal sampling-based path planning focused via direct sampling of an admissible ellipsoidal heuristic,” in *2014 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2014, pp. 2997–3004.
- [19] J. D. Gammell, S. S. Srinivasa, and T. D. Barfoot, “Batch informed trees (bit*): Sampling-based optimal planning via the heuristically guided search of implicit random geometric graphs,” in *2015 IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, May 2015, pp. 3067–3074.
- [20] S. M. LaValle and J. J. Kuffner, “Randomized kinodynamic planning,” *The International Journal of Robotics Research*, vol. 20, no. 5, pp. 378–400, 2001.
- [21] D. J. Webb and J. van den Berg, “Kinodynamic RRT*: Asymptotically optimal motion planning for robots with linear dynamics,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, Karlsruhe, Germany, 2013, pp. 5054–5061.
- [22] Y. Li, Z. Littlefield, and K. E. Bekris, “Asymptotically optimal sampling-based kinodynamic planning,” *The International Journal of Robotics Research*, vol. 35, no. 5, pp. 528–564, 2016.

- [23] D. Hsu, R. Kindel, J.-C. Latombe, and S. Rock, “Randomized kinodynamic motion planning with moving obstacles,” *The International Journal of Robotics Research*, vol. 21, no. 3, pp. 233–255, 2002.
- [24] J. van den Berg and M. H. Overmars, “Roadmap-based motion planning in dynamic environments,” *IEEE Transactions on Robotics*, vol. 21, no. 5, pp. 885–897, 2005.
- [25] J. van den Berg and M. H. Overmars, “Planning time-minimal safe paths amidst unpredictably moving obstacles,” *The International Journal of Robotics Research*, vol. 27, no. 11-12, pp. 1274–1294, 2008.
- [26] M. Phillips and M. Likhachev, “SIPP: Safe interval path planning for dynamic environments,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2011, pp. 5628–5635.
- [27] A. Sintov and A. Shapiro, “Time-based RRT algorithm for rendezvous planning of two dynamic systems,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2014, pp. 6745–6750.
- [28] F. Grothe, V. N. Hartmann, A. Orthey, and M. Toussaint, “ST-RRT*: Asymptotically-optimal bidirectional motion planning through space-time,” in *2022 International Conference on Robotics and Automation (ICRA)*, 2022, pp. 3314–3320.
- [29] J. Yu and S. M. LaValle, “Structure and intractability of optimal multi-robot path planning on graphs,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 27, 2013, pp. 1443–1449.
- [30] G. Sharon, R. Stern, A. Felner, and N. R. Sturtevant, “Conflict-based search for optimal multi-agent pathfinding,” *Artificial Intelligence*, vol. 219, pp. 40–66, 2015.
- [31] A. Andreychuk, K. Yakovlev, P. Surynek, D. Atzmon, and R. Stern, “Multi-agent pathfinding with continuous time,” *Artificial Intelligence*, vol. 305, p. 103 662, 2022.
- [32] G. Wagner and H. Choset, “M*: A complete multirobot path planning algorithm with performance bounds,” in *2011 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2011, pp. 3260–3267.
- [33] K. Solovey, O. Salzman, and D. Halperin, “Finding a needle in an exponential haystack: Discrete RRT for exploration of implicit roadmaps in multi-robot motion planning,” *The International Journal of Robotics Research*, vol. 35, no. 5, pp. 501–513, 2016.

- [34] R. Shome, K. Solovey, A. Dobson, D. Halperin, and K. E. Bekris, “dRRT*: Scalable and informed asymptotically-optimal multi-robot motion planning,” *Autonomous Robots*, vol. 44, no. 3–4, pp. 443–467, 2020.
- [35] M. Erdmann and T. Lozano-Pérez, “On multiple moving objects,” *Algorithmica*, vol. 2, no. 1, pp. 477–521, 1987.
- [36] M. Bennewitz, W. Burgard, and S. Thrun, “Finding and optimizing solvable priority schemes for decoupled path planning techniques for teams of mobile robots,” *Robotics and Autonomous Systems*, vol. 41, no. 2-3, pp. 89–99, 2002.
- [37] M. Čáp, P. Novák, A. Kleiner, and M. Selecký, “Prioritized planning algorithms for trajectory coordination of multiple mobile robots,” *IEEE Transactions on Automation Science and Engineering*, vol. 12, no. 3, pp. 835–849, 2015.
- [38] H. Ma, D. Harabor, P. J. Stuckey, J. Li, and S. Koenig, “Searching with consistent prioritization for multi-agent path finding,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, 2019, pp. 7643–7650.
- [39] C. R. Garrett et al., “Integrated task and motion planning,” *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 4, pp. 265–293, 2021.
- [40] L. P. Kaelbling and T. Lozano-Perez, “Hierarchical task and motion planning in the now,” in *2011 IEEE International Conference on Robotics and Automation*, 2011, pp. 1470–1477.
- [41] S. Srivastava, E. Fang, L. Riano, R. Chitnis, S. Russell, and P. Abbeel, “Combined task and motion planning through an extensible planner-independent interface layer,” in *2014 IEEE International Conference on Robotics and Automation (ICRA)*, 2014, pp. 639–646.
- [42] C. R. Garrett, T. Lozano-Pérez, and L. P. Kaelbling, “Pddlstream: Integrating symbolic planners and blackbox samplers via optimistic adaptive planning,” in *Proceedings of the international conference on automated planning and scheduling*, vol. 30, 2020, pp. 440–448.
- [43] M. Ghallab et al., “Pddl - the planning domain definition language,” Yale Center for Computational Vision and Control, Tech. Rep. CVC TR-98-003/DCS TR-1165, Aug. 1998.
- [44] T. Siméon, J.-P. Laumond, J. Cortés, and A. Sahbani, “Manipulation planning with probabilistic roadmaps,” *The International Journal of Robotics Research*, vol. 23, no. 7-8, pp. 729–746, 2004.
- [45] K. Hauser and J.-C. Latombe, “Multi-modal motion planning in non-expansive spaces,” *The International Journal of Robotics Research*, vol. 29, no. 7, pp. 897–915, 2010.

- [46] A. Krontiris and K. E. Bekris, “Dealing with difficult instances of object re-arrangement,” in *Proceedings of Robotics: Science and Systems (RSS)*, 2015.
- [47] W. Vega-Brown and N. Roy, “Asymptotically optimal planning under piecewise-analytic constraints,” in *Algorithmic Foundations of Robotics XII (WAFR 2016)*, ser. Springer Proceedings in Advanced Robotics, vol. 13, Springer, 2020, pp. 528–543.
- [48] M. Toussaint, “Logic-geometric programming: An optimization-based approach to combined task and motion planning,” in *Proceedings of the 24th International Conference on Artificial Intelligence*, ser. IJCAI’15, Buenos Aires, Argentina: AAAI Press, 2015, pp. 1930–1936, ISBN: 9781577357384.
- [49] M. Toussaint, “A tutorial on Newton methods for constrained trajectory optimization and relations to SLAM, Gaussian process smoothing, optimal control, and probabilistic inference,” in *Geometric and Numerical Foundations of Movements*, ser. Springer Tracts in Advanced Robotics, vol. 117, Springer, 2017, pp. 361–392.
- [50] M. Toussaint, K. R. Allen, K. A. Smith, and J. B. Tenenbaum, “Differentiable physics and stable modes for tool-use and manipulation planning,” in *Proceedings of Robotics: Science and Systems (RSS)*, 2018.
- [51] T. Pan, A. M. Wells, R. Shome, and L. E. Kavraki, “A general task and motion planning framework for multiple manipulators,” in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2021, pp. 3168–3174.
- [52] M. Toussaint and M. Lopes, “Multi-bound tree search for logic-geometric programming in cooperative manipulation domains,” in *2017 IEEE International Conference on Robotics and Automation (ICRA)*, 2017, pp. 4044–4051.
- [53] V. N. Hartmann, A. Orthey, D. Driess, O. S. Oguz, and M. Toussaint, “Long-horizon multi-robot rearrangement planning for construction assembly,” *IEEE Transactions on Robotics*, vol. 39, no. 1, pp. 239–252, 2023.
- [54] V. N. Hartmann and M. Toussaint, “Towards computing low-makespan solutions for multi-arm multi-task planning problems,” in *ICAPS Workshop on Planning and Robotics (PlanRob)*, 2023. arXiv: [2305.17527 \[cs.R0\]](#).
- [55] J. Chen et al., “Cooperative task and motion planning for multi-arm assembly systems,” *arXiv preprint arXiv:2203.02475*, 2022. arXiv: [2203.02475 \[cs.R0\]](#).
- [56] A. Lambert, “Disassembly sequencing: A survey,” *International Journal of Production Research - INT J PROD RES*, vol. 41, pp. 3721–3759, Nov. 2003.

- [57] B. Lazzerini and F. Marcelloni, “A genetic algorithm for generating optimal assembly plans,” *Artificial Intelligence in Engineering*, vol. 14, no. 4, pp. 319–329, 2000.
- [58] M. Kheder, M. Trigui, and N. Aifaoui, “Disassembly sequence planning based on a genetic algorithm,” *Proceedings of the Institution of Mechanical Engineers, Part C: Journal of Mechanical Engineering Science*, vol. 229, Nov. 2014.
- [59] H. Poschmann, H. Brüggemann, and D. Goldmann, “Fostering end-of-life utilization by information-driven robotic disassembly,” *Procedia CIRP*, vol. 98, pp. 282–287, 2021.
- [60] B. Siciliano and O. Khatib, *Springer Handbook of Robotics*. Springer, 2008, ISBN: 978-3-540-23957-4.
- [61] K. M. Lynch and F. C. Park, *Modern robotics*. Cambridge University Press, 2017.
- [62] E. G. Gilbert, D. W. Johnson, and S. S. Keerthi, “A fast procedure for computing the distance between complex objects in three-dimensional space,” *IEEE Journal on Robotics and Automation*, vol. 4, no. 2, pp. 193–203, 1988.
- [63] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, pp. 269–271, 1959.
- [64] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [65] K. Solovey, L. Janson, E. Schmerling, E. Frazzoli, and M. Pavone, “Revisiting the asymptotic optimality of RRT,” in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, 2020, pp. 2189–2195.
- [66] S. M. LaValle, *Planning Algorithms*. Cambridge, U.K.: Cambridge University Press, 2006.
- [67] R. Geraerts and M. H. Overmars, “Creating high-quality paths for motion planning,” *The International Journal of Robotics Research*, vol. 26, no. 8, pp. 845–863, 2007.
- [68] C. de Boor, *A Practical Guide to Splines* (Applied Mathematical Sciences). New York: Springer, 1978, vol. 27.
- [69] J. Shintake, V. Cacucciolo, D. Floreano, and H. Shea, “Soft robotic grippers,” *Advanced Materials*, vol. 30, p. 1707035, May 2018.
- [70] Z. Samadikhoshkho, K. Zareinia, and F. Janabi-Sharifi, “A brief review on robotic grippers classifications,” in *2019 IEEE Canadian Conference of Electrical and Computer Engineering (CCECE)*, 2019, pp. 1–4.

- [71] J. Hernandez et al., “Current designs of robotic arm grippers: A comprehensive systematic review,” *Robotics*, vol. 12, no. 1, 2023.
- [72] J. F. Canny, *The Complexity of Robot Motion Planning*. Cambridge, MA: MIT Press, 1988.
- [73] J. Lee et al., “DART: Dynamic animation and robotics toolkit,” *Journal of Open Source Software*, vol. 3, no. 22, p. 500, 2018.
- [74] I. A. Şucan, M. Moll, and L. E. Kavraki, “The Open Motion Planning Library,” *IEEE Robotics & Automation Magazine*, vol. 19, no. 4, pp. 72–82, Dec. 2012.
- [75] *Grabcad, Planetary gearbox*, <https://grabcad.com/library/planetary-gearbox>, Accessed: 2026-03-04, 2011.
- [76] C. Chamzas et al., “Motionbenchmaker: A tool to generate and benchmark motion planning datasets,” *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 882–889, Apr. 2022.
- [77] K. D. D. Willis et al., “JoinABLE: Learning bottom-up assembly of parametric CAD joints,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 15 828–15 839.
- [78] D. Sliwowski, S. Jadav, S. Stanovcic, J. Orbik, J. Heidersberger, and D. Lee, “Demonstrating REASSEMBLE: A multimodal dataset for contact-rich robotic assembly and disassembly,” in *Proceedings of Robotics: Science and Systems*, Los Angeles, USA, Jun. 2025.
- [79] K. Kimble, J. Albrecht, M. Zimmerman, and J. A. Falco, “Performance measures to benchmark the grasping, manipulation and assembly of deformable objects typical to manufacturing applications,” *Frontiers in Robotics and AI*, Nov. 2022.

A Disassembly Sequences

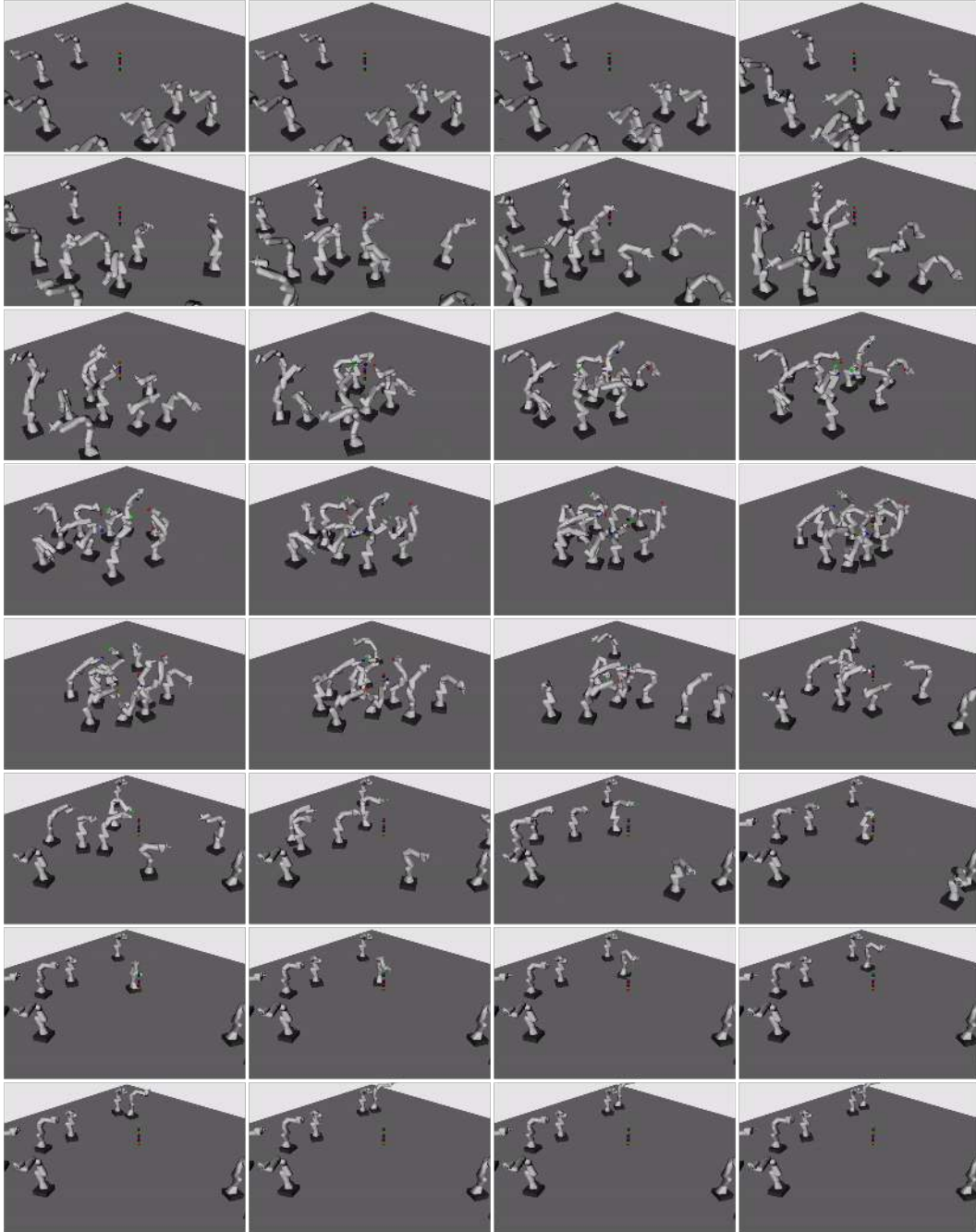


Figure A.1: Tower disassembly sequence with ten robots

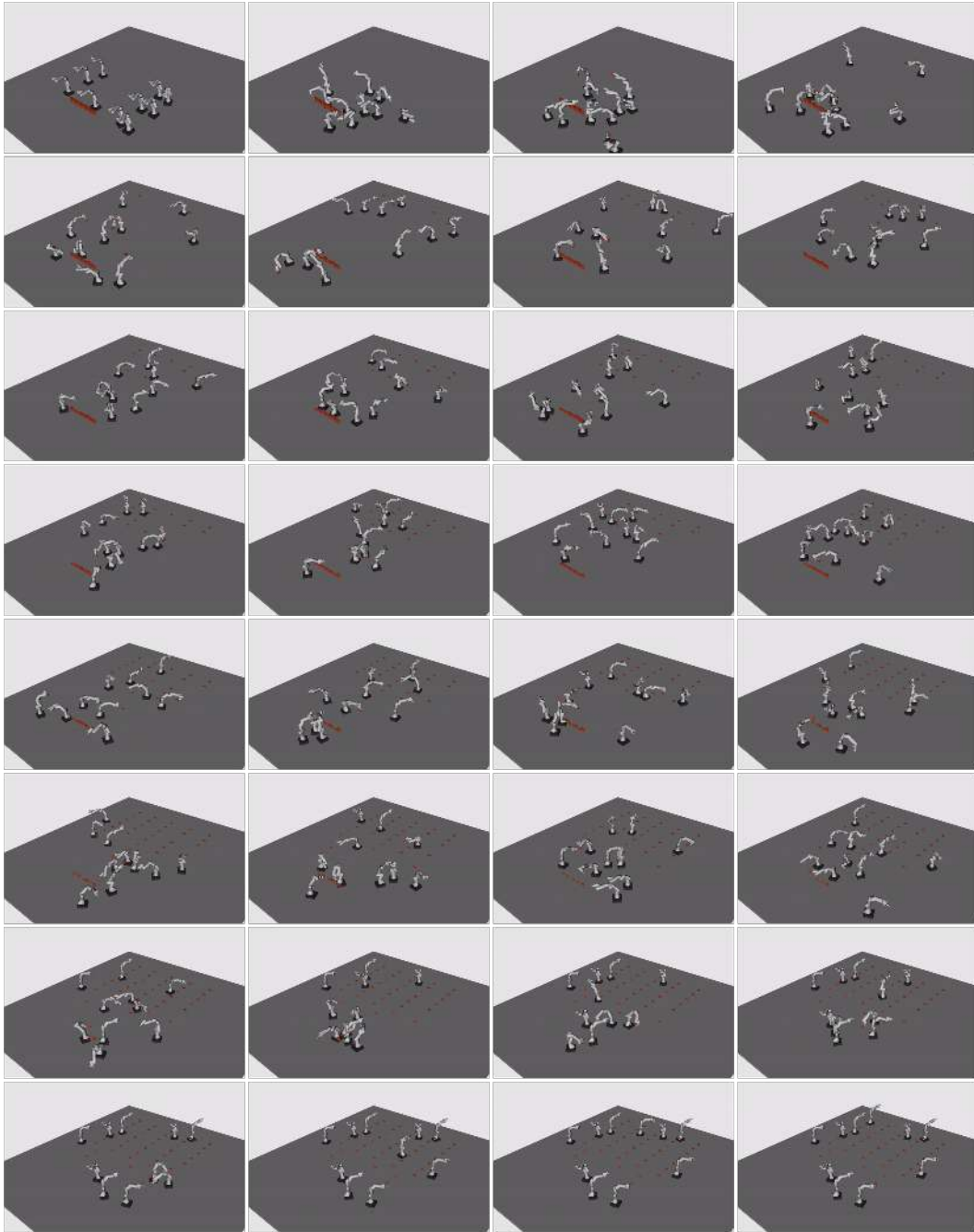


Figure A.2: Wall disassembly sequence ten robots

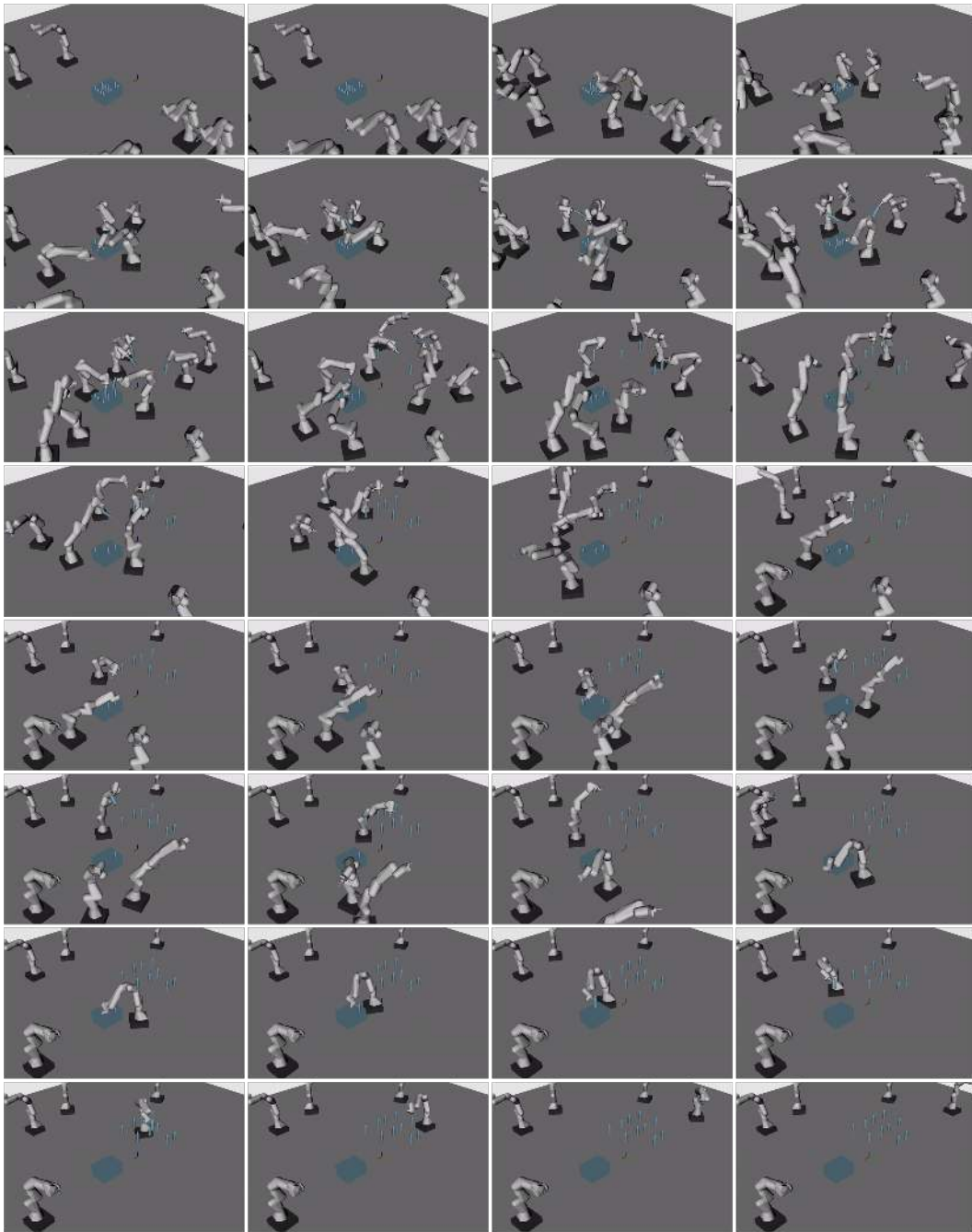


Figure A.3: Crate disassembly sequence with ten robots

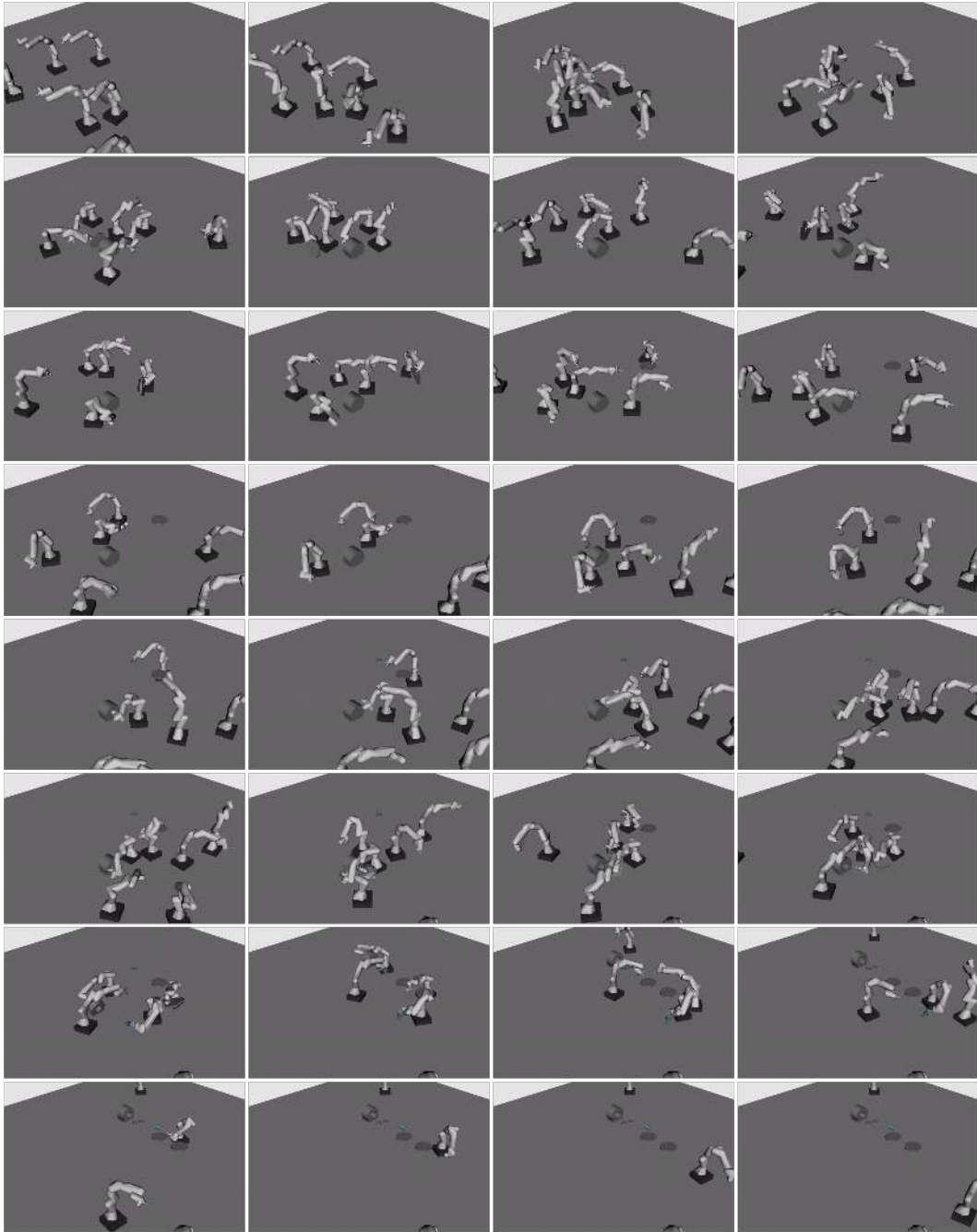


Figure A.4: Motor disassembly sequence with six robots

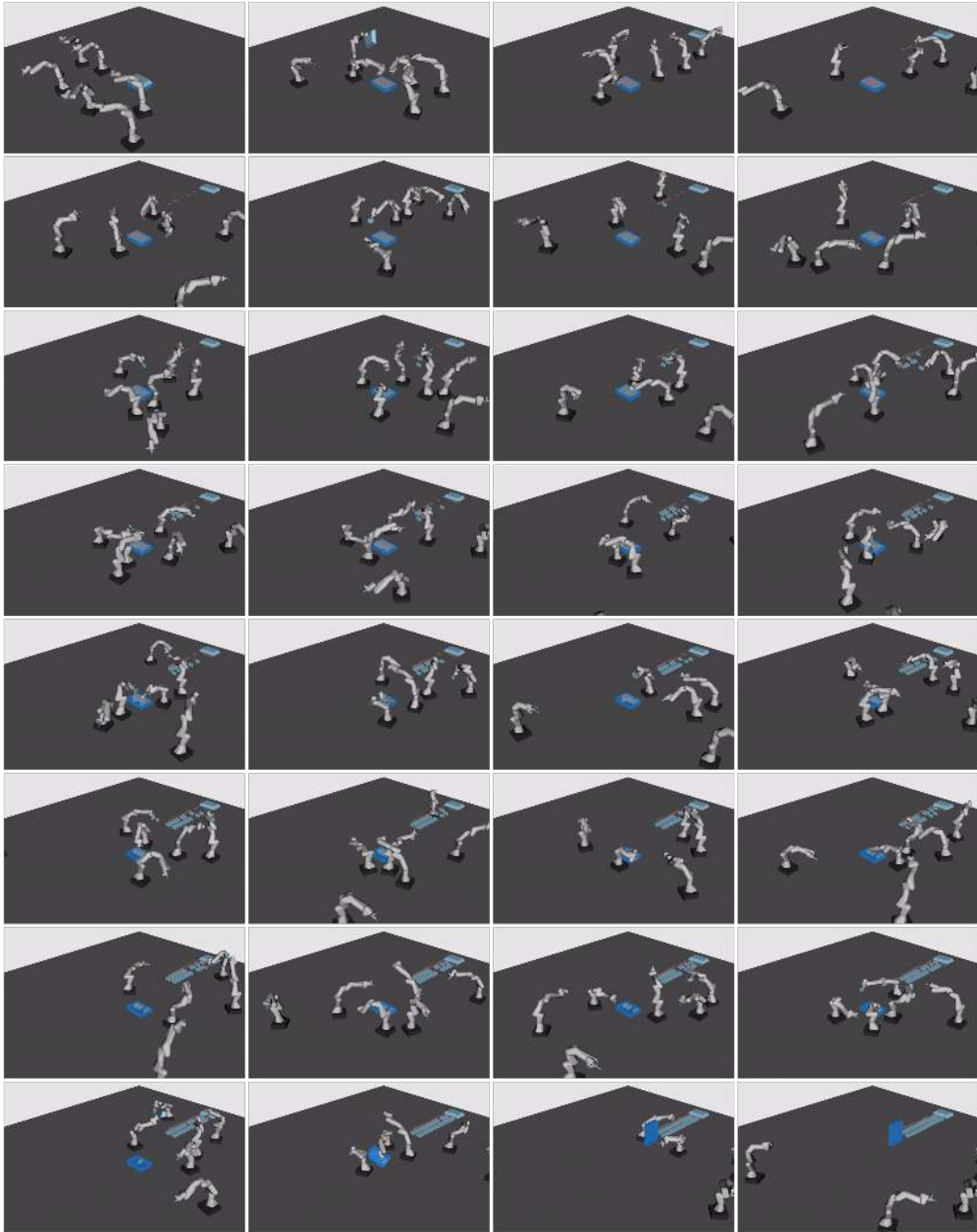


Figure A.5: Battery disassembly sequence with six robots

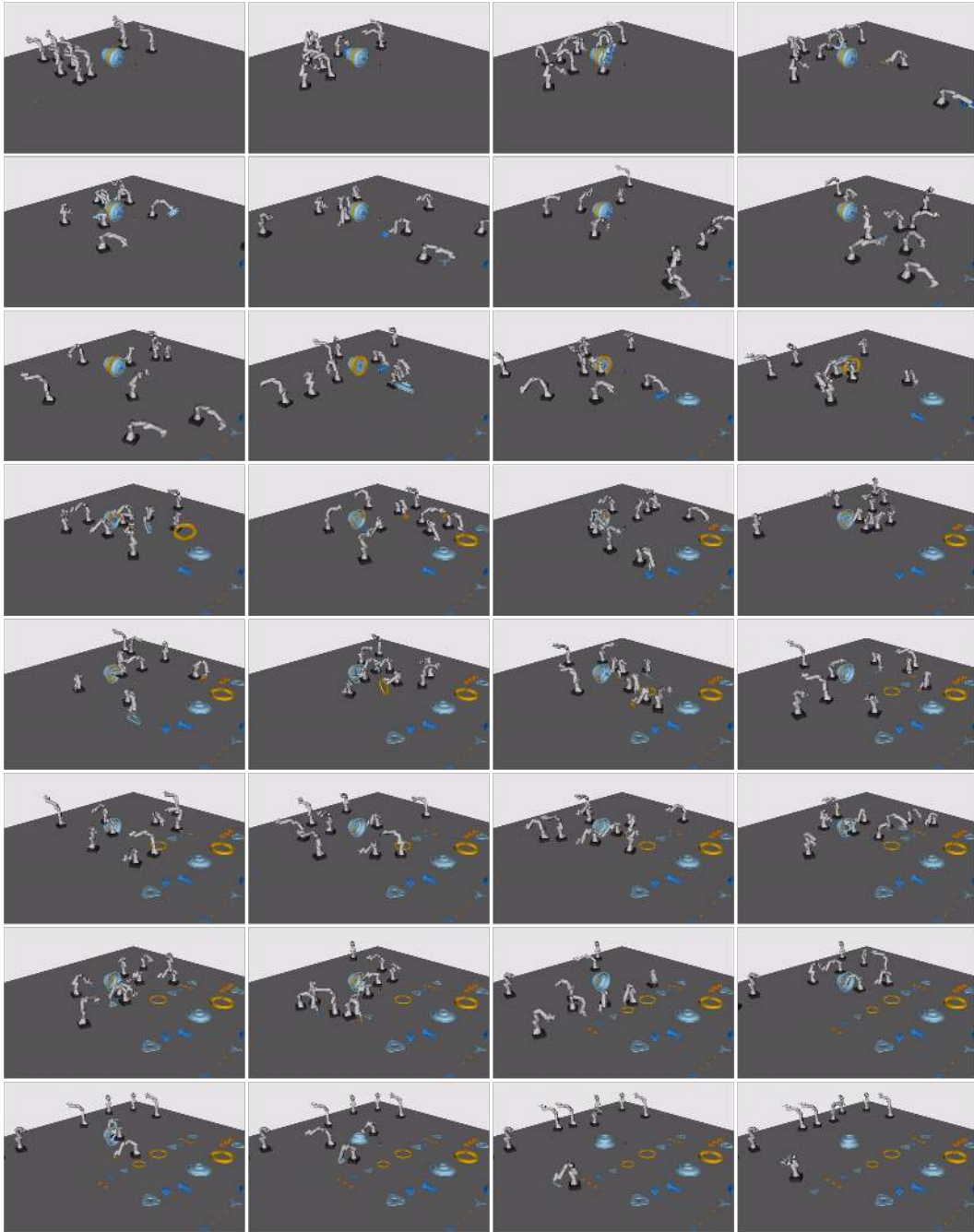


Figure A.6: Gearbox disassembly sequence with nine robots